

EE6427

导论

图像视频基础Image & Video Basics

图像

图像是一种在信息存储、处理与呈现中重要的媒体类型。

图像由一个二维像素矩阵（行和列）表示。

常见的图像格式包括：

JPEG (Joint Photographic Expert Group)

TIFF (Tagged Image File Format)

BMP (Bitmap) 等。

像素与分辨率

Pixel (图像元素)：数字图像中的最小单位。

Resolution (分辨率)：图像中的像素总数。

可以用总像素数、宽度 x 高度或每英寸像素数 (ppi) 表示。

例如：5 百万像素、640 x 480 像素，或 300 ppi。

用于衡量区分图像精细细节的能力。

位深度/颜色深度

Bit depth / Color depth (位深度/颜色深度)：每个像素所用的比特数。

二值图像 (1 bit)，灰度图像 (8 bits)，彩色图像 (24 bits)。

颜色

彩色图像可以用不同的色彩空间来表达。

常见的色彩空间：RGB, HSI, YCbCr 等。

RGB

彩色图像可表示为红(R)、绿(G)、蓝(B)三个分量的三元组。

RGB 色彩模型图示及立方体模型展示了三原色及其混合效果。

24-bit color image: 每个像素包含三个分量 —— RGB (Red, Green, Blue)。

每个分量使用 8 位，取值范围为 0 到 255。

黑色为 (0, 0, 0)，白色为 (255, 255, 255)。

YUV, YIQ, YCbCr

这些是 YUV 类型模型的变体，如 YIQ 和 YCbCr。

Y (亮度) 代表视频信号的明暗部分，**UV** (色度) 代表色彩部分。

YUV 适用于电视广播，因为它能有效利用带宽。

应用于电视工业：

YUV (PAL, SECAM), YIQ (NTSC), YC_bCr (JPEG, MPEG)

数码相机信号处理

数码相机信号处理流程图：

光线通过镜头照射到 CCD 传感器，经 AD 转换器转换为数字数据。

数字数据送入图像处理引擎进行处理，然后压缩并存储到存储卡。

影响数码图像质量的因素：

镜头 (Lens)

CCD 传感器性能

图像处理引擎性能

除了标准 CMOS/CCD 传感器外，还有其他类型的相机传感器：

RGB-D Cameras (深度感知相机)

Near Infrared Cameras (近红外相机)

Thermal Cameras (热成像相机)

Event-based Cameras (事件驱动相机)

等等。

视频

视频是由一系列帧/图像/画面组成的，并附带音频信号。

以高速率播放，给人以流畅运动的视觉感受。

依赖于人类视觉的持久性。

常见的视频格式：

MPEG (Moving Pictures Expert Group)

AVI (Intel)

WMV (Microsoft)

结构

Shot (镜头)：所有视频都由多个镜头组成。

Frame (帧)：视频中的每一幅单独的画面。

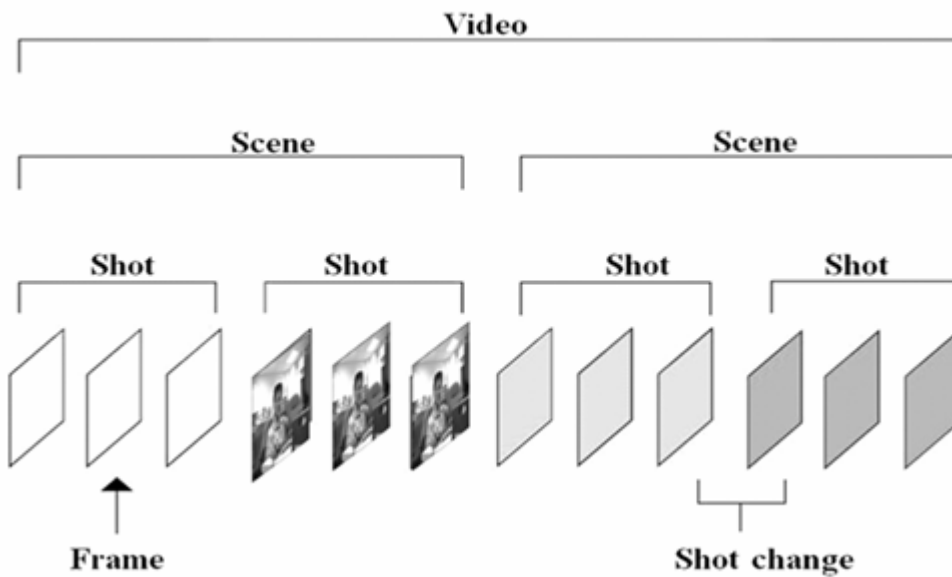
Composition (构图)：画面内容的布局。

VWS (Very Wide Shot)：展示主体的环境。

WS (Wide Shot)：展示整个主体。

CU (Close Up)：展示主体的某个特征。

视频结构层级图：



Frame rate (帧率): 每秒的帧数。

Spatial resolution (空间分辨率): 每帧图像中的像素数量。

Aspect ratio (宽高比): 图像宽度与高度的比例。

Color resolution (色彩分辨率): 帧的位深度。

Video data rate & file size (视频数据速率与文件大小)。

分辨率

标准清晰度 (SD):

NTSC (480i): 720×480 , 640×480

PAL (576i): 720×576

高清 (HD):

720p: 1280×720 progressive

1080i: 1920×1080 interlaced

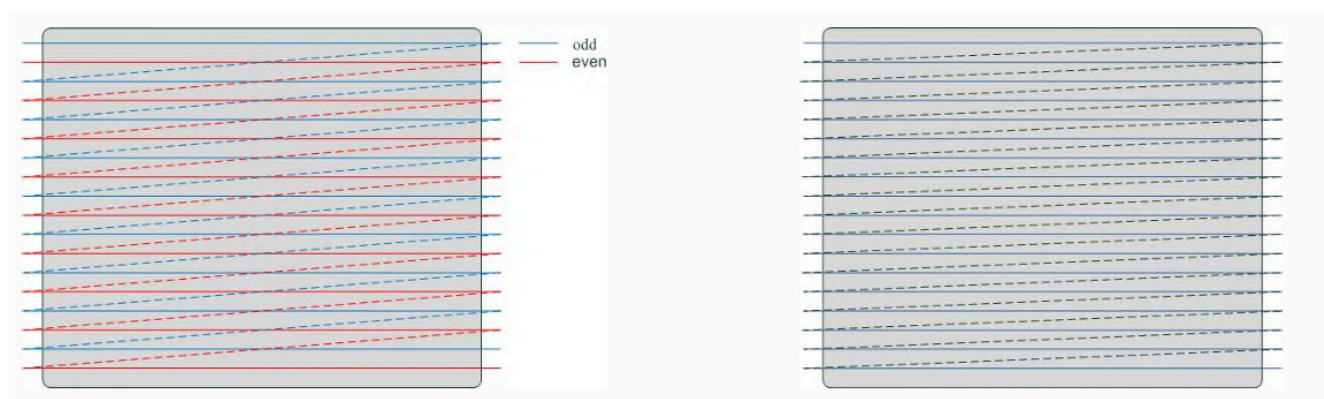
1080p: 1920×1080 progressive (Full HD / FHD)

超高清 (UHD):

4K ~ 4 倍 FHD

8K ~ 4 倍 4K

扫描



Interlaced (隔行扫描):

将图像分为两个场（奇数行和偶数行），交替扫描。

Progressive (逐行扫描):

所有行按顺序依次扫描。

帧率

帧率 (Frame Rate): 每秒播放的帧数，单位为 fps (frames per second)。

标准:

PAL: 25 fps，用于澳大利亚、新加坡等地区。

NTSC: 30 fps，用于美国、日本等地区。

视频信号处理&应用

视频信号处理涉及数字视频信号的不同方面，包括呈现、压缩、存储、传输、处理、分析与理解。

商业:

流媒体电商等。

娱乐:

视频流媒体、点播视频等。

教育:

技术支持的学习、在线讲座等。

智慧城市、交通等:

自动驾驶汽车、视频分析等。

趋势

新的人工智能与深度学习算法。

大数据。

更快的处理器和更大容量的存储设备。

新兴的消费者行为。

更好的压缩算法。

更强大的视频分析技术。

技术挑战

编码与通信:

更高的质量。

更低的比特率。

服务质量。

内容分析与理解:

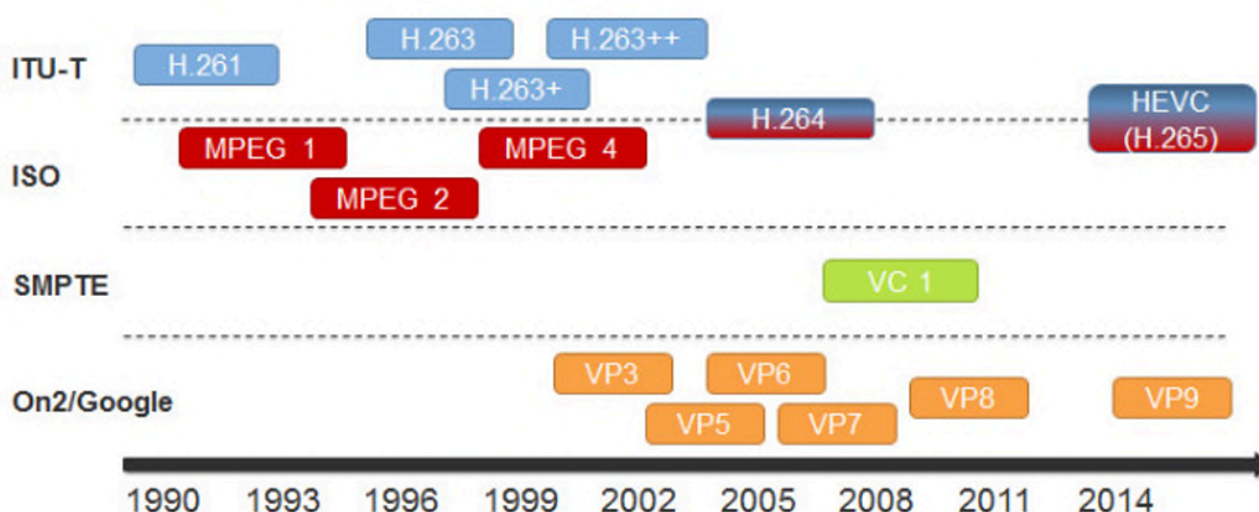
鲁棒性性能。

领域差距/转移/泛化。

昂贵的数据标注。

图像视频标准

A Brief History of Video Codes



JPEG

一种流行的图像压缩标准。

被称为 ISO/IEC 国际标准 10918 或 ITU-T 建议 T.81。

由一个名为“联合图像专家组”(JPEG) 的国际机构开发。

MPEG

MPEG-1 (1992):

视频和音频编码 (CD-ROM, 1.5Mbps)。

相关产品: VCD。

MPEG-2 / H.262 (1994):

具有不同配置文件的视频和音频编码 (2-80Mbps)。

相关产品: DVD。

MPEG-4 Advanced Video Coding (AVC) / H.264:

流行的视频编码标准。

应用: 互联网视频、蓝光光盘、移动设备等。

H.26x

H.261:

由 CCITT 于 1990 年开发。(注: CCITT 后更名为 ITU-T)

基于 DCT 的视频压缩方案。

许多特性与 MPEG-1 视频编码相似。

目标应用: 视频会议。

比特率为 $p \times 64$ Kbps, 其中 p 的取值范围为 1 到 30。

H.263:

ITU-T 在 1995 年推荐的 H.263 v1。

优于 H.261, 是视频会议的标准。

H.263 v2 (H.263+, 1998)

H.263 v3 (H.263++, 2000)

H.264 / Advanced Video Coding (AVC) / MPEG-4 Part 10:

于 2003 年标准化。

压缩效率比 H.263 提高约 30%。

应用：互联网视频、电脑、HDTV 广播、蓝光光盘、移动和便携设备。

H.265 / High Efficiency Video Coding (HEVC):

基本结构类似于 H.264。

支持最高 8K 分辨率和 120 fps。

在比特率-质量权衡上比 H.264 更高效。

其它标准

VC-1:

由电影与电视工程师协会 (SMPTE) 标准化。

由微软 Windows Media Video (WMV) 实现。

VP8:

面向网络视频等环境。

支持 Web 视频格式和最高 4K 分辨率。

VP9:

基本结构类似于 VP8。

支持最高 8K 分辨率和 120 fps。

视频内容分析与理解

执行视频内容分析与理解。

一些重要任务包括：

目标检测与跟踪 (Object Detection & Tracking)

姿态估计 (Pose Estimation)

人体动作识别 (Human Action Recognition)

新兴话题

生成式AI

一类能够生成新内容（如文本、图像、音频、视频等）的 AI 模型。

示例：

文本生成：生成式预训练变换器 (GPT)。

图像生成：Stable Diffusion。

支持多样化的应用，如聊天机器人、内容创作、虚拟助手等。

Foundation Models (FMs)

一类在大规模通用数据上训练的模型，可通过微调 (finetuned) 适应广泛的下游任务/应用。

示例：大型语言模型（如 GPT）、视觉-语言模型（如 CLIP）。

图像压缩

术语&概念

目的是减少表示特定信息所需的总比特数。

通用数据压缩方案流程图：

输入数据 → 编码器 (压缩) → 存储或网络 → 解码器 (解压缩) → 输出数据。

压缩率

一个用于衡量数据被压缩程度的指标。

$$\text{compression ratio} = \frac{B_0}{B_1}$$

B_0 ：压缩前的比特数。

B_1 ：压缩后的比特数。

信息论

信息源 $S = \{s_1, s_2, \dots, s_n\}$ 的熵 η 定义为：

$$\eta = H(S) = \sum_{i=1}^n p_i \log_2 \frac{1}{p_i} \quad (7.2)$$

$$= - \sum_{i=1}^n p_i \log_2 p_i \quad (7.3)$$

p_i ：符号 s_i 在 S 中出现的概率。

熵编码

霍夫曼编码

一种可变长度编码 (VLC)。

高频模式（符号）被分配更短的码字。

表示每个模式/符号所需的平均比特数将减少。

哈夫曼码树是一棵二叉树，其分支被赋予值 0 或 1。

树结构：

根节点 (Root node)：树的起点。

分支节点 (Branch node)：分支分叉的点。

叶节点 (Leaf node)：分支的终点。

在每个分支节点，0 和 1 被分别分配给左分支和右分支。

每个字符/符号的码字通过从根节点追踪到叶节点的路径来确定。

图像视频压缩基础

图像/视频压缩是必要的，原因有两点：

减少图像/视频的存储需求。

减少通过网络传输它们所需的比特率。

冗余

图像和视频之所以可以被压缩，是因为存在两种类型的冗余：

统计冗余 (Statistical Redundancy)

空间冗余 (Spatial redundancy)

时间冗余 (Temporal redundancy)

编码冗余 (Coding redundancy)

心理视觉冗余 (Psycho-visual Redundancy)

频率掩蔽 (Frequency masking)

颜色掩蔽 (Color masking)

有损无损压缩

无损压缩 (Lossless compression):

用于重要数据（例如，医学图像）。

解压后重建的图像与原始图像完全相同。

有损压缩 (Lossy compression):

对于图像或视频等媒体，无需显示超出人耳或人眼感知能力的信息。

压缩技术可以丢弃数据，而人类几乎察觉不到差异。

解压后重建的图像与原始图像不完全相同。

失真度量

图像压缩中最常用的三种失真度量是：

均方误差 (MSE) σ_e^2 :

$$\sigma_e^2 = \frac{1}{N} \sum_{n=1}^N (x_n - y_n)^2 \quad (8.1)$$

其中 x_n, y_n 和 N 分别是输入数据序列、重建数据序列和数据序列长度。

信噪比 (SNR)，单位为分贝 (dB):

$$SNR = 10 \log_{10} \frac{\sigma_x^2}{\sigma_e^2} \quad (8.2)$$

其中 σ_x^2 是原始数据序列的平均平方值， σ_e^2 是 MSE。

峰值信噪比 (PSNR):

$$PSNR = 10 \log_{10} \frac{x_{max}^2}{\sigma_e^2} \quad (8.3)$$

基于变换的编码/压缩

为什么使用变换编码?

将数据转换为更适合压缩的形式。

变换能提供良好的压缩特性:

提供能量集中。

提供冗余减少（即，降低变换系数间的相关性）。

可逆过程：

通过应用逆变换可以获得原始信号。

一个典型的基于变换的图像压缩系统包含以下步骤：图像分割、变换、量化和编码。

流程图：

输入图像 → 分割为子块 → 正向变换 → 量化器 → 符号编码器 → 压缩图像 → 符号解码器 → 逆变换 → 合并子块 → 解压图像。

变换

对输入图像数据应用一对一的变换。

变换后的输出是一种比原始图像数据更**适合进行有效压缩**的表示形式。

一些知名的变换：

离散傅里叶变换 (DFT)

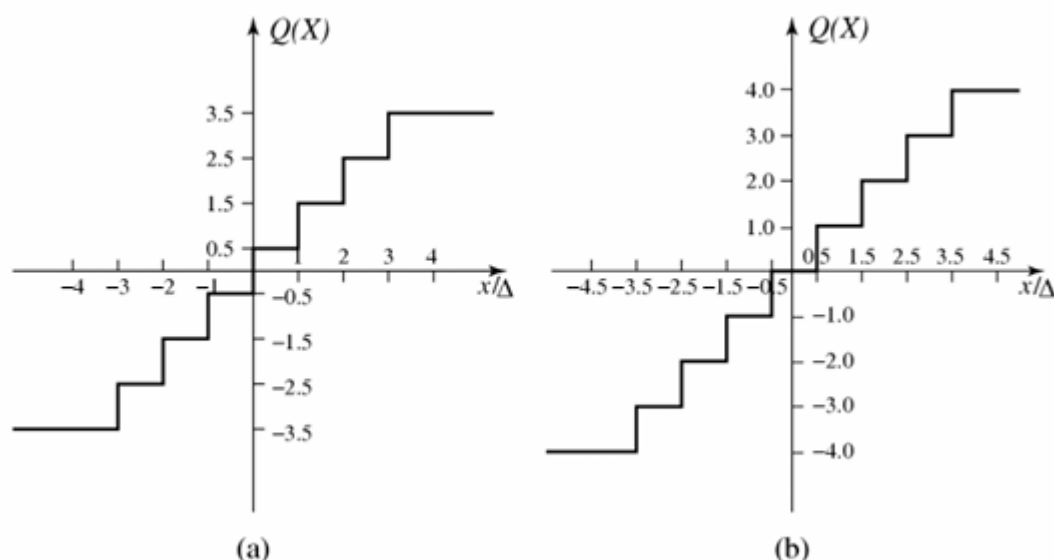
离散余弦变换 (DCT)

离散小波变换 (DWT)

量化

从变换系数生成有限数量的符号。

一种不可逆的多对一映射，会导致信息丢失。



中点法 (Midrise)

中平法 (Midtread)

编码

为量化器输出的每个符号分配一个码字或二进制比特流。

可以采用定长编码或变长编码。

变长编码 (VLC) 或熵编码：

以最小化符号二进制表示的平均长度的方式分配码字。

这是通过为更可能出现的符号分配更短的码字来实现的，这是熵编码（如哈夫曼编码）的基本原理。

离散余弦变换DCT

公式：

$$S_{uv} = \alpha(u)\alpha(v) \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} s_{ij} \cos \frac{(2i+1)u\pi}{2N} \cos \frac{(2j+1)v\pi}{2N}, \quad u, v = 0, \dots, N-1$$

$$\alpha(k) = \begin{cases} \sqrt{\frac{1}{N}} & \text{for } k = 0 \\ \sqrt{\frac{2}{N}} & \text{for } k = 1, 2, \dots, N-1 \end{cases}$$

2D-DCT 是一种在基于变换的图像压缩中广泛使用的变换。

它可以提供以下好处：

为变换系数提供能量集中。

减少变换系数间的冗余。

优点：压缩效果好，基函数是固定的，不依赖于图像。

缺点：压缩效果不如其他一些变换（例如，Karhunen Loeve 变换）。

二维DCT矩阵实现

将 2-D DCT 分解为两个连续的一维 DCT 矩阵乘法：

$$F(u, v) = \mathbf{T} \cdot f(i, j) \cdot \mathbf{T}^T$$

定义 DCT 矩阵 $\mathbf{T}$ ：

$$\mathbf{T}(i, j) = \begin{cases} \frac{1}{\sqrt{N}}, & i = 0 \\ \sqrt{\frac{2}{N}} \cdot \cos \frac{(2j+1)i\pi}{2N}, & i > 0 \end{cases}$$

其中 $i, j = 0, 1, \dots, N-1$ 。

JPEG

流行的图像压缩标准，支持有损与无损模式。

压缩比范围：10:1 至 20:1。

四种操作模式：

顺序 DCT 基础模式（sequential DCT-based mode）

渐进 DCT 模式（progressive DCT-based mode）

无损模式（lossless mode）

层次模式（hierarchical mode）

架构

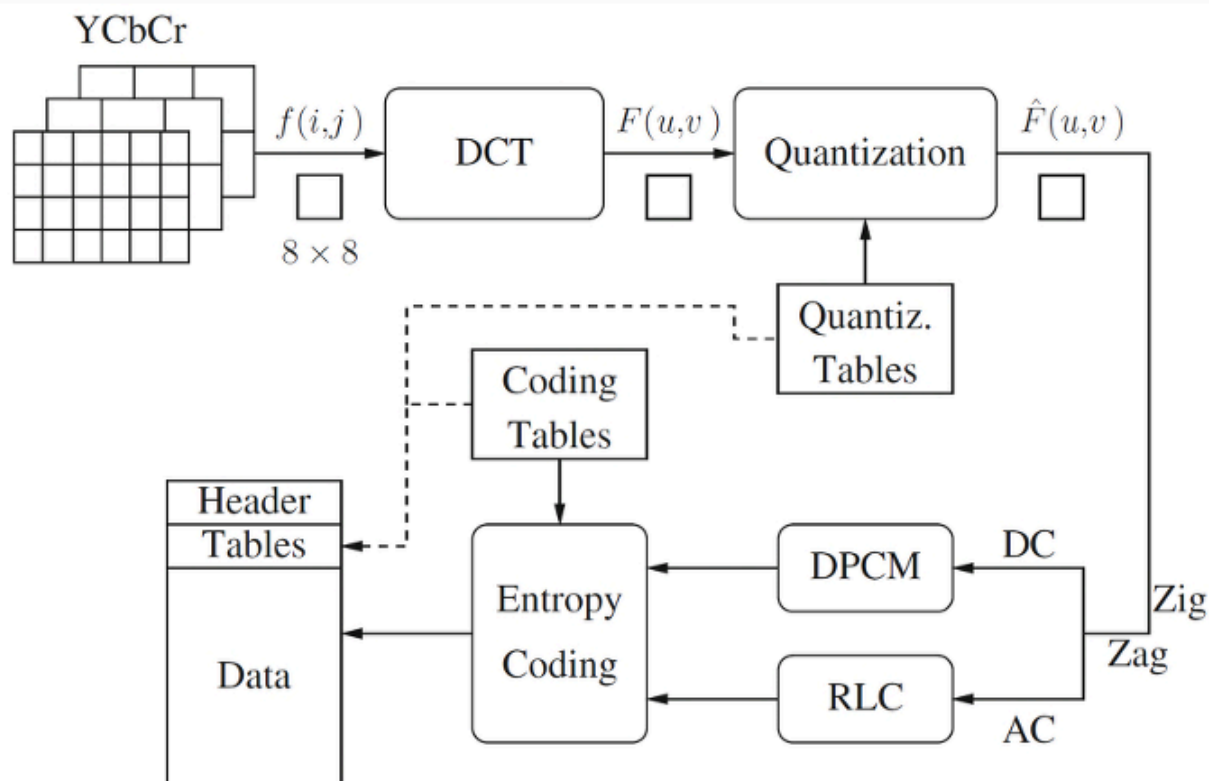


Fig. 9.1: Block diagram for JPEG encoder.

1. 图像/块处理 (Image/block processing)
2. 对图像块进行 DCT (DCT on image blocks)
3. 量化 (Quantization)
4. 熵编码 (Entropy coding)
5. 帧构建 (Frame building)

分块

图像首先被划分为多个 8×8 像素块。

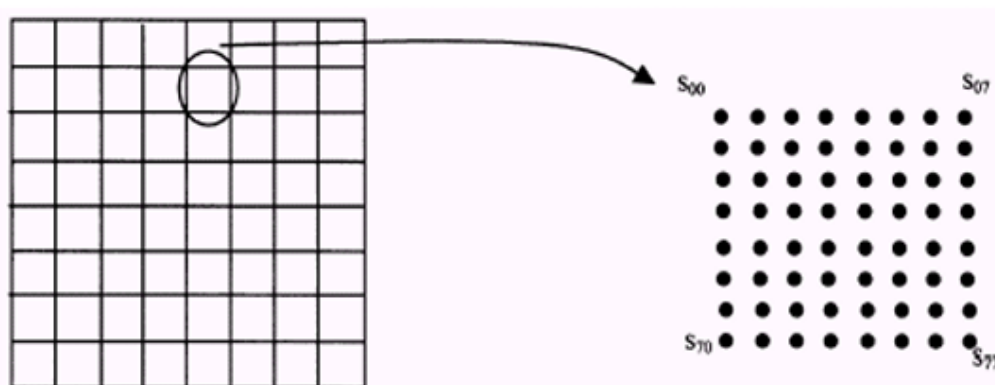
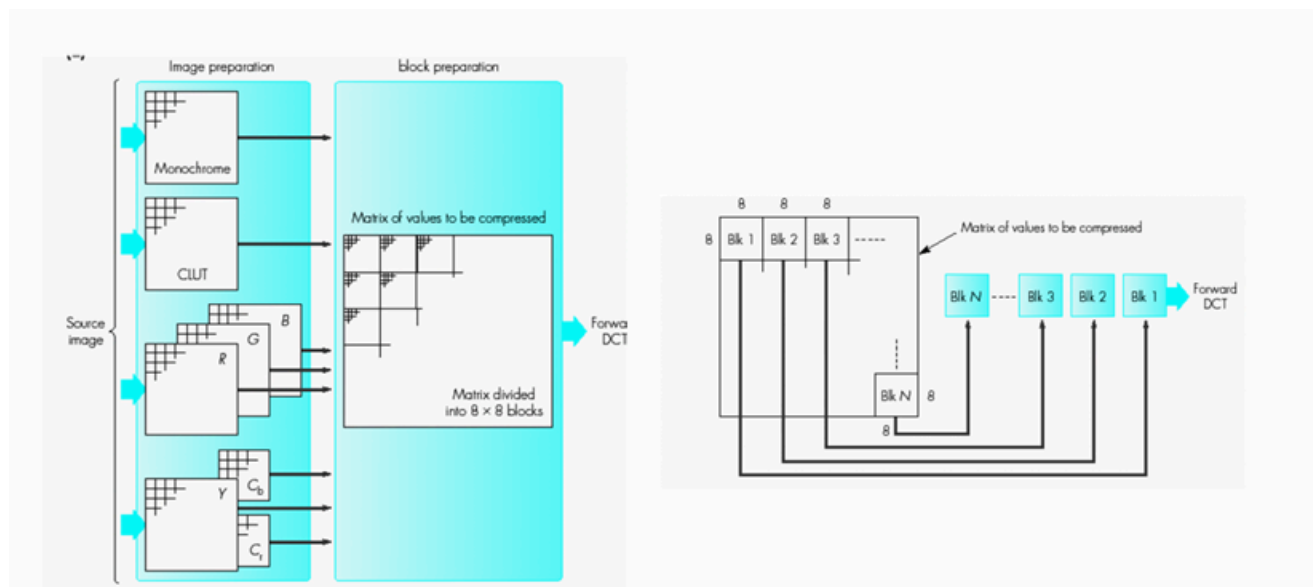


FIGURE 7.4 Partitioning to 8 × 8 blocks.

预处理

包括色彩空间转换（RGB → YCbCr）、下采样（Chroma Subsampling，如 4:2:0）等预处理步骤。



前向DCT变换

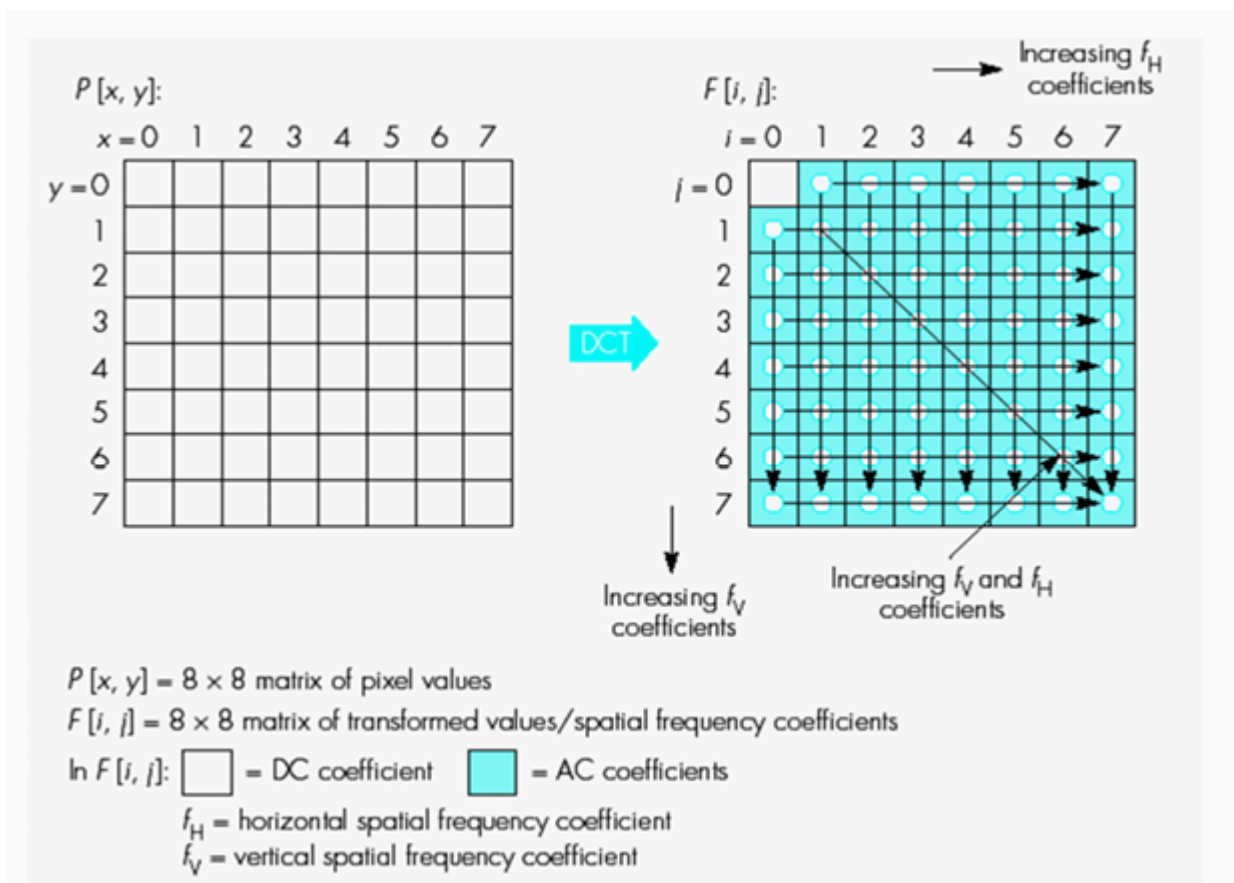
对每个 8×8 像素块应用前向 2D-DCT：

$$F(u, v) = \frac{C(u)C(v)}{4} \sum_{i=0}^7 \sum_{j=0}^7 f(i, j) \cos \frac{(2i+1)u\pi}{16} \cos \frac{(2j+1)v\pi}{16}$$

其中：

$$C(\xi) = \begin{cases} \frac{1}{\sqrt{2}}, & \xi = 0 \\ 1, & \xi = 1, 2, \dots, 7 \end{cases}$$

DCT系数



左：原始 8×8 像素块 $f(x, y)$ ；

右：对应的 8×8 DCT 系数块 $F(u, v)$ ，其中：

$u = 0, v = 0$ 为 **DC 系数**（左上角，代表块平均亮度）；

其余为 **AC 系数**（高频分量），按“Zigzag”顺序排列（从低频到高频）；

量化

定义：用有限比特数表示 DCT 系数，需将连续值映射为离散值 → **信息损失不可避免。**

方式：通过**量化表（Quantization Tables）** 对每个 DCT 系数除以对应步长并取整：

$$F_Q(u, v) = \text{round}\left(\frac{F(u, v)}{Q(u, v)}\right)$$

量化是 JPEG 中有损压缩的核心步骤。

量化表

人眼感知特性驱动设计：

人类对 **DC 和低频 AC 系数**更敏感 → 采用**较小量化步长**（保留更多细节）；

人类对 **高频 AC 系数**不敏感 → 采用**较大量化步长**（大幅舍入，引入失真但不易察觉）。

权衡：量化步长大小是**压缩比 vs 重建质量**的关键折衷。

熵编码

JPEG 的熵编码包含 4 步：

1. Vectoring（Zig-zag 扫描）

将 8×8 量化矩阵按 Z 字形展开为 1D 向量 → 利用**大量连续零值**提升后续编码效率。

2. Differential Encoding (DC 系数差分编码)

DC 系数：对当前块 DC 与前一块 DC 做差分（DPCM），因相邻块 DC 相关性强。

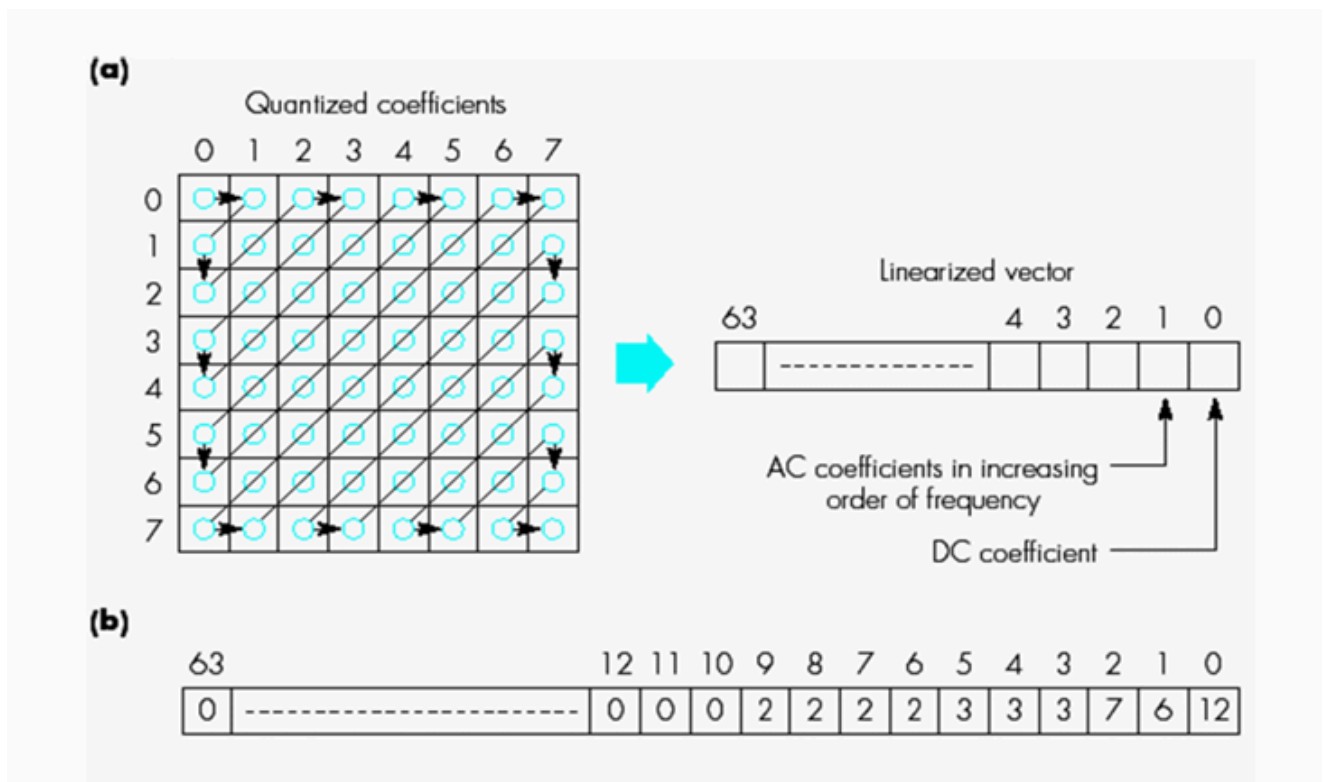
3. Run-Length Encoding (RLE)

AC 系数：编码为 (skip, value) 对，其中 skip = 连续零的个数，value = 下一个非零系数。

4. Huffman Encoding

对差分 DC 和 RLE 后的 (skip, value) 对进行霍夫曼编码 → 高频模式用短码字。

Zig-zag扫描



目的：将 2D 量化块转换为 1D 序列，使低频系数靠前、高频零值集中于尾部。

DC 系数差分编码

DC 系数单独处理:

当前块 DC 与前一块 DC 的差值被编码（而非原始值）；

因相邻块亮度相似，差值小 → 更易压缩；

该差值再经 Huffman 编码。

Run-Length Encoding (RLE)

格式：每对 (skip, value) 表示：

skip：当前非零系数前连续零的个数（0-15，因 JPEG 规定最大 run=15）；

value：下一个非零 AC 系数（带符号，范围受限）。

示例：序列 $[0, 0, 0, 5, 0, 0, -3] \rightarrow (3, 5), (2, -3)$

Huffman Encoding

应用于：

差分 DC 系数（1 个符号）

RLE 后的 (skip, value) 对（共 162 种可能组合，JPEG 定义了标准 Huffman 表）

优势：高频出现的模式（如 (0,0)、(0,1)）分配短码字 → 实现高压缩。

图片质量和DCT的关系

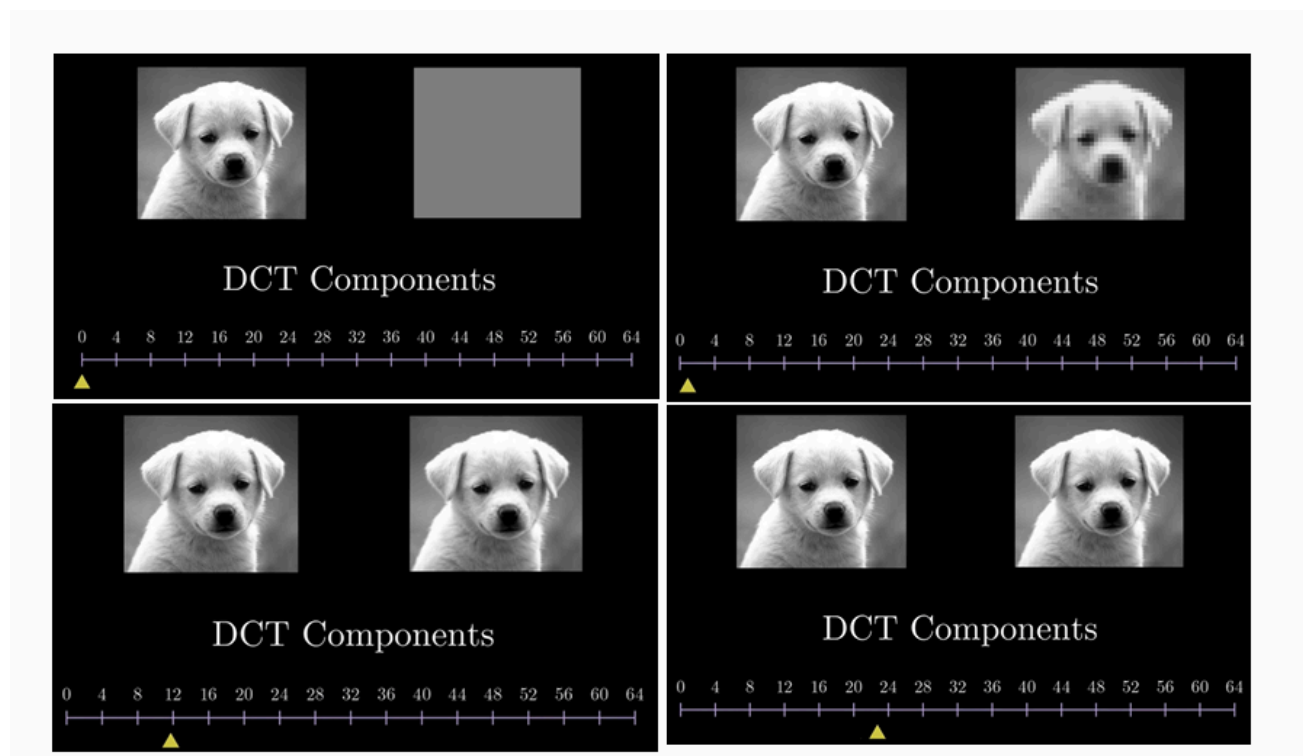
仅保留 DC 系数 → 图像只剩块级平均色块（极度模糊）；

加入少量低频 AC → 出现轮廓和主要结构；

加入中频 AC → 细节、边缘显现；

完整 DCT 系数 → 接近原始图像。

结论：DC + 低频 AC 主导主观质量；高频 AC 主要贡献纹理细节，人眼不敏感 → 可安全量化丢弃。



比特流结构

Frame Building：JPEG 定义了比特流语法结构，用于封装所有信息（头、表、扫描数据等）。

JPEG Bitstream 结构 (树状层次)

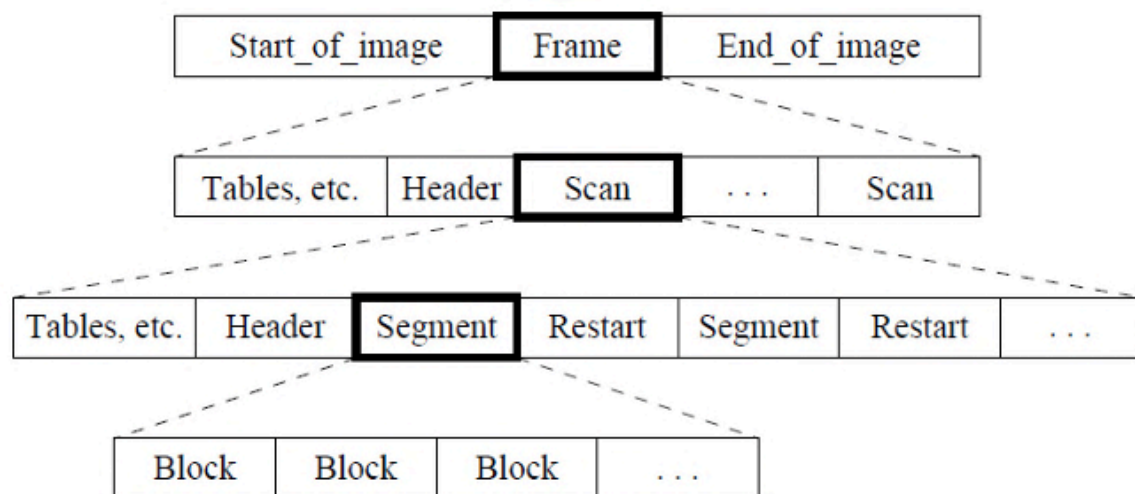
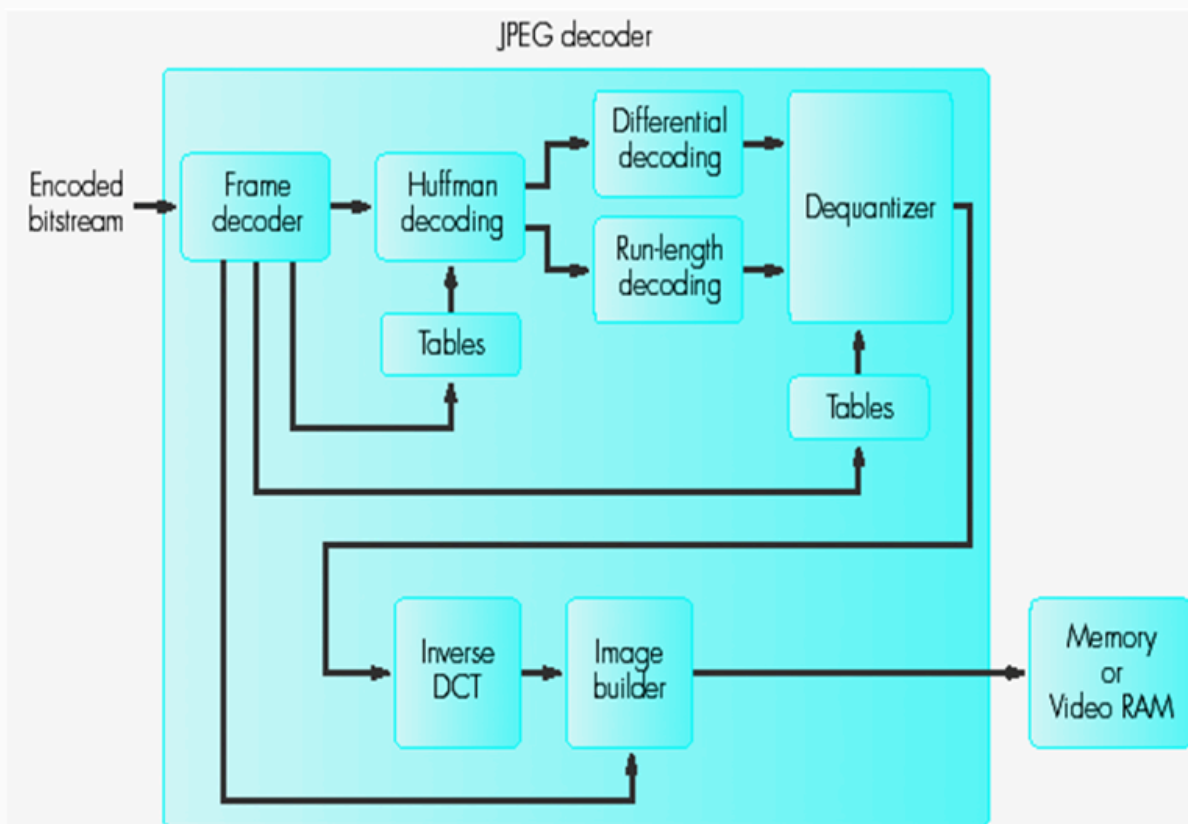


Fig. 9.6: JPEG bitstream.

解码器



Video Coding & Standards 视频编码&标准

Video Compression Basics 视频压缩基础

三大冗余

空间冗余 (Spatial Redundancy)

定义：同一帧内像素间的统计相关性（如邻近像素亮度相似）。

又称 **帧内冗余 (intraframe redundancy)**。

时间冗余 (Temporal Redundancy)

定义：连续帧之间像素的统计相关性（如背景静止、物体缓慢移动）。

又称 **帧间冗余 (interframe redundancy)**。

心理视觉冗余 (Psychovisual Redundancy)

人类视觉系统 (HVS) 对某些信息不敏感：

频率掩蔽 (Frequency masking)：人眼对高频噪声/失真更不敏感。

颜色掩蔽 (Color masking)：人眼对亮度 (Luma, Y) 比色度 (Chroma, C_b, C_r) 更敏感。

视频压缩的核心策略

三大冗余

空间冗余

时间冗余

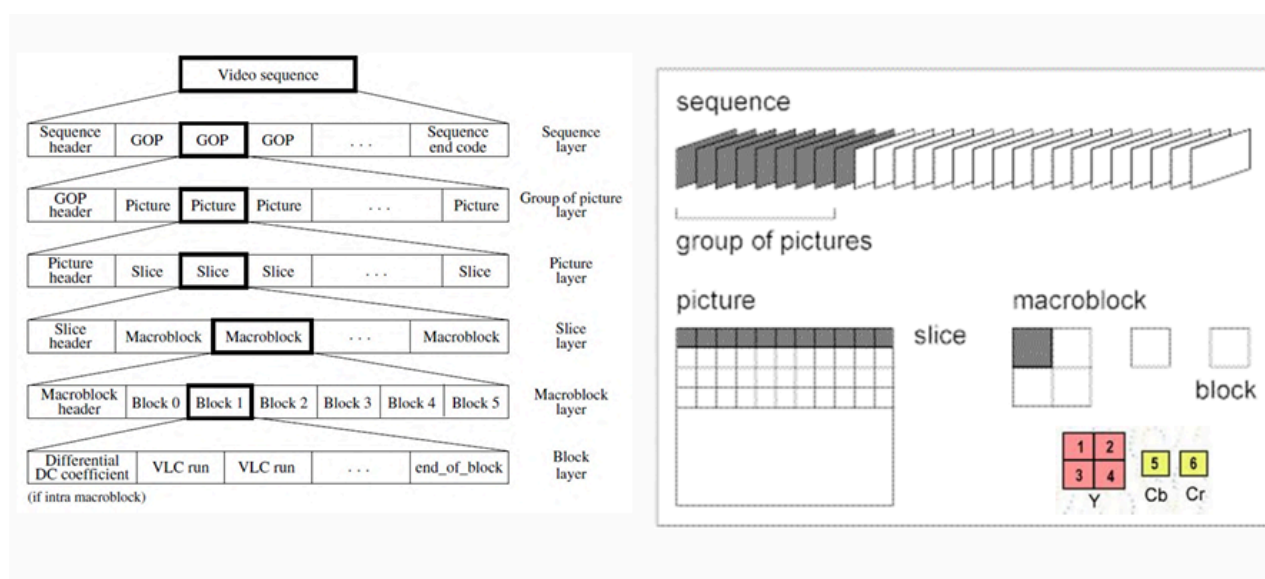
心理视觉冗余

提升编码效率

霍夫曼编码 (Huffman coding)

预测编码 / 差分编码 (Predictive / Differential coding)

视频结构层级 (Video Structure)



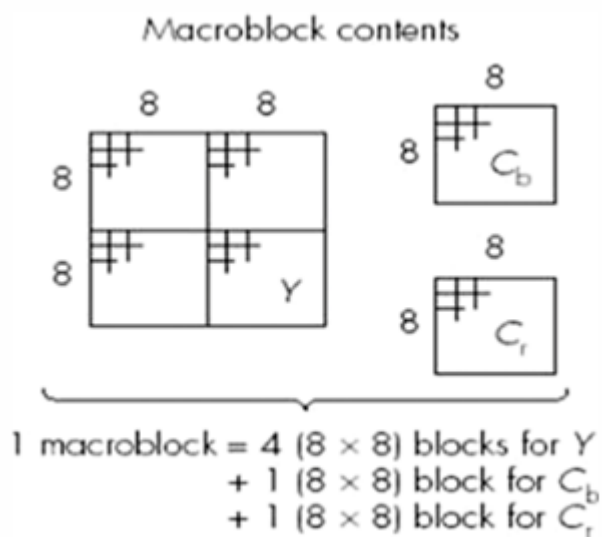
宏块 (Macroblock, MB)：

尺寸： 16×16 像素 (Y 分量)；

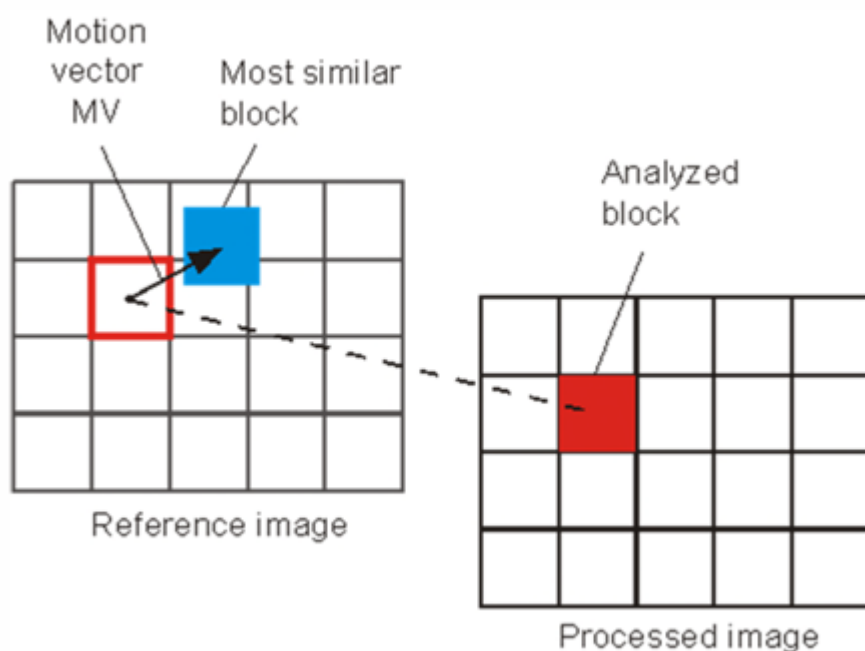
对应色度分量： 8×8 (C_b, C_r)；

组成：4 个 8×8 亮度块 + 2 个 8×8 色度块；

是 **DCT 变换** 与 **运动补偿** 的基本处理单元。



ME&MC运动估计与补偿



帧类型与压缩方式

Intraframe compression (帧内压缩):

如 I-frame (MPEG 中), 视作静态图像 (类似 JPEG);

消除空间冗余;

不依赖其他帧。

Interframe compression (帧间压缩):

如 P/B-frame;

利用时间域相关性;

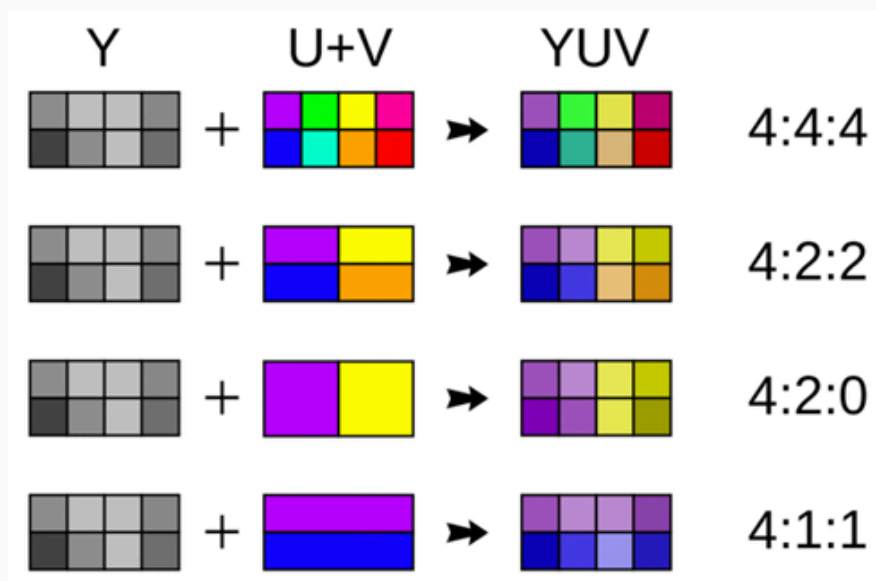
依赖运动估计与补偿 (ME/MC);

显著减少数据量。

亮度与色度分离

■ Subsampling formats:

- ◆ 4:4:4 - No subsampling
- ◆ 4:2:2, 4:1:1 - horizontal subsampling
- ◆ 4:2:0 - horizontal and vertical subsampling



比例	含义拆解	视觉效果（看图中的 U+V 列）
4:4:4	第一行 4 个色度，第二行 4 个色度。	全采样：每个亮度像素都有对应的色彩。画质最高。
4:2:2	第一行 2 个色度，第二行 2 个色度。	水平压缩：每两个水平像素共用一个色彩。
4:2:0	第一行 2 个色度，第二行 0 个色度。	水平+垂直压缩：第二行直接复用第一行的色彩。这是互联网视频（H.264/H.265）最常用的格式。
4:1:1	第一行 1 个色度，第二行 1 个色度。	极度压缩：每四个水平像素才有一个色彩采样。

MPEG基础

目标：对视频信号进行高效编码/压缩。

核心技术基础：

DCT 编码：消除帧内空间冗余；

运动估计与补偿（ME/MC）：消除帧间时间冗余。

YCbCr 色彩空间（通常采用 4 : 2 : 0 子采样）：

1 个亮度分量 Y (16×16)；

2 个色度分量 C_b, C_r (各 8×8)；

MPEG视频结构

(a)

One (or more) stored video(s):

----- Sequence -----

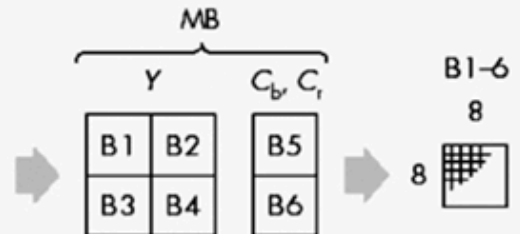
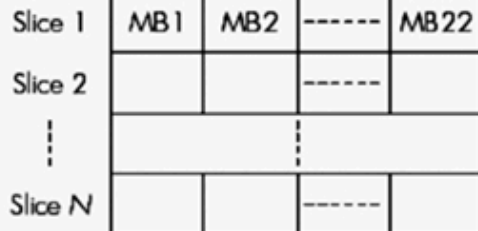
Multiple GOPs:

GOP1 GOP2 GOP3 GOP4 ----- GOPN

GOP format: -----

I B B P B B P B B P B B I -----

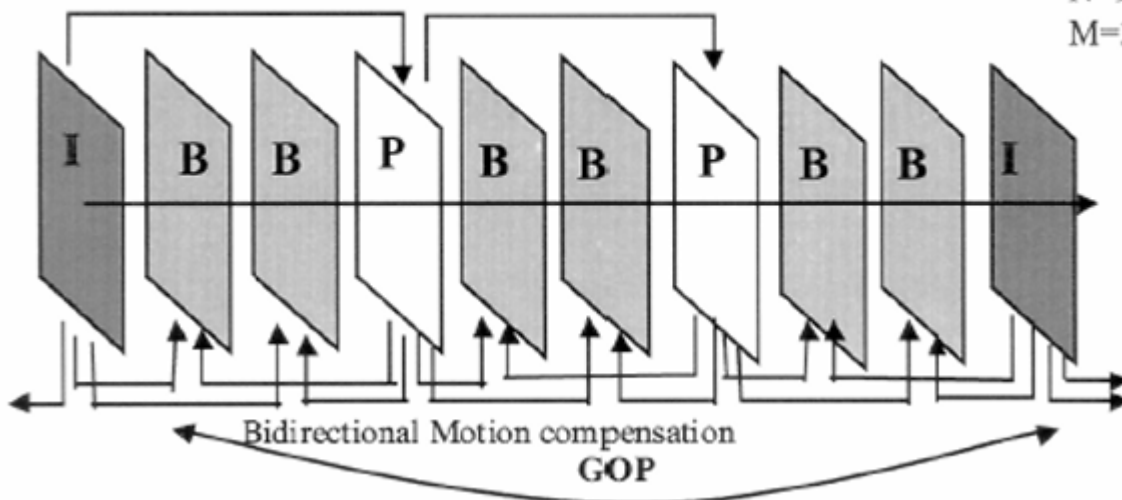
I, P or B
pictures/frames:



Forward Motion
Compensation

Forward Motion
Compensation

N=9
M=3



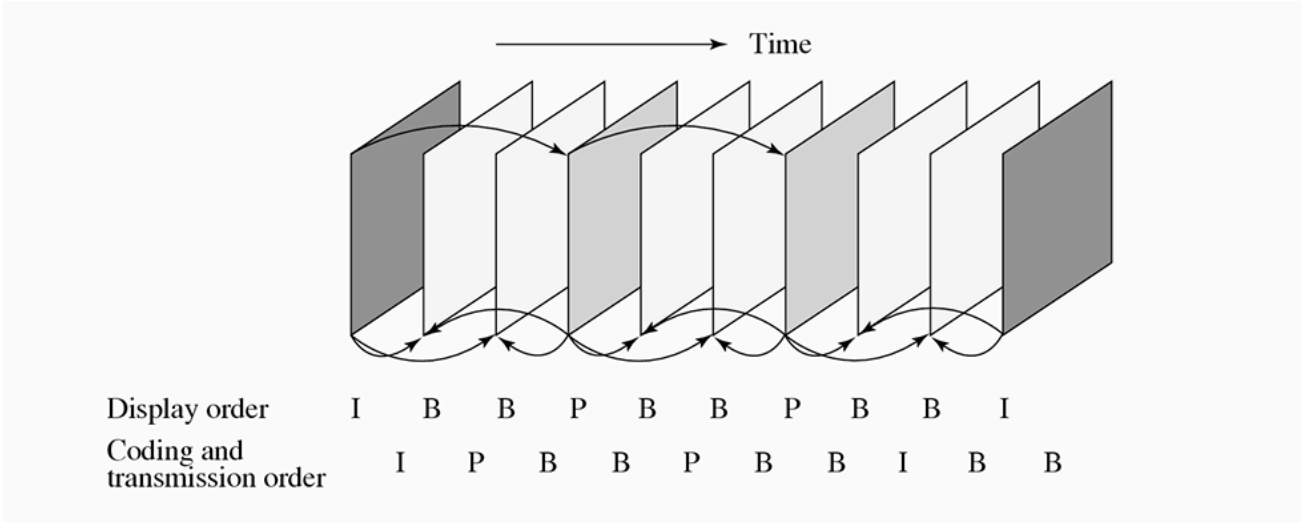
GOP

视频序列被划分为 **图像组 (GOP)**，是随机访问与错误恢复的基本单元。

帧类型	全称	编码方式	参考关系	是否可独立解码
I	Intra-coded frame	帧内预测 + DCT + 熵编码	无参考	✅ 是
P	Predictive-coded frame	单向运动补偿 (前向)	仅参考前一 I/P 帧	❌ 否
B	Bidirectionally-coded frame	双向运动补偿 (前向 + 后向)	参考前后 I/P 帧	❌ 否

I-frame:
独立编码，类似 JPEG；
 Y, C_b, C_r 块均用 JPEG 算法独立编码；

P/B-frame:
仅编码 **运动矢量 (MV) + 残差 (prediction error)**；
B-frame 可用“双向预测”，压缩率最高，但引入解码延迟。



最大压缩比: B-frame
原因：利用前后帧信息做双向预测，残差极小；且不作为其他帧参考，可高度压缩。

最小压缩比: I-frame
原因：无参考帧，必须完整编码所有像素信息；空间冗余虽经 DCT 压缩，仍远高于 P/B 帧。

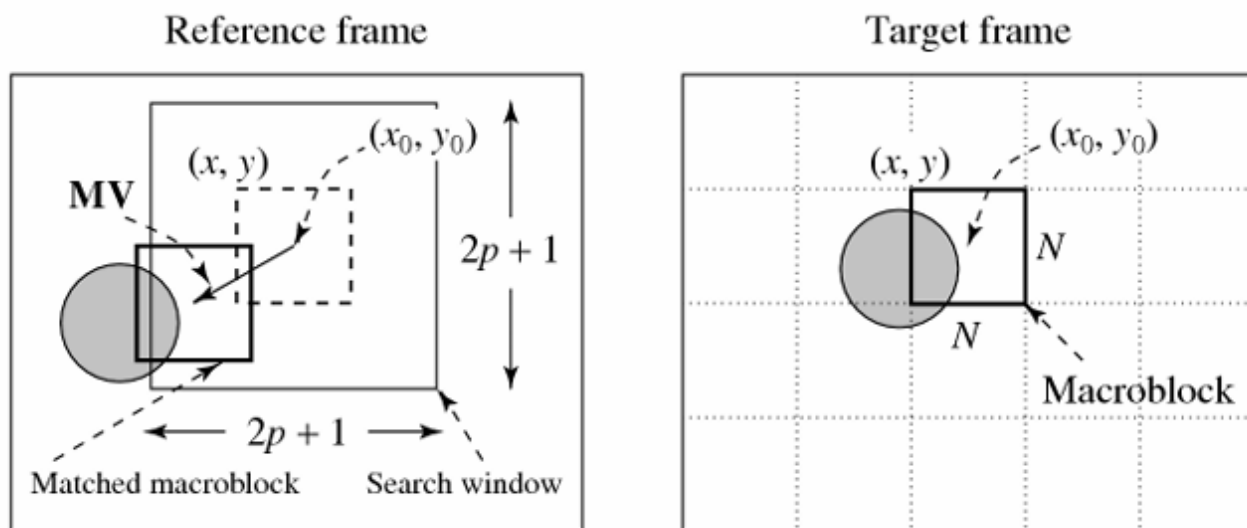
目标	推荐 GOP 参数	理由
高压缩效率	N 大, M 大 (如 $N = 15, M = 5$)	更多 B/P 帧, 冗余利用充分
低延迟 / 快速随机访问	N 小, M 小 (如 $N = 6, M = 2$)	I 帧更密集, 跳转更快
抗误码 / 错误传播控制	M 不宜过大 (≤ 12)	防止一个 I 帧错误导致整个 GOP 失效
实时通信 (如视频会议)	多用 I/P, 少用 B	B 帧需缓存, 增加延迟

运动估计与补偿

利用**时间冗余**：相邻帧内容相似 $\rightarrow$ 仅编码“变化部分”(残差) + “如何移动”(MV)。

运动矢量 (Motion Vector, MV)：(u, v)，表示目标宏块在参考帧中的偏移。

运动估计 (Motion Estimation)



搜索范围：通常限定在 $[-p, p]$ 内（如 $p = 16$ ），搜索窗口尺寸为 $(2p + 1) \times (2p + 1)$ 。

匹配准则：常用 **Mean Absolute Difference (MAD)**：

$$\text{MAD}(i, j) = \frac{1}{N^2} \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} |C(x + i, y + j) - R(x, y)|$$

其中：

C : 当前宏块像素；

R : 参考帧中候选位置像素；

N : 宏块边长（如 16）；

i, j : 水平/垂直位移（即 MV 分量）。

目标：找到 $(i^*, j^*) = \arg \min_{i, j} \text{MAD}(i, j)$ ，使 MAD 最小。

运动补偿（Motion Compensation）

解码端根据 MV 从参考帧中取出预测块 $\hat{C}$ ；

重建块： $\tilde{C} = \hat{C} + E$ ，其中 E 为编码传输的残差。

运动估计方法对比

方法	描述	速度	精度	适用场景
Full Search	遍历整个搜索窗口所有点	⚠️ 慢	✅ 最高	离线编码、质量优先
Three-step Search	分三步缩小搜索范围：粗→中→细（每次步长 halve）	✅ 快	⚠️ 中等	实时系统常用
2D Logarithmic Search	步长按 2^k 递减（如 8→4→2→1），沿坐标轴+对角线搜索	✅ 快	⚠️ 中等	平衡型方案
Hierarchical Search	多分辨率金字塔搜索（先低分辨率粗搜，再高分辨率精搜）	中	✅ 高	高性能编码器
Diamond Search	菱形模板搜索（中心→菱形点→更小菱形），收敛快，抗局部极小值能力强	✅ 快	✅ 高	H.264/AVC 默认

方法	描述	速度	精度	适用场景
Cross Search	十字形搜索（水平+垂直线），简单高效	✓ 快	⚠ 中低	嵌入式设备

全搜索（Full Search, FS）

策略：遍历整个搜索窗口 $[-p, p] \times [-p, p]$ 中所有整数点 (u, v) 。

计算复杂度： $O((2p + 1)^2)$ —— 当 $p = 16$ 时需 1089 次 MAD 计算/宏块。

优点：

- 全局最优解（无局部极小陷阱）；
- 压缩率最高（残差最小）。

缺点：

- 计算开销大，难以满足实时要求；
- 实际硬件中常被近似算法替代

三步搜索（Three-Step Search, TSS）

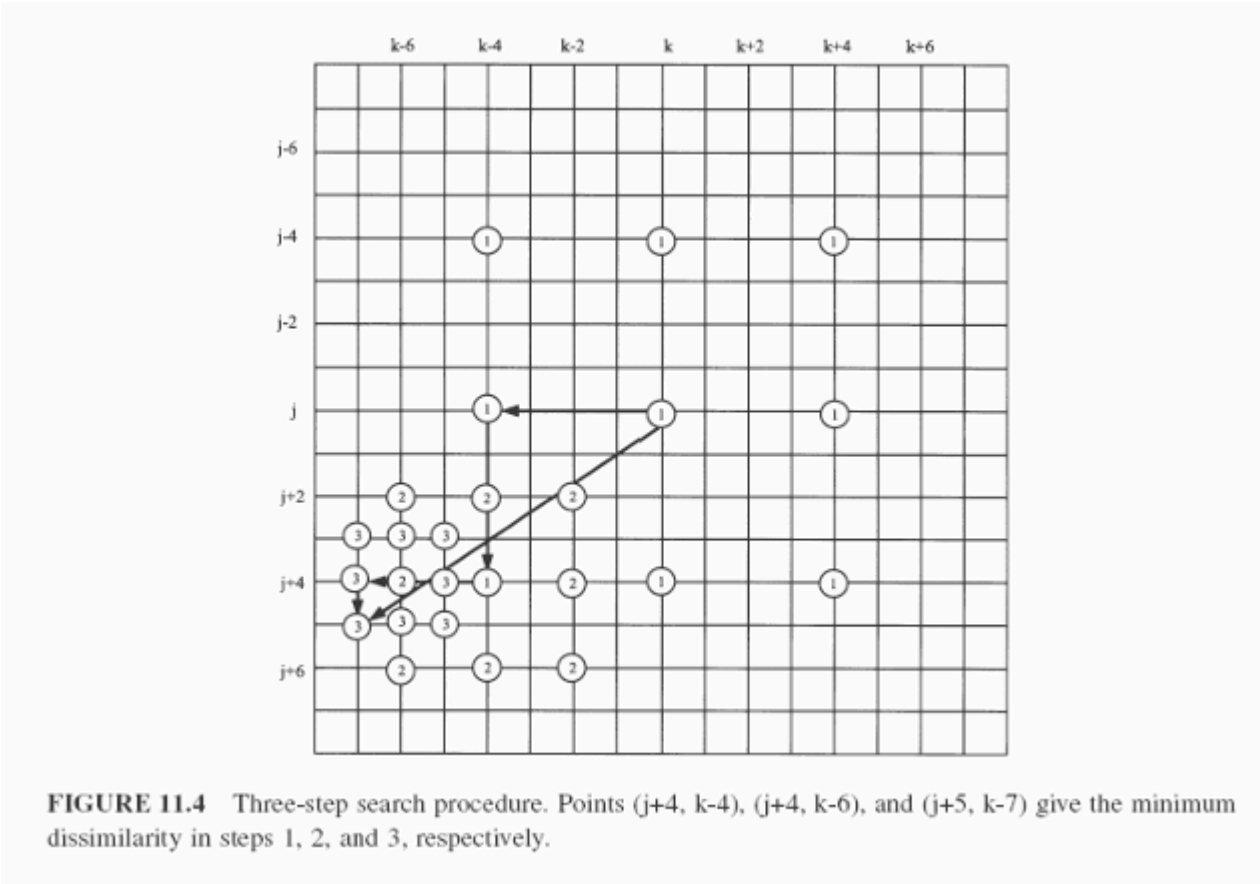


FIGURE 11.4 Three-step search procedure. Points (j+4, k-4), (j+4, k-6), and (j+5, k-7) give the minimum dissimilarity in steps 1, 2, and 3, respectively.

步骤（固定 3 步，总 25 点）：

- Step 1：**步长 $s = p$ ，以中心 $(0, 0)$ 为原点，搜索 9 点；
- Step 2：**以 Step1 最优点为中心，步长 $s/2$ ，再搜 9 点；
- Step 3：**以 Step2 最优点为中心，步长 1，搜 9 点 → 输出最终 MV。

优势：

- 速度提升；

局限：

- 压缩率小，不精确

2D 对数搜索 (2D Logarithmic Search, LS)

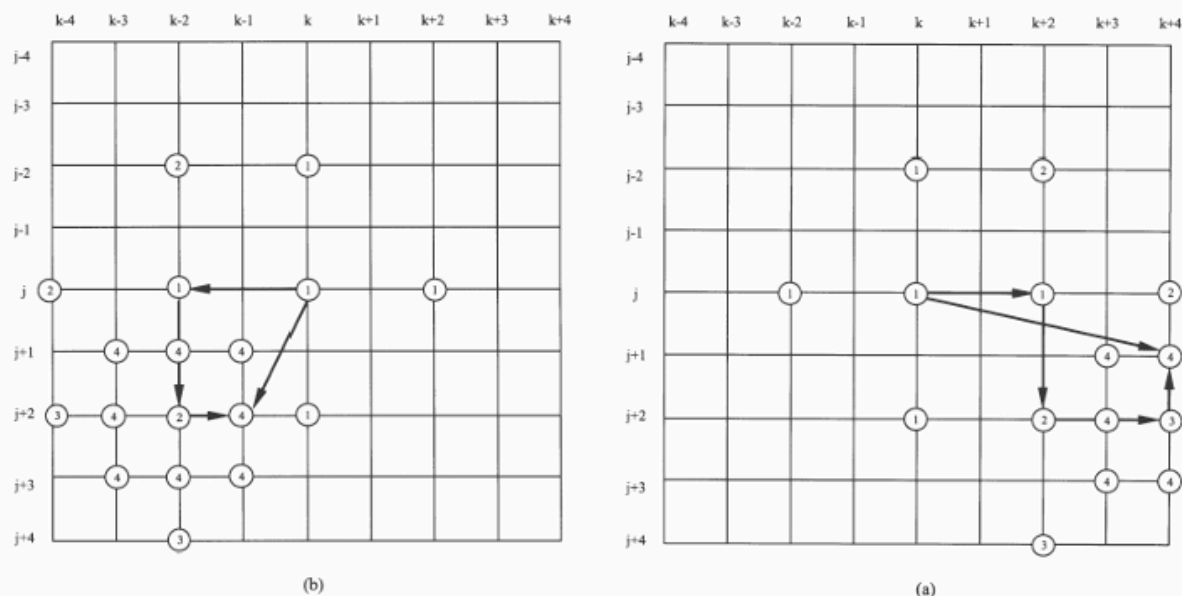


FIGURE 11.3 (a) A 2-D logarithmic search procedure. Points at $(j, k+2)$, $(j+2, k+2)$, $(j+2, k+4)$, and $(j+1, k+4)$ are found to give the minimum dissimilarity in steps 1, 2, 3, and 4, respectively. (b) A 2-D logarithmic search procedure. Points at $(j, k-2)$, $(j+2, k-2)$, and $(j+2, k-1)$ are found to give the minimum dissimilarity in steps 1, 2, 3, and 4, respectively.

38

策略：步长按 2^k 递减（如 $p = 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$ ），每步搜索 **5 点**（中心 + 上/下/左/右）。

流程：

1. Step 1: 步长 $s = p$ ，搜 5 点；
2. Step 2: 若当前点非边界，则步长 $s/2$ ，以该点为中心再搜 5 点；
3. 重复至 $s = 1$ ，返回最佳点。

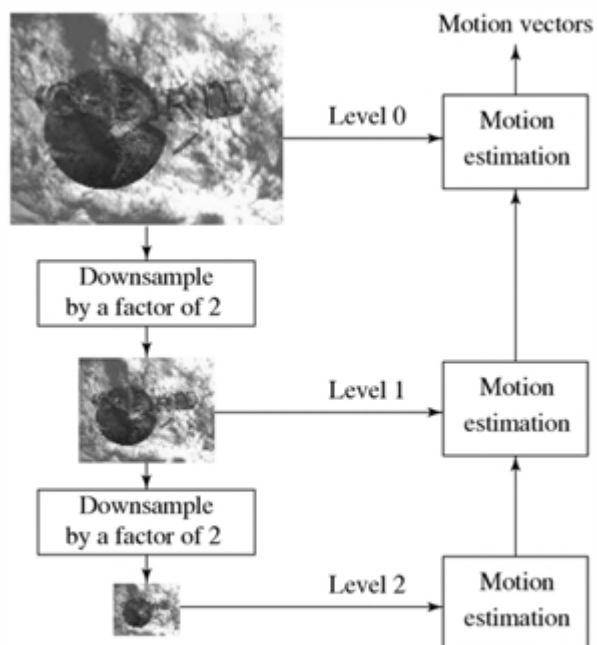
特点：

比 TSS 更适应非均匀运动（因沿坐标轴搜索）；

但对斜向运动效率略低（缺少对角线点）。

速度和精确度介于全搜索和三步搜索

层次化搜索 (Hierarchical Search)



A Three-level Hierarchical Search

核心思想：构建图像金字塔（Level 0: 原图；Level 1: $\downarrow 2$ ；Level 2: $\downarrow 4$ ）。

流程（三级为例）：

1. **Level 2**（最低分辨率）：粗略搜索，得初始 MV_2 ；
2. **Level 1**：以 $MV_2 \times 2$ 为起点，在原图 1/2 尺寸下精搜，得 MV_1 ；
3. **Level 0**（全分辨率）：以 $MV_1 \times 2$ 为起点，精细搜索，得最终 MV_0 。

关键收益：

初始 MV 估计误差小 $\rightarrow$ 搜索窗口可大幅缩小（如 p 从 16 $\rightarrow$ 4）；

宏块尺寸可同比例缩小（如 $16 \times 16 \rightarrow 4 \times 4$ ），计算量指数下降；

总操作数可降至 Full Search 的 **1/10 ~ 1/20**。

适用场景：高分辨率视频（4K/8K）、GPU 加速编码器。

历史与里程碑

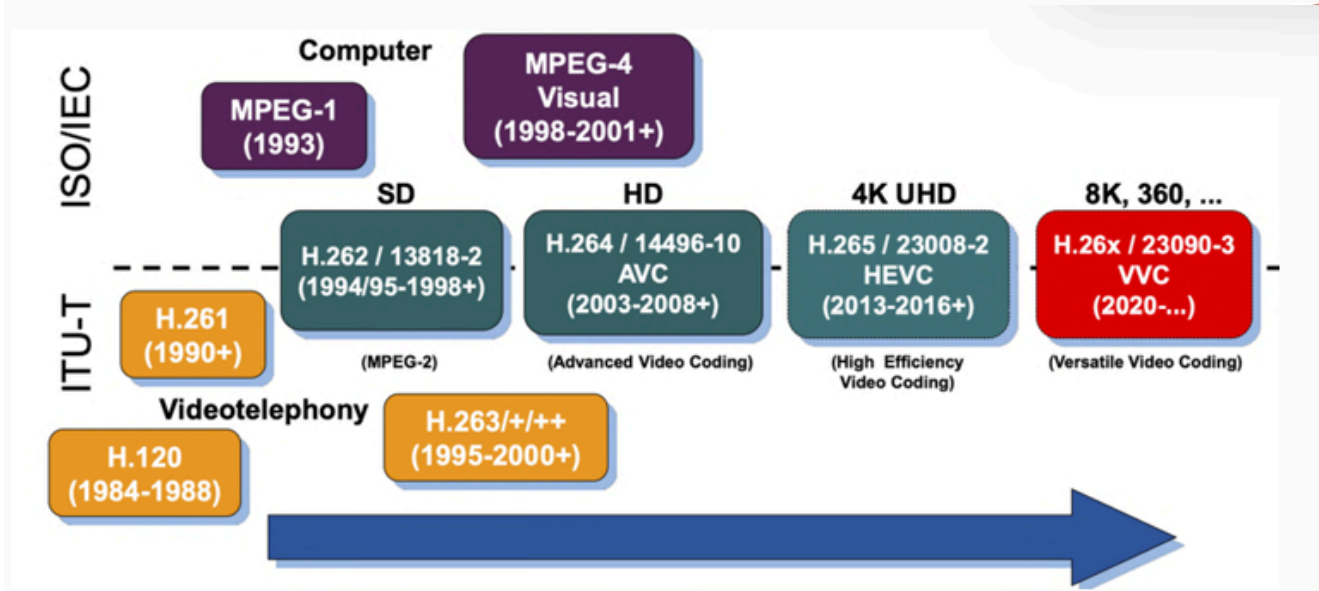
MPEG标准制定背景：

ISO 和 IEC 创建 **Moving Picture Experts Group (MPEG)**，负责发布 MPEG 标准。

ITU 发布 H.26x 编码标准（如 H.261、H.263）。

ITU 与 ISO/IEC 合作发布 H.262、H.264、H.265、H.266。

Abbreviation	IEC	ITU	ISO
Official Name	International Electrotechnical Commission	International Telecommunication Union	International Organization for Standardization
Target Fields	Electrical engineering and electronic technology	Wireless and electrical communication	Industrial products



MPEG

MPEG-1

总览

由 ISO/IEC 于 **1992年11月** 开发。

用于**有损视频/音频压缩**，支持数字存储介质（如 VCD）的 1.5 Mbps 视频及关联音频编码。是 JPEG 标准的扩展（对应 H.261）。

仅支持**渐进扫描**（Progressive Scan）。

支持分辨率：

NTSC 视频：352 × 240 @ 30 fps

PAL 视频：352 × 288 @ 25 fps

色度子采样：4 : 2 : 0

部分

也称为 **ISO/IEC 11172**，包含 5 个部分：

Part 1: MPEG-1 系统

Part 2: MPEG-1 视频

Part 3: MPEG-1 音频

Part 4：一致性测试

Part 5：参考软件

局限性

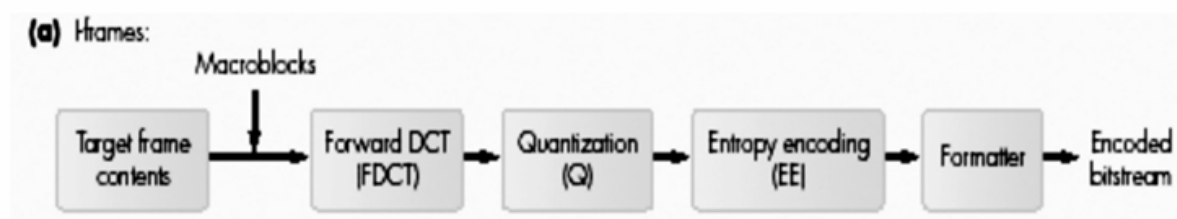
仅支持渐进扫描。

画面质量较低。

压缩比偏低。

I/P/B 帧编码机制

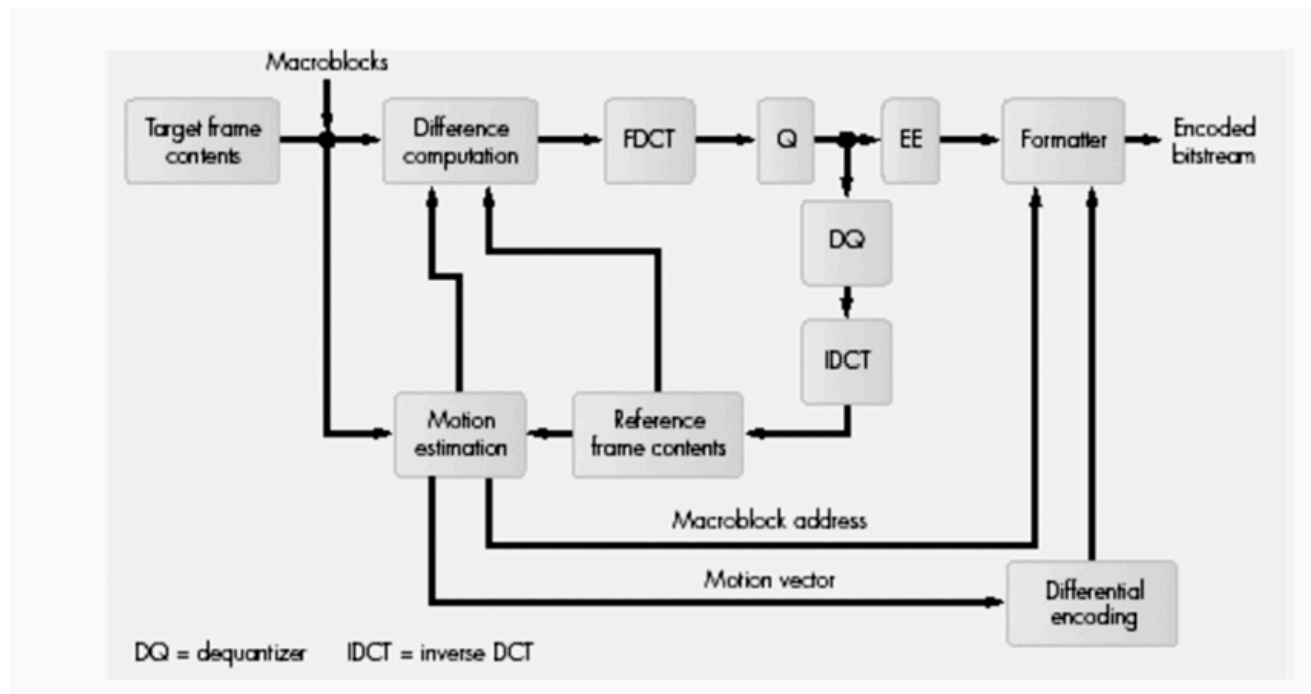
I-帧 (Intra Frame)



编码方式：独立编码，不依赖其他帧。

流程：宏块 → 前向 DCT → 量化 → 熵编码 → 变换 → 编码比特流。

P-帧 (Predictive Frame)



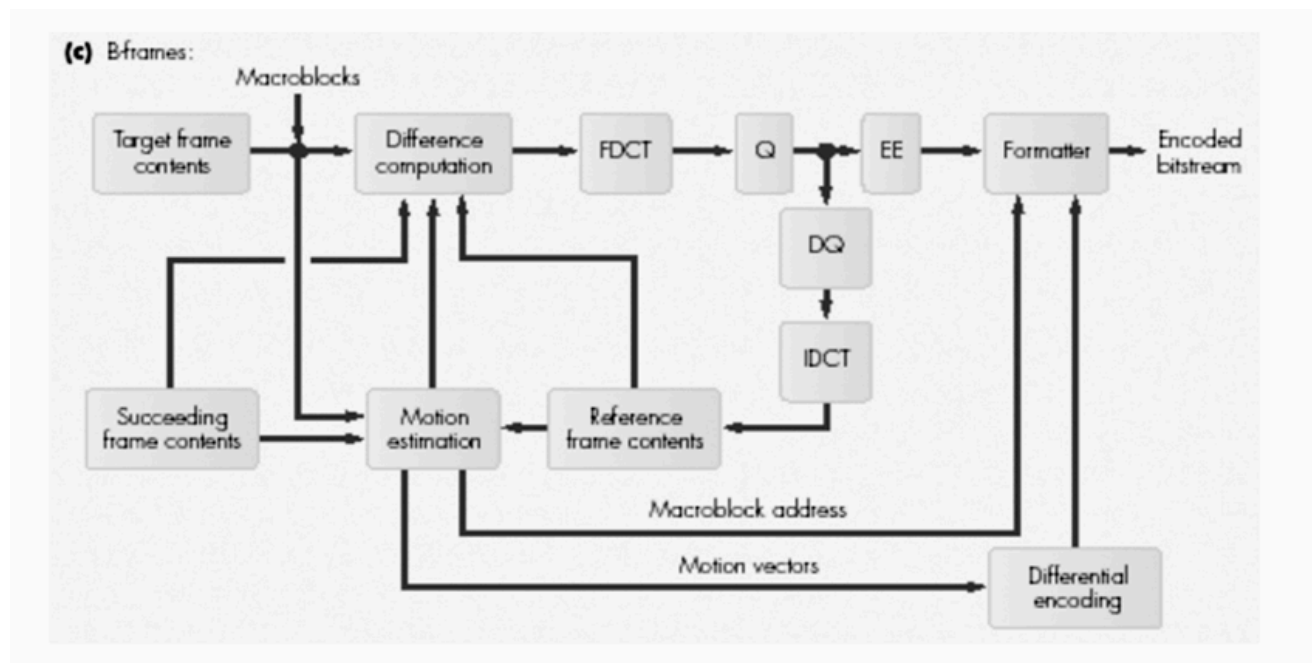
核心原理：基于前一帧I 或 P 帧进行运动估计与补偿。

关键步骤：

1. 对目标帧的每个宏块，在参考帧中搜索匹配块（以当前块为中心的搜索窗口）。
2. 若**绝对差和Sum of Absolute Difference, SAD**小于阈值，则判定匹配。
3. 匹配时确定两个参数：**运动矢量（Motion Vector）** 和 **预测误差矩阵（Prediction Error Matrix）**。

- 运动矢量通过差分编码，使用 **Huffman 编码** 表示。
- 预测误差矩阵 (Y, C_b, C_r) 编码方式与 I 帧相同。
- 无匹配时，宏块按 I 帧独立编码。

B-帧 (Bi-directional Frame)



核心原理：基于前一帧和后一帧I/P 帧进行双向运动估计与补偿。

关键步骤：**

- 为每个宏块生成两组参数：
 - 基于前一参考帧的（运动矢量、预测误差）
 - 基于后一参考帧的（运动矢量、预测误差）
- 计算第三组差分矩阵：目标宏块与**前后最佳宏块均值**的差值。
- 选择**预测误差最小**的一组进行编码（编码方式同 P 帧）。

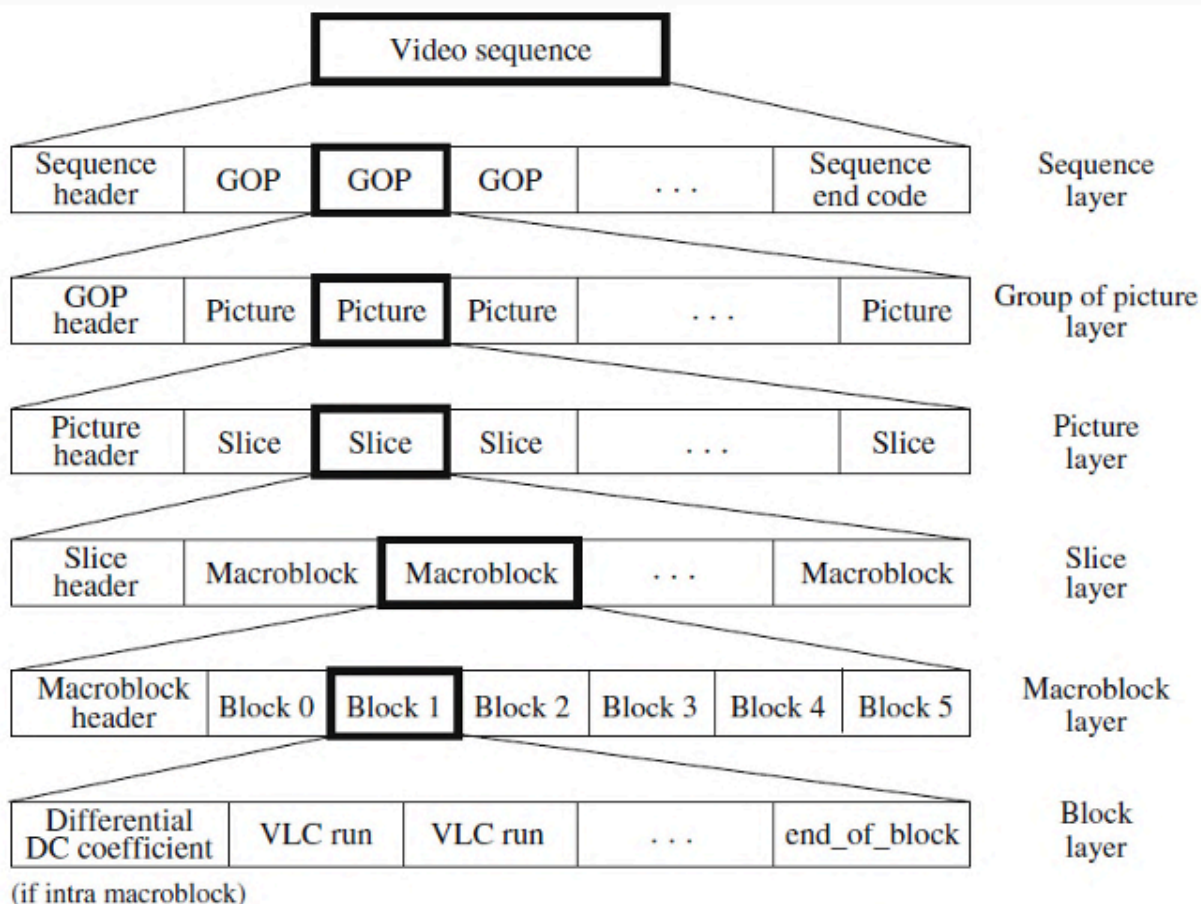
帧大小对比

P 帧 < I 帧：利用**时间冗余**（前向预测）。

B 帧 < P 帧：利用**双向预测**优势。

帧类型	大小	压缩比
I	18 kB	7:1
P	6 kB	20:1
B	2.5 kB	50:1
平均	4.8 kB	27:1

比特流结构



MPEG-2

概述

设计目标：解决 MPEG-1 的局限性（如低比特率 1.5 Mbps、仅支持渐进扫描）。

标准化时间：1995 年。

应用场景：为数字广播电视（隔行扫描）设计，支持高比特率（4 Mbps）。

核心特性：

定义多种配置文件（Profiles）适配不同应用场景。

支持**可扩展编码**（Scalable Coding）。

配置文件与级别

Table 11.5 Profiles and Levels in MPEG-2

Level	Simple profile	Main profile	SNR scalable profile	Spatially scalable profile	High profile	4:2:2 profile	Multiview profile
High		*			*	*	
High 1440		*		*	*	*	
Main	*	*	*		*	*	*
Low	*	*	*				

Table 11.6 Four levels in the main profile of MPEG-2

Level	Maximum resolution	Maximum fps	Maximum pixels/sec	Maximum coded data rate (Mbps)	Application
High	1,920 × 1,152	60	62.7×10^6	80	Film production
High 1440	1,440 × 1,152	60	47.0×10^6	60	Consumer HDTV
Main	720 × 576	30	10.4×10^6	15	Studio TV
Low	352 × 288	30	3.0×10^6	4	Consumer tape equivalent

组成部分

Part-1: 系统 (Systems)

Part-2: 视频 (Video)

Part-3: 音频 (Audio)

Part-4: 一致性 (Conformance)

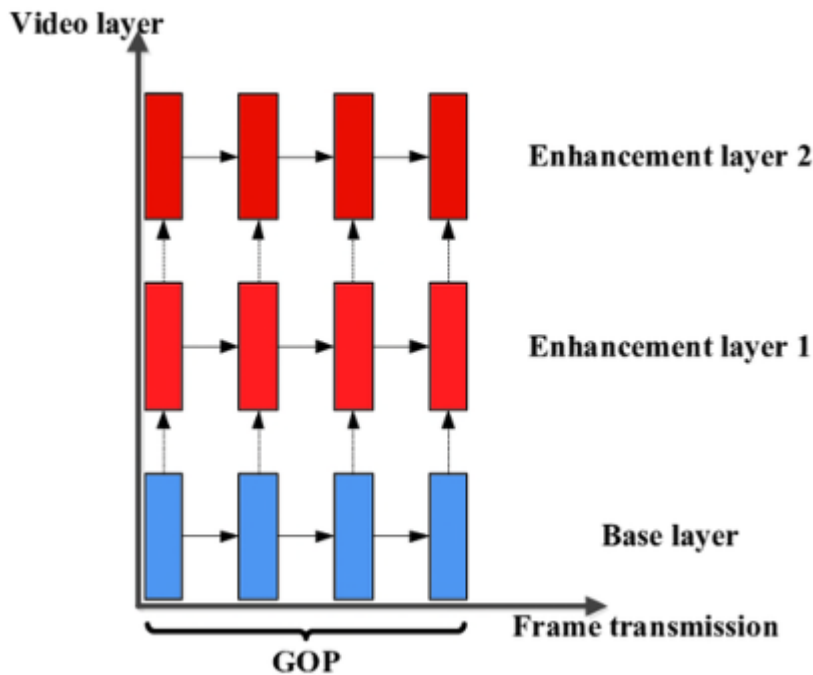
Part-5: 参考软件 (Reference software)

Part-6: DSM CC (Digital Storage Media Command and Control)

Part-7: AAC (Advanced Audio Coding)

Part-9: RTI (Real Time Interface)

可扩展性



分层结构：基础层（Base layer）+ 增强层（Enhancement layer）

基础层独立编码，提供基础视频质量。

增强层依赖基础层，提升视频质量（如分辨率、帧率、信噪比）。

传输顺序：先传输基础层，再传输增强层。

SNR

基础层：对 *DCT* 系数进行coarse粗量化，生成低质量视频。

增强层：对基础层的 *DCT* 系数进行finely精细量化，提升信噪比（*SNR*）。

空间

基础层：生成低分辨率reduced resolution视频比特流。

增强层：结合基础层，重建原始分辨率图像。

时间

输入视频按时间分片，基础层处理一半帧率（如 15 fps）。

增强层处理另一半帧率（如 30 fps），组合后恢复原始帧率。

混合

组合方式：任意两种可扩展性组合（如空间+时间、SNR+空间）。

典型结构：三层架构

基础层（Base layer）

增强层 1（Enhancement Layer 1）

增强层 2（Enhancement Layer 2）

数据分区

基础分区：包含低频 *DCT* 系数（关键信息）。

增强分区：包含高频 *DCT* 系数（细节信息）。

优势：适用于噪声信道，优先保障基础分区传输。

优势

特性	MPEG-1	MPEG-2
画面质量	较低	更高
扫描方式	仅渐进扫描	支持渐进+隔行
可扩展性	不支持	支持
抗误码能力	较弱	更强
色度子采样	仅 4 : 2 : 0	支持 4 : 2 : 2 和 4 : 4 : 4
视频格式灵活性	有限	支持 DVD/HDTV 多种格式

MPEG-4

技术特点：

MPEG-1和MPEG-2采用**基于块的编码技术**。

MPEG-4采用**基于对象object-based**的编码，对数字视频合成、操作、索引和检索有益。

关键参数：

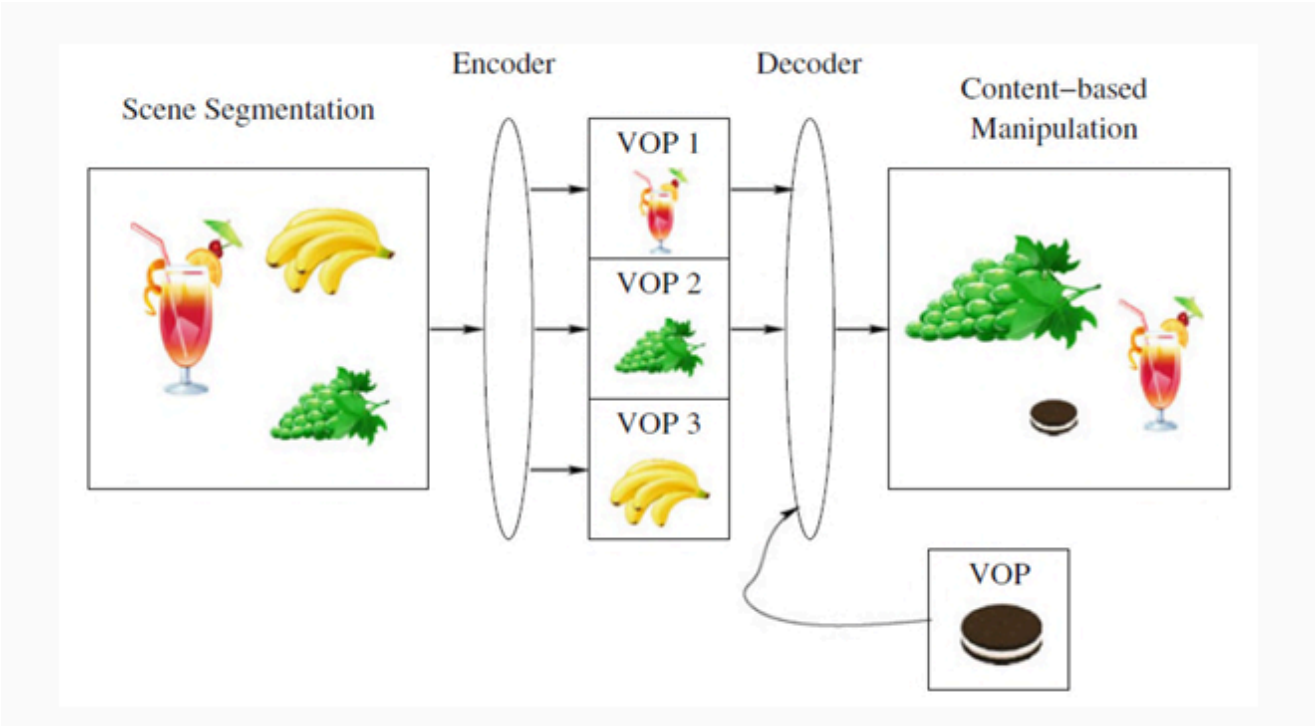
1998年10月完成标准化。

比特率范围：**5 kbps到10 Mbps**。

提供**基于视频对象的分层场景描述**。

支持**媒体对象合成**以创建视听场景。

为媒体数据实体提供**比特流复用和同步**。



优势

集成自然和合成内容，以**对象形式**表示。

支持**2D/3D内容**及用户交互性。

编码率范围广：

语音：**2 Kbit/s**

视频：**5 Kbit/s**（低质量）至 **5 Mbit/s**（透明质量）

音频：**64 Kbit/s/声道**（CD质量）

支持**知识产权管理与保护**。

MPEG-7

目标：基于audiovisual content-音频视觉内容的检索。

应用场景：多媒体内容创建与消费。

关键信息：

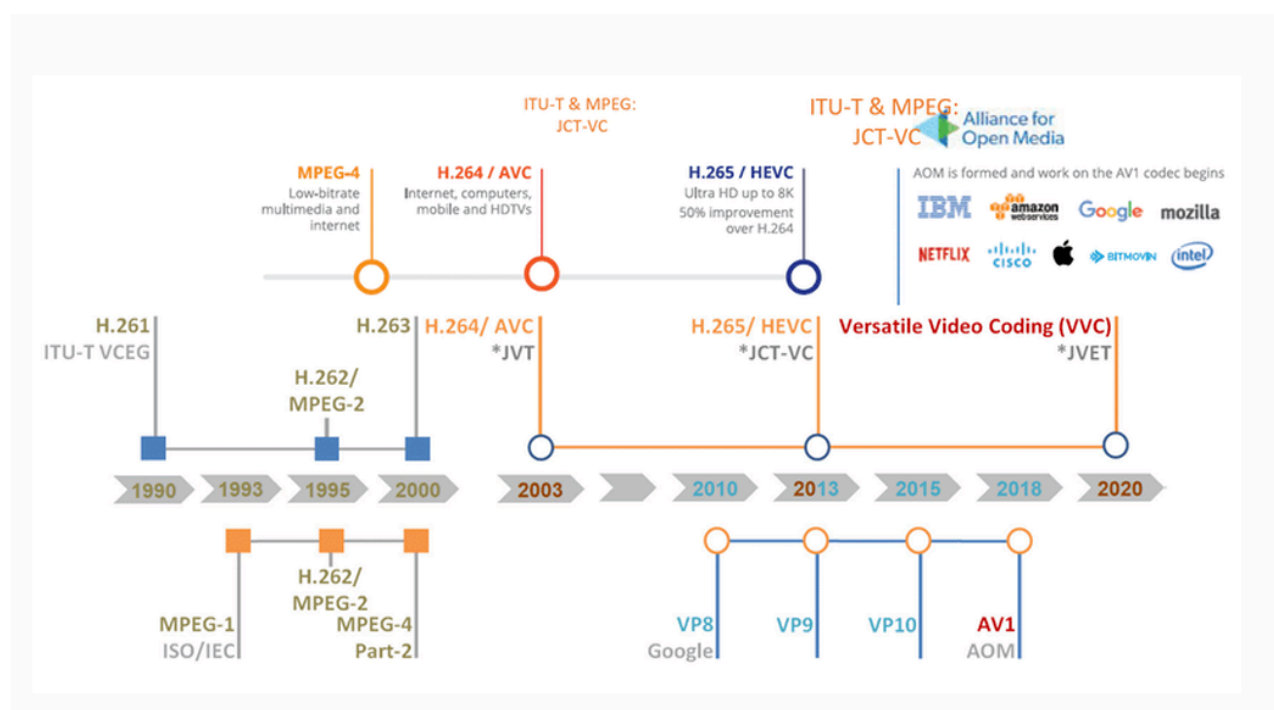
2001年9月发布。

正式名称：**Multimedia Content Description Interface**。

不是视频编码标准，而是内容描述接口标准。

H.26x

时间线



H.261

只有**IP帧**

由ITU-T于**1990年**开发，专为视频会议设计。

采用**运动补偿(MC)**为基础的压缩原理。

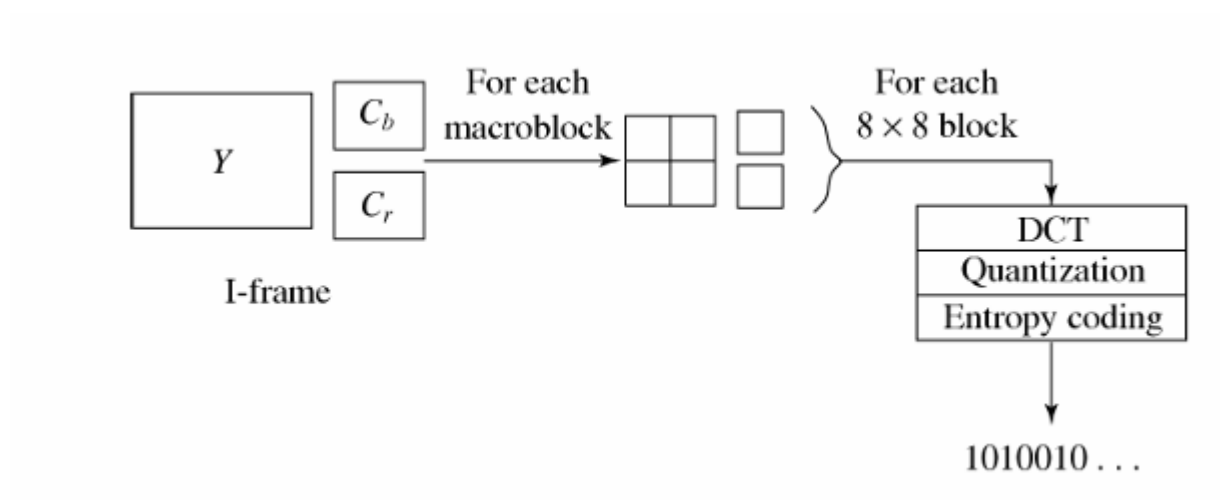
支持**YUV 4:2:0**视频格式。

帧类型：

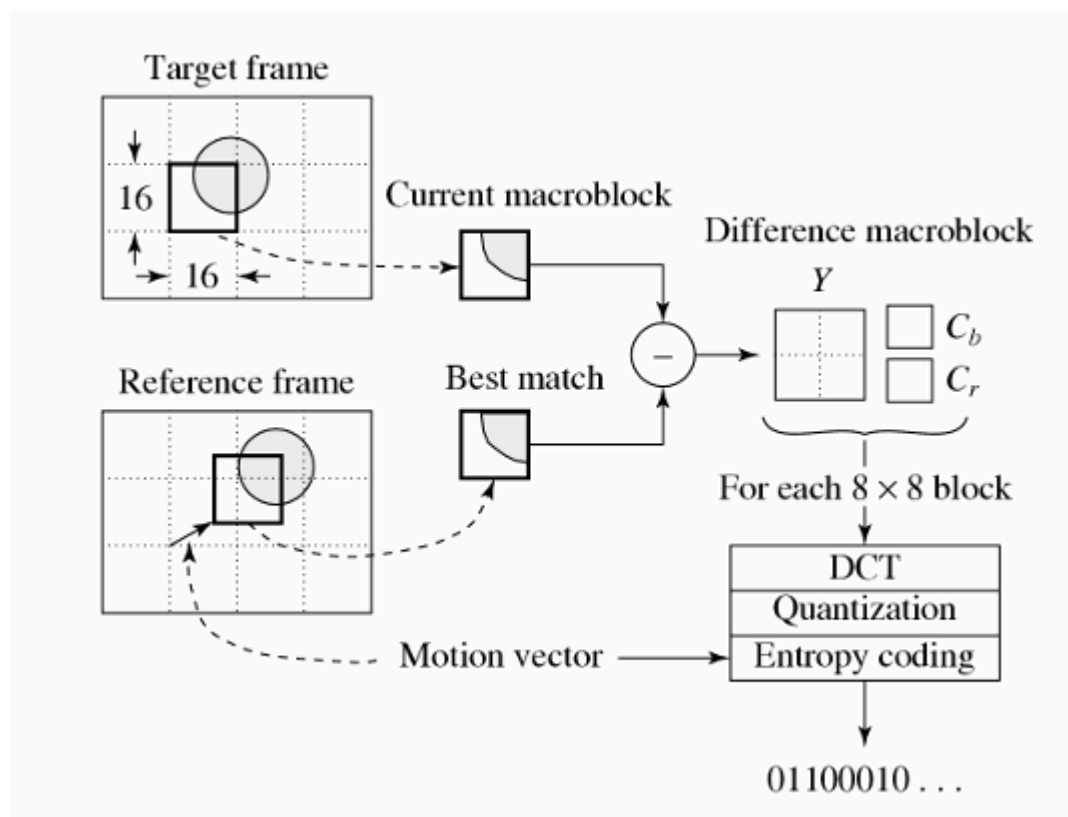
I帧（帧内编码）：独立图像。

P帧（帧间编码）：基于前一个I帧或P帧。

I帧编码



P帧编码



量化机制

DC系数（帧内模式）：

$$QDCT = \text{round} \left(\frac{DCT}{\text{step_size}} \right) = \text{round} \left(\frac{DCT}{8} \right)$$

其他系数：

$$QDCT = \left\lfloor \frac{DCT}{\text{step_size}} \right\rfloor = \left\lfloor \frac{DCT}{2^{\text{scale}}} \right\rfloor$$

scale：范围 [1, 31] 的整数

H.262

IPB帧都有

开发背景：由ITU-T VCEG和ISO/IEC MPEG联合开发。

标准化：1995年作为**MPEG-2 Part 2 (ISO/IEC 13818-2)**。

帧类型：

I帧：帧内编码

P帧：预测编码

B帧：双向预测编码

H.263 与 H.263+

H.263 主要针对低比特率视频会议设计，是后来的 MPEG-4 Part 2 的发展基础。

标准演进：H.263 (1996年) -> H.263+ (1998年)。

核心改进：

相比 H.261，在更低的比特率下提供了更好的图像质量。

H.263+：支持亚像素运动向量（精度达 1/8 像素），增强了压缩效率。

灵活性：H.263+ 增加了对比特率、分辨率和宽高比的适应性，并支持可伸缩性（SNR、空间、时间）。

环路滤波：引入**去块效应滤波器**（Deblocking Filter）以减少方块效应。

Video Formats Supported by H.263

Video format	Luminance image resolution	Chrominance image resolution	Bit-rate (Mbps) (if 30 fps and uncompressed)	Bit-rate (kbps) BPPmaxKb (compressed)
sub-QCIF	128 × 96	64 × 48	4.4	64
QCIF	176 × 144	88 × 72	9.1	64
CIF	352 × 288	176 × 144	36.5	256
4CIF	704 × 576	352 × 288	146.0	512
16CIF	1,408 × 1,152	704 × 576	583.9	1024

H.264 (Advanced Video Coding, AVC)

H.264 是目前应用最广泛的标准之一，旨在提高编码效率并适应从移动设备到 HDTV、蓝光等多种应用场景。

高压缩率：比 H.262 提高约 50%，比 H.263 提高约 30%。

运动补偿：

多尺寸块：支持从 16 × 16 到 4 × 4 的可变块大小。

多参考帧：支持从多个之前的帧中进行运动估计。

变换与编码：

采用 4 × 4 整数变换（**低复杂度complexity，无漂移问题drifting**）。

熵编码：支持 CAVLC（上下文自适应变长编码）和 CABAC（上下文自适应二进制算术编码）。

帧内预测：支持方向性空间预测，利用相邻像素进行编码。

运动补偿计算 (Motion Compensation)

H.264 使用 1/4 像素精度的运动补偿，通过插值算法实现。

半像素位置 (Half-pixel)：使用 6 抽头滤波器（6-tap filter）计算。

水平半像素 b_1 计算公式：

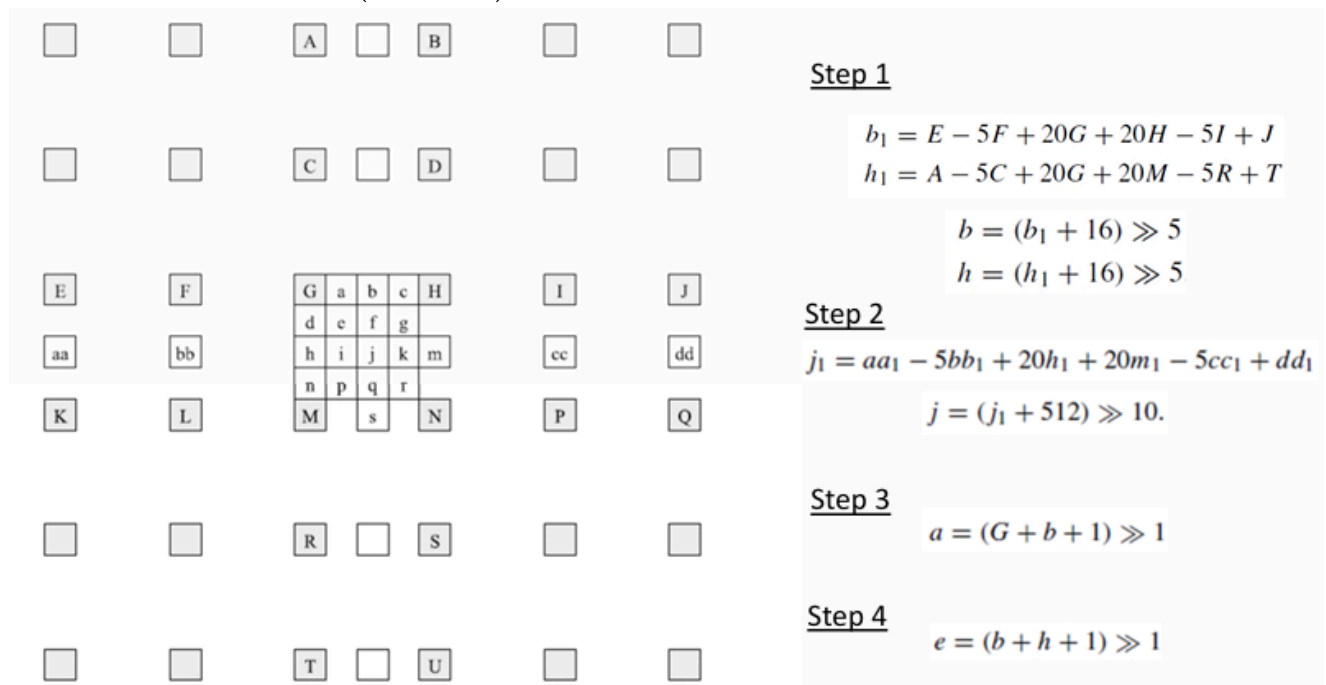
$$b_1 = E - 5F + 20G + 20H - 5I + J$$

最终半像素值： $b = (b_1 + 16) \gg 5$ （即 $\text{round}(b_1/32)$ ）。

1/4 像素位置 (Quarter-pixel)：利用已知像素和半像素值进行线性插值。

例如位置 a 的计算： $a = (G + b + 1) \gg 1$ 。

例如位置 e 的计算： $e = (b + h + 1) \gg 1$ 。



GOP增强选项

无 B 帧模式 (No B-frames)

适用场景：低延迟应用（视频会议、远程手术、VR 实时渲染）；

优势：

消除 B 帧带来的解码延迟（无需缓存未来帧）；

减少存储开销（无双向预测依赖）；

代价：

压缩效率下降约 10–15%（相比含 B 帧 GOP）；

仅支持 I/P 帧 → 随机访问点更密集。

多参考帧 (Multiple Reference Frames)

H.264 允许 P/B 帧参考 **最多 16 个历史帧**（由 `num_ref_frames` 控制）；

收益：

更精准运动补偿（尤其对遮挡、重复纹理场景）；

提升压缩率 5–10%；

代价：

DPB（Decoded Picture Buffer）内存需求↑；

运动估计计算量↑（需搜索更多参考帧）；

编码复杂度显著增加。

整数变换（Integer Transform）

浮点 DCT 在编/解码端因舍入误差累积 → 预测残差失真传播（prediction drift）；

H.264 改用 **可逆整数变换**，确保：

编码端： $\hat{F} = \text{round}(H \cdot f \cdot H^T)$

解码端： $\tilde{f} = \text{round}(H_{\text{inv}} \cdot \hat{F} \cdot H_{\text{inv}}^T)$

且 $\tilde{f} = f$ （无漂移，当量化参数 QP=0 时严格相等）。

从标准 DCT 矩阵 T_4 出发：

$$T_4 = \begin{bmatrix} a & a & a & a \\ b & c & -c & -b \\ a & -a & -a & a \\ c & -b & b & -c \end{bmatrix}, \quad \begin{cases} a = \frac{1}{2} \\ b = \sqrt{\frac{1}{2}} \cos \frac{\pi}{8} \\ c = \sqrt{\frac{1}{2}} \cos \frac{3\pi}{8} \end{cases}$$

$H = \text{round}(\alpha \times T_4)$

缩放并取整： 令 $\alpha = 2.5$ ，定义整数矩阵：

$$H = \text{round}(\alpha \cdot T_4) = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 2 & 1 & -1 & -2 \\ 1 & -1 & -1 & 1 \\ 1 & -2 & 2 & -1 \end{bmatrix}$$

正交性： H 是近似正交矩阵（非严格正交），满足：

$$H \cdot H^T = \begin{bmatrix} 4 & 0 & 0 & 0 \\ 0 & 10 & 0 & 0 \\ 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 10 \end{bmatrix} = D \quad (\text{对角矩阵})$$

为保证无漂移，定义：

$H_{\text{inv}} = (H^T \cdot D)^{-1}$ 或更直接地： $H_{\text{inv}} = H^T \cdot$ 归一化因子

$$H_{\text{inv}} = \begin{bmatrix} 1 & 1 & 1 & \frac{1}{2} \\ 1 & \frac{1}{2} & -1 & -1 \\ 1 & -\frac{1}{2} & -1 & 1 \\ 1 & -1 & 1 & -\frac{1}{2} \end{bmatrix}$$

H.264 为了减少计算量并避免浮点数运算产生的误差，采用了整数变换和缩放量化结合的方法。

$$\hat{\mathbf{F}} = \text{round} \left[(\mathbf{H} \times \mathbf{f} \times \mathbf{H}^T) \cdot \mathbf{M}_f / 2^{15} \right]$$

$\mathbf{f}$: 4×4 的输入残差矩阵（原始图像像素与预测像素之差）。

$\mathbf{H}$: 整数变换矩阵（类似于离散余弦变换 DCT，但全是整数）。

$\mathbf{M}_f$: 量化乘法矩阵（Quantization Matrix），其值取决于量化参数 QP 。

For $0 \leq QP < 6$, we have

$$\mathbf{M}_f = \begin{bmatrix} m(QP, 0) & m(QP, 2) & m(QP, 0) & m(QP, 2) \\ m(QP, 2) & m(QP, 1) & m(QP, 2) & m(QP, 1) \\ m(QP, 0) & m(QP, 2) & m(QP, 0) & m(QP, 2) \\ m(QP, 2) & m(QP, 1) & m(QP, 2) & m(QP, 1) \end{bmatrix}$$

For $QP \geq 6$, each element $m(QP, k)$ is replaced by $m(QP \% 6, k) / 2^{\lfloor QP/6 \rfloor}$.

The matrix $\mathbf{m}$ used to generate $\mathbf{M}_f$

QP	Positions in $\mathbf{M}_f$ (0, 0), (0, 2) (2, 0), (2, 2)	Positions in $\mathbf{M}_f$ (1, 1), (1, 3) (3, 1), (3, 3)	Remaining $\mathbf{M}_f$ positions
0	13107	5243	8066
1	11916	4660	7490
2	10082	4194	6554
3	9362	3647	5825
4	8192	3355	5243
5	7282	2893	4559

$$\tilde{\mathbf{f}} = \text{round} \left[(\mathbf{H}_{inv} \times (\hat{\mathbf{F}} \cdot \mathbf{V}_i) \times \mathbf{H}_{inv}^T) / 2^6 \right]$$

V_i : 反量化乘法矩阵。

For $0 \leq QP < 6$,

$$V_i = \begin{bmatrix} v(QP, 0) & v(QP, 2) & v(QP, 0) & v(QP, 2) \\ v(QP, 2) & v(QP, 1) & v(QP, 2) & v(QP, 1) \\ v(QP, 0) & v(QP, 2) & v(QP, 0) & v(QP, 2) \\ v(QP, 2) & v(QP, 1) & v(QP, 2) & v(QP, 1) \end{bmatrix}$$

For $QP \geq 6$, each element $v(QP, k)$ is replaced by $v(QP \% 6, k) \cdot 2^{\lfloor QP/6 \rfloor}$.

The matrix v used to generate V_i

QP	Positions in V_i (0, 0), (0, 2) (2, 0), (2, 2)	Positions in V_i (1, 1), (1, 3) (3, 1), (3, 3)	Remaining V_i positions
0	10	16	13
1	11	18	14
2	13	20	16
3	14	23	18
4	16	25	20
5	18	29	23

QP的影响

QP越大量化越大误差越大

Integer Transform and Quantization

$$\hat{F} = \text{round} \left[(\mathbf{H} \times \mathbf{f} \times \mathbf{H}^T) \cdot \mathbf{M}_f / 2^{15} \right]$$

72	82	85	79	13107	8066	13107	8066	507	-12	-2	2
74	75	86	82	8066	5243	8066	5243	0	-7	-14	5
84	73	78	80	13107	8066	13107	8066	2	0	-8	-11
77	81	76	84	8066	5243	8066	5243	-1	8	4	3
f				M_f				$\hat{F}$			

Inverse Integer Transform
and De-Quantization

$$\tilde{f} = \text{round} \left[(\mathbf{H}_{\text{inv}} \times (\hat{F} \cdot \mathbf{V}_i) \times \mathbf{H}_{\text{inv}}^T) / 2^6 \right]$$

10	13	10	13	72	82	85	79	0	0	0	0
13	16	13	16	74	75	86	82	0	0	0	0
10	13	10	13	84	73	78	80	0	0	0	0
13	16	13	16	77	81	76	84	0	0	0	0
V_i				$\tilde{f}$				$\epsilon = f - \tilde{f}$			

(a) QP = 0

72	82	85	79	6554	4033	6554	4033	254	-6	-1	1
74	75	86	82	4033	2622	4033	2622	0	-4	-7	3
84	73	78	80	6554	4033	6554	4033	1	0	-4	-6
77	81	76	84	4033	2622	4033	2622	0	4	2	1
$\mathbf{f}$				$\mathbf{M}_f$				$\hat{\mathbf{F}}$			
20	26	20	26	72	82	85	79	0	0	0	0
26	32	26	32	74	75	86	82	0	0	0	0
20	26	20	26	84	74	78	80	0	-1	0	0
26	32	26	32	77	82	76	84	0	-1	0	0
$\mathbf{V}_i$				$\tilde{\mathbf{f}}$				$\epsilon = \mathbf{f} - \tilde{\mathbf{f}}$			

(b) QP = 6.

72	82	85	79	410	252	410	252	16	0	0	0
74	75	86	82	252	164	252	164	0	0	0	0
84	73	78	80	410	252	410	252	0	0	0	0
77	81	76	84	252	164	252	164	0	0	0	0
$\mathbf{f}$				$\mathbf{M}_f$				$\hat{\mathbf{F}}$			
320	416	320	416	80	80	80	80	-8	2	5	-1
416	512	416	512	80	80	80	80	-6	-5	6	2
320	416	320	416	80	80	80	80	4	-7	-2	0
416	512	416	512	80	80	80	80	-3	1	-4	4
$\mathbf{V}_i$				$\tilde{\mathbf{f}}$				$\epsilon = \mathbf{f} - \tilde{\mathbf{f}}$			

(c) QP = 30

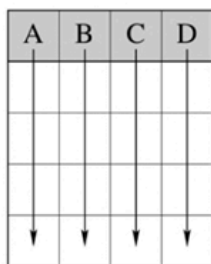
帧内编码 (Intra Coding)

宏块不依赖其他帧，但可利用**已重建的邻域像素**进行预测；

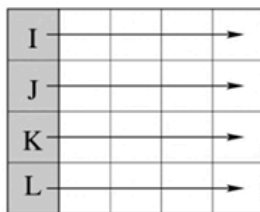
目标：最小化预测残差 $\epsilon = \text{block} - \text{prediction}$ ；

选择使 $\sum |\epsilon|$ 最小的预测模式。

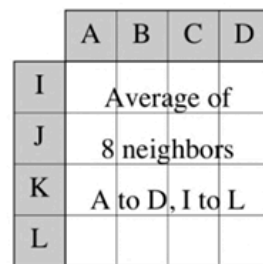
Mode 0: Vertical



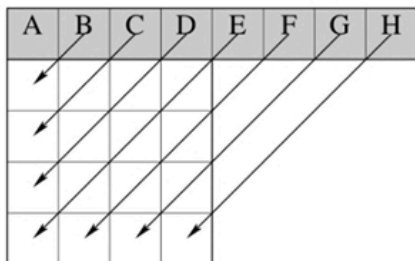
Mode 1: Horizontal



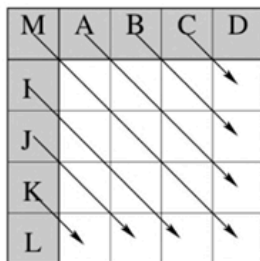
Mode 2: DC



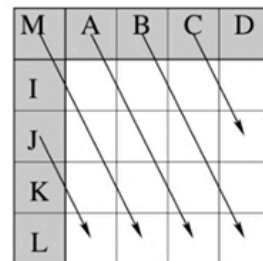
Mode 3: Diagonal Down-Left



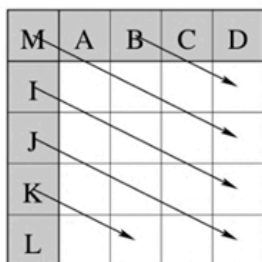
Mode 4: Diagonal Down-Right



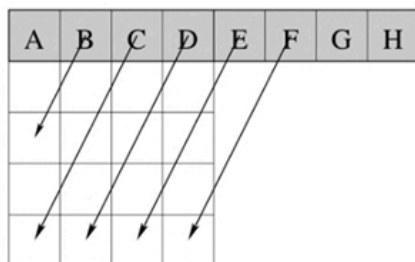
Mode 5: Vertical-Right



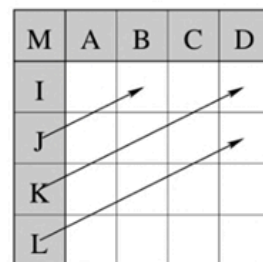
Mode 6: Horizontal-Down



Mode 7: Vertical-Left



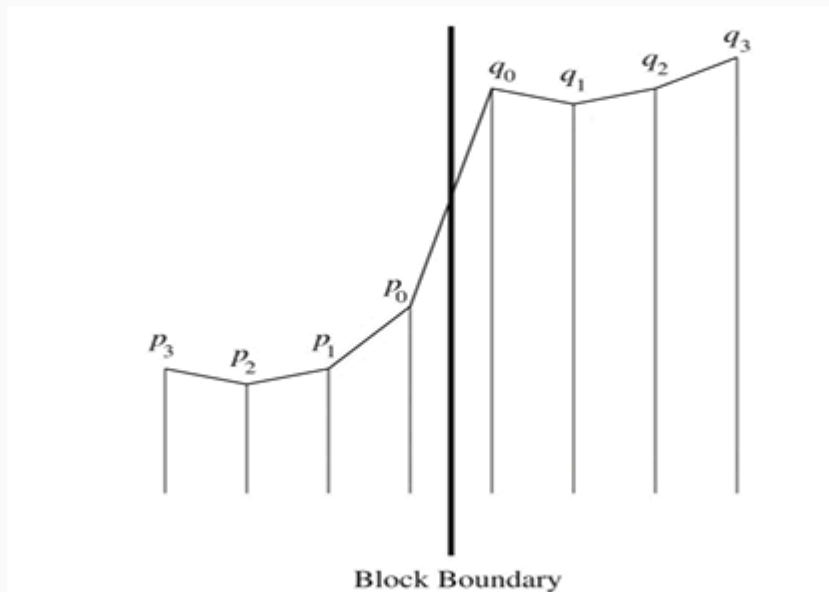
Mode 8: Horizontal-Up



9 种预测模式

模式编号	名称	预测方向
0	Vertical	上方像素向下复制
1	Horizontal	左侧像素向右复制
2	DC	邻域平均值填充
3	Diagonal Down-Right	左上→右下对角线插值
4	Diagonal Down-Left	右上→左下对角线插值
5	Vertical-Right	垂直+右偏
6	Horizontal-Down	水平+下偏
7	Vertical-Left	垂直+左偏
8	Horizontal-Up	水平+上偏

环路去块滤波 (In-Loop Deblocking Filtering)



背景: 基于块的编码会导致块效应 (blocking artifacts)。

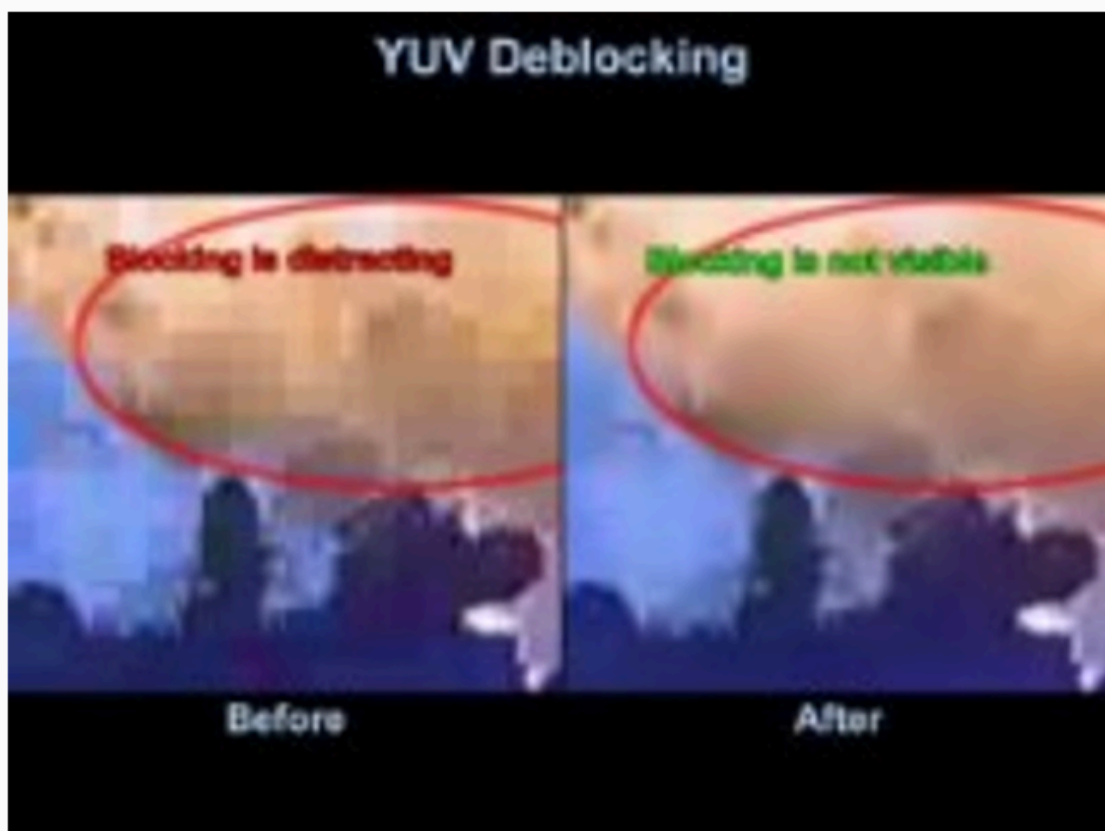
目的: 环路去块滤波器旨在消除这些块效应，提高视频的主观质量。

机制:

应用一种信号自适应的去块滤波器于 4×4 的块边缘。

例如，“4抽头滤波”(4-tap filtering) 会对 p_1, p_0, q_0, q_1 的值进行加权平均，以生成新的 p_0 或 q_0 。

滤波器的长度、强度和类型（去块/平滑）经过设计，确保在消除块效应的同时不扭曲视觉特征。



上下文自适应可变长编码 (CAVLC)

对比: 之前的视频编码标准（如 MPEG-2）使用固定 VLC。

原理:

在 CAVLC 中，为每种数据类型预定义了多个 VLC 表。

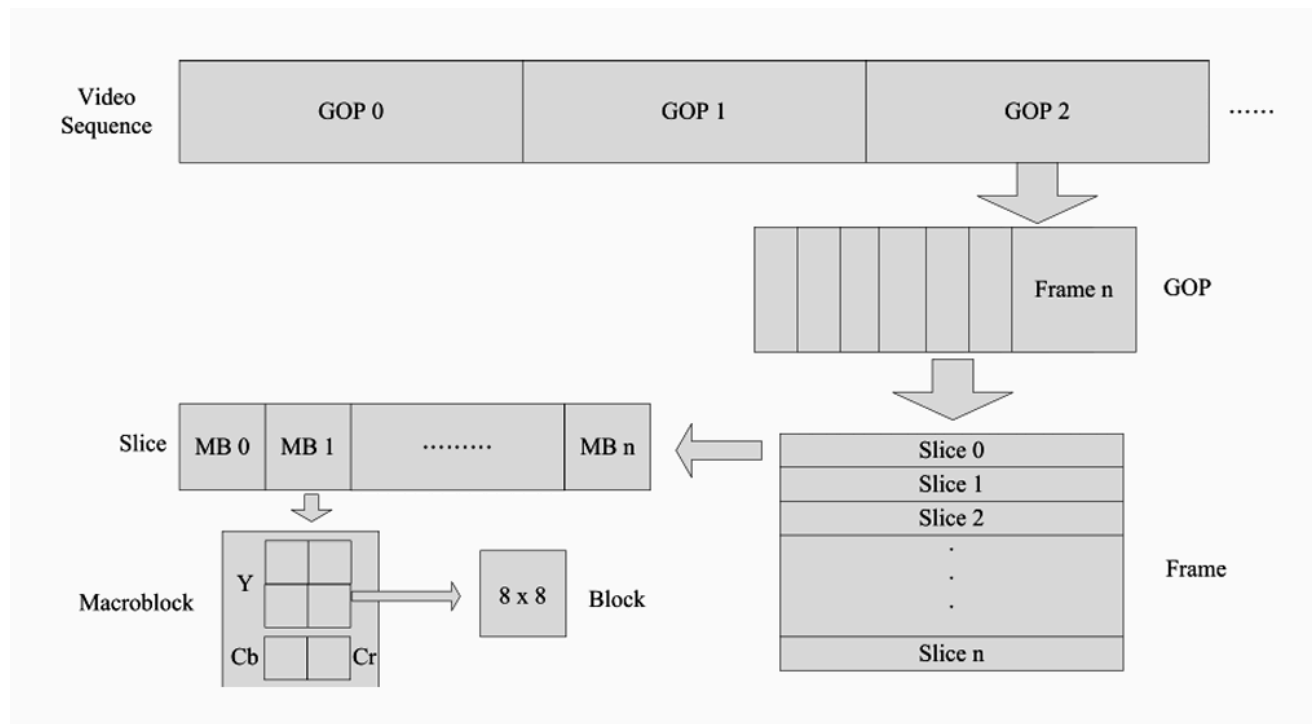
预定义的规则根据上下文（例如来自块数据的参数）预测最佳的 VLC 表。

上下文自适应二进制算术编码 (CABAC)

目的: 为了在 H.264 Main 和 High 配置文件中实现高编码效率。

特点: 高复杂度，实现细节极其庞大。

码流结构 (Bitstream Structure)



配置文件 (Profiles)

Feature	MPEG-2 (MP@ML)	MPEG-4 (ASP)	H.264		
			Baseline	Main	Extended
Block size	16 x 16	16 x 16	Variable: 16 x 16 to 4 x 4		
Frequency transform	8 x 8 DCT	8 x 8 DCT	Hierarchical bit-exact 4 x 4 integer DCT		
1/4-pixel motion vector resolution	No	Yes	Yes		
In-loop deblocking	No	No	Yes		
Reference frames for block prediction	Up to 2	Up to 2	Can be more than 2		
Motion vector prediction	No	No	Yes		
Intra-frame prediction	No	Yes	Yes, several directional spatial modes		
Weighted prediction	No	No	No	Yes	Yes
Entropy coding	Huffman	Huffman	CAVLC	CAVLC/CABAC	CAVLC
Interlaced video support	Yes	Yes	No	Yes	No
Frame/slice types	I, P, B	I, P, B	I, P	I, P, B	I, P, B
Slice groups and Arbitrary Slice Order	No	No	Yes	No	Yes
Redundant slices	No	No	Yes	No	Yes
Switching slices (SP & SI)	No	No	No	No	Yes
Implementation complexity	Low	Low	Medium	High	Medium
Computational requirements	Low	Low	Medium	Very high	High

Table 1 Comparison of some of the key features of H.264 to MPEG-2 and MPEG-4. Refer to 'Squeeze play: how video compression works' in the March 2004 edition of *Inside DSP* for useful background and definitions of terms used in this table.

主要类型: 包含 4 种不同的配置文件: Baseline, Extended, Main, High。

应用场景: 涵盖从移动设备到广播 HDTV 的各种应用。

特性对比:

Baseline: 主要用于低复杂度应用 (如视频会议), 不支持 B 帧和 CABAC。

Main: 支持 B 帧和 CABAC, 广泛用于数字电视。

High: 支持 4:2:0 以上色度格式, 支持 Fidelity Range Extensions, 用于蓝光等高保真应用。

可伸缩视频编码 (SVC)

功能: 为具有不同带宽的网络提供码流的可伸缩性。

类型: 提供时间可伸缩性、空间可伸缩性、质量可伸缩性及其可能的组合。

优势: 与之前的标准相比, 编码效率大大提高。

多视图视频编码 (MVC)

定义: 多视图视频编码是一个新兴领域。

应用: 具有潜在的新应用领域, 如自由视点视频 (FVV), 用户可以在其中指定他们喜欢的视角。

H.265

全称: H.265/HEVC (High Efficiency Video Coding, 高效视频编码)。

开发组织: 由视频编码联合协作组 (JCT-VC) 开发, 成员来自 ITU-T VCEG 和 ISO/IEC

MPEG。

标准化: 2013 年标准化 (ITU-T H.265 / MPEG-H Part 2)。

目标:

由于分辨率增加 (例如 UHD)，提高编码效率。

通过利用并行处理加速更复杂的方法。

现状: H.264 和 H.265 是许多视频应用的当前选择。

改进

支持规格: 支持高达 8K 分辨率的 UltraHD 视频，帧率高达 120 fps。

灵活性: 预测模式和变换块大小具有更大的灵活性。

滤波: 更复杂的插值和去块滤波器。

环路去块滤波器尺寸：H.265 为 8×8 ，而 H.264 为 4×4 。

处理: 支持并行处理。

效率: 在相同画质下，相比 H.264 更节省码率。

主要特性 (Main Features)

结构变化: 宏块结构被各级别和大小的编码单元 (Coding Units) 的四叉树 **quadtree** 结构取代。

运动矢量: 改进了插值方法，实现运动矢量的四分之一像素精度。

帧内预测: 扩展了方向性空间预测 (33 个角度方向) 用于帧内编码。

帧内编码 (Intra-Coding)

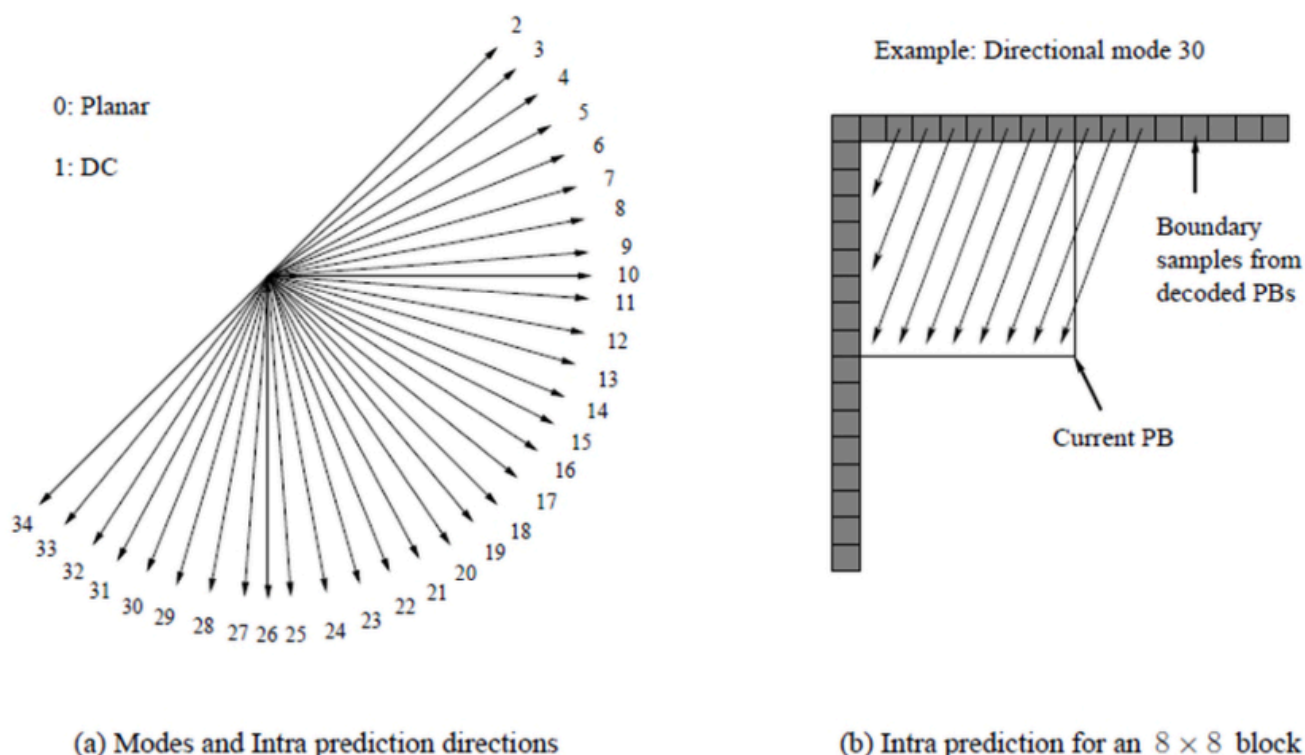


Fig. 12.16 H.265 Intra prediction

预测模式:

包含 Planar 模式 (0) 和 DC 模式 (1)。

包含 33 种角度方向模式 (2-34)。
示例: 展示了 8×8 块的帧内预测，利用边界样本（Boundary samples from decoded PUs）预测当前 PU（Current PB）。

H.266 / VVC (Versatile Video Coding)

作为最新一代视频编码标准，H.266 是为了应对 8K、16K 及 VR 时代的到来。
超高压缩率： 在同等感知画质下，比 H.265 (HEVC) 节省约 **50%** 的数据量。
超高清支持： 原生支持最高 **16K** 分辨率。
全场景适配：
动态范围： SDR, HDR。
内容类型： 360° 全景视频、屏幕共享内容。
位深 (Bit Depth)： 支持 **10-bit 到 16-bit**，极大缓解了色彩断层（Banding）问题。
采样支持： 兼容从 4:2:0 到 4:4:4 的所有采样模式。

新兴编码&标准

MPEG 沉浸式视频 (MPEG Immersive Video, MIV)

标准归属: 属于 ISO/IEC 23090 MPEG-I 标准的一部分（第 12 部分）。
目的: MIV 正在进行标准化，旨在以数字方式表示沉浸式媒体。
核心功能:
MIV 草案标准支持**六自由度**（*6DoF*）视频，也被称为**体积视频**（volumetric video）。使得用户能够观看由多个摄像机捕捉的、具有 *6DoF* 的沉浸式体积内容。

AI模型&架构

常见 DNN 架构概览

架构类型	全称	核心特点	典型应用
CNN	Convolutional Neural Network	局部感受野 + 权重共享 + 池化 → 平移不变性	图像分类、目标检测、视频理解
RNN	Recurrent Neural Network	隐藏状态循环连接 → 处理序列依赖	时序预测、语言建模、语音识别
Transformer	—	自注意力机制 (Self-Attention) → 并行处理长序列	NLP (BERT/GPT)、ViT (视觉)、多模态
LLM	Large Language Model	超大规模 Transformer + 预训练微调范式	通用对话、代码生成、知识推理
Foundation Models (FM _s)	—	在海量通用数据上预训练，可适配下游任务	CLIP (图文对齐)、Stable Diffusion (生成)、VLM _s

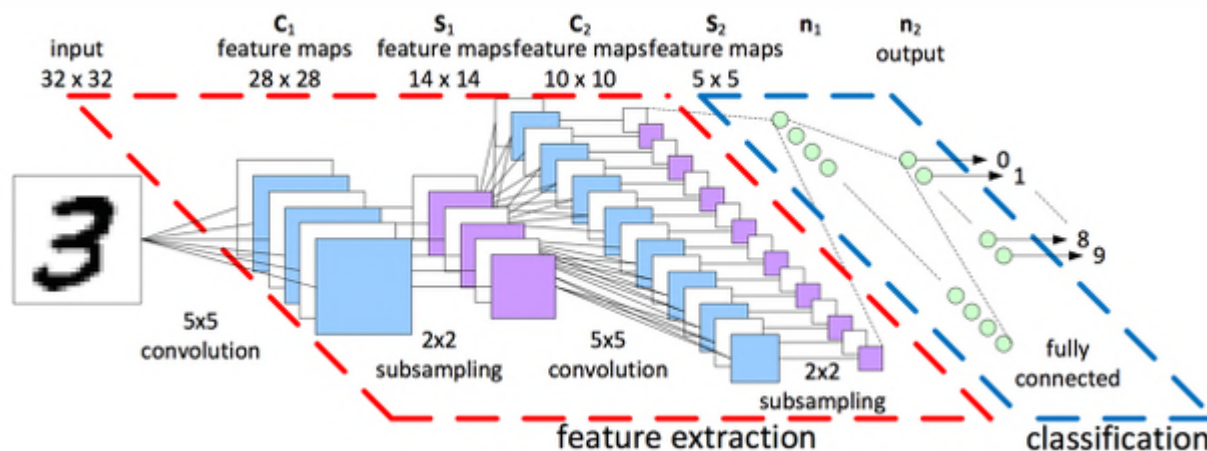
卷积神经网络 (CNN)

核心操作：

卷积层 (Conv)：提取局部特征（如边缘、纹理）；

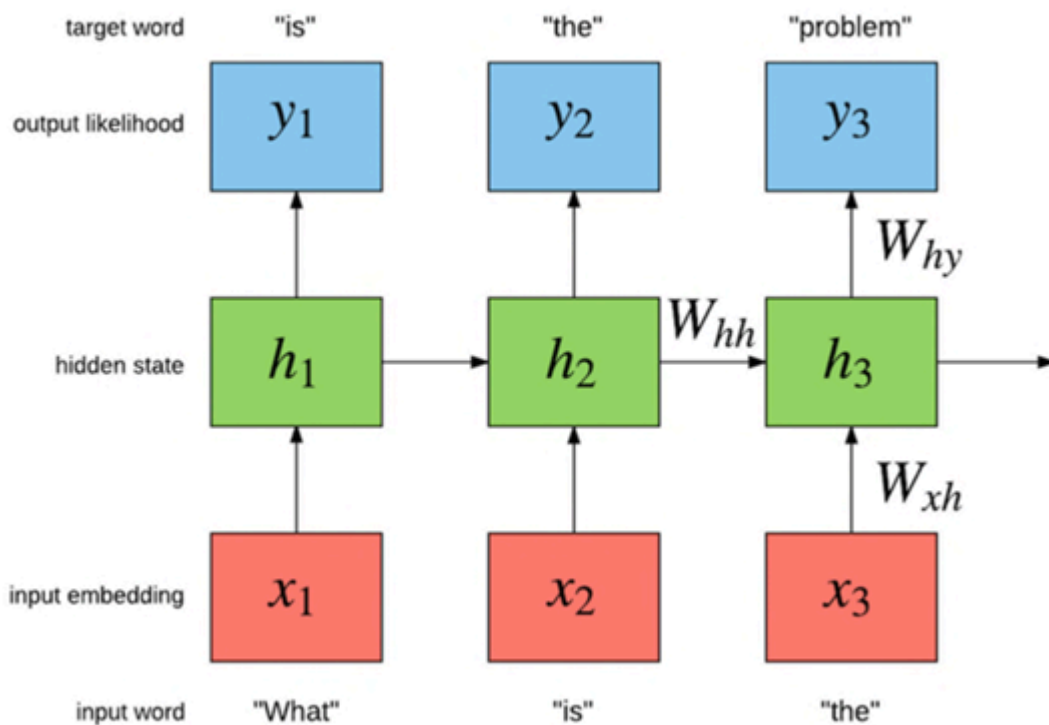
池化层 (Pooling)：下采样，增强平移鲁棒性；

全连接层 (FC)：分类决策。



优势：参数效率高、空间层次化表征能力强。

循环神经网络 (RNN)



结构：隐藏状态 $h_t = \sigma(W_h h_{t-1} + W_x x_t + b)$

变体：

LSTM：引入门控机制（输入门、遗忘门、输出门）缓解梯度消失；

GRU：简化版 LSTM，性能相当但参数更少。

适用场景：股票价格预测、机器翻译、语音合成。

Transformer

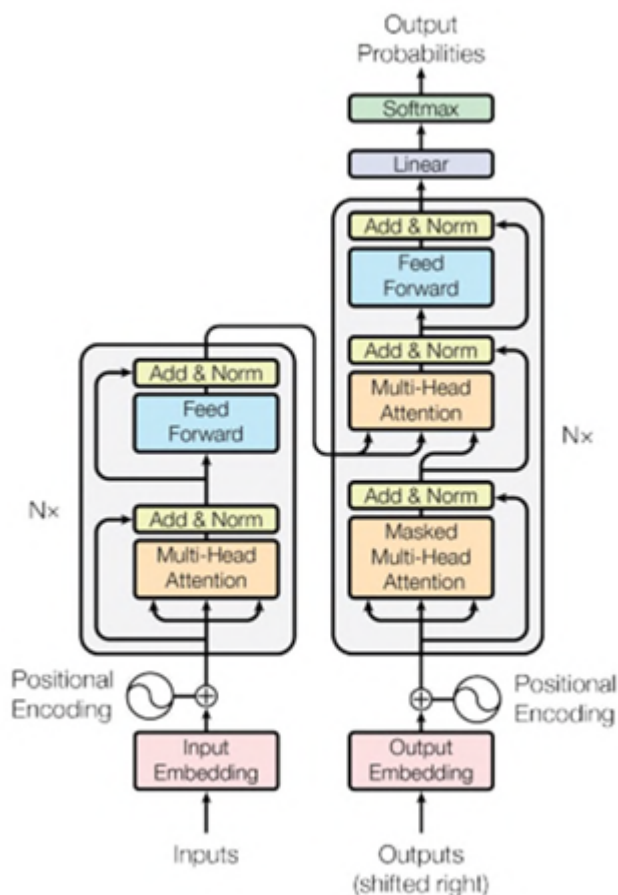


Figure 1: The Transformer - model architecture.

核心创新：自注意力机制（Self-Attention）

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

其中 $Q = XW_Q$, $K = XW_K$, $V = XW_V$ 。

优势：

序列元素间**全局依赖建模**（无距离衰减）；
完全并行计算 → 训练速度远超 RNN；
成为 LLM 和 Vision Transformer (ViT) 基石。

大语言模型（LLMs）



规模特征：

参数量： $10^8 \sim 10^{12}$ （如 GPT-3: 175B）；

训练数据：TB 级文本（Common Crawl, Wikipedia, Books）；

硬件需求：数千 GPU / TPU 集群 + 万级小时训练。

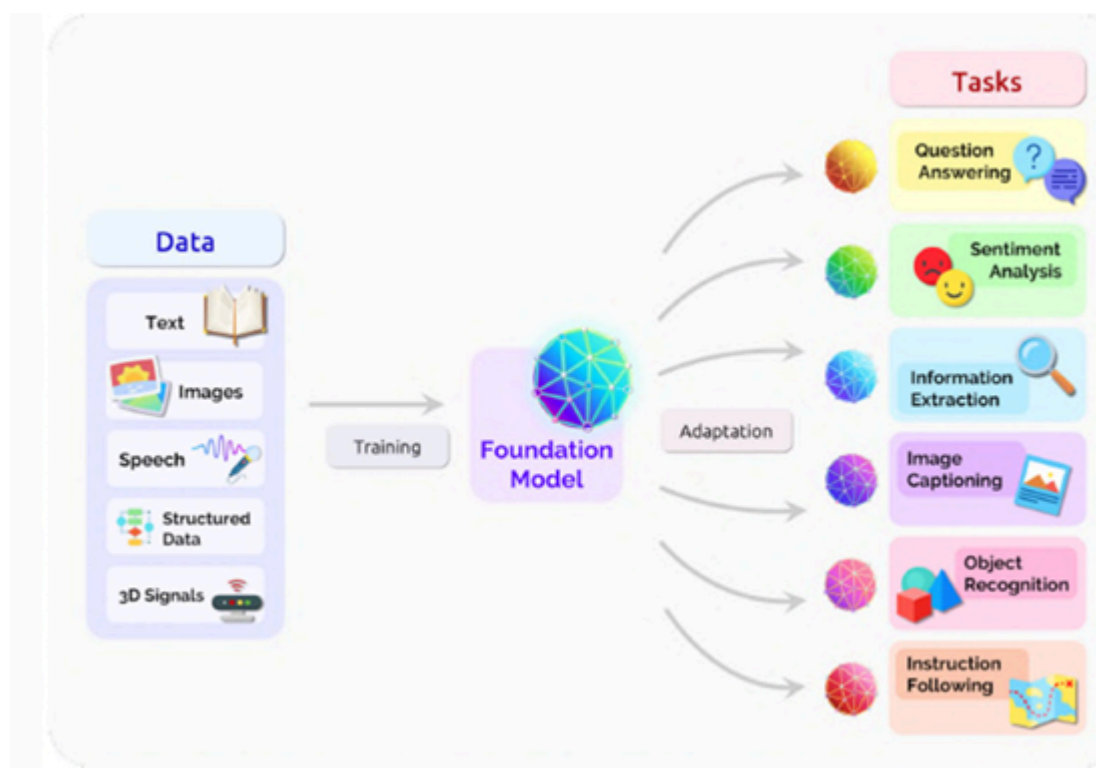
能力涌现（Emergent Abilities）：

少样本学习（Few-shot Learning）；

思维链（Chain-of-Thought）推理；

工具调用（Tool Use）与代码生成。

基础模型（Foundation Models, FMs）



定义：在大规模广义数据上预训练，可通过微调/提示工程（Prompting）适配广泛下游任务。

代表模型：

CLIP：图文对比学习 → 零样本图像分类；

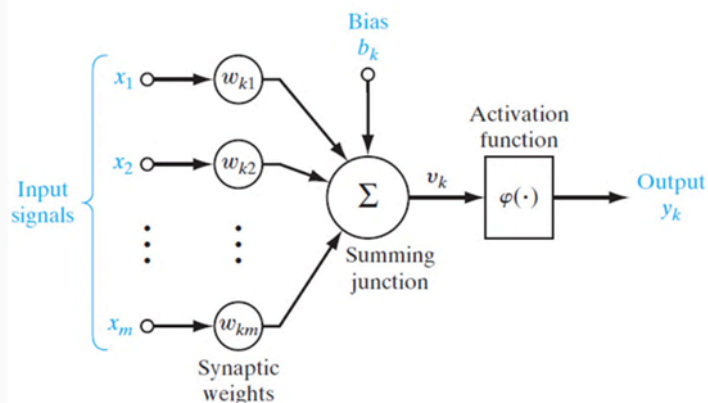
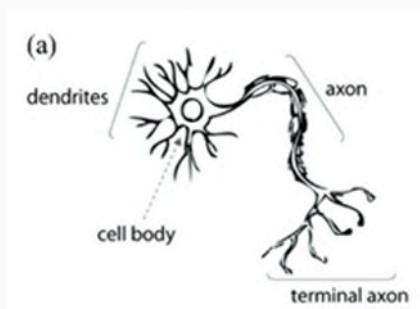
ViLT / Flamingo：多模态基础模型；

SAM（Segment Anything Model）：通用图像分割基础模型。

价值：降低 AI 应用门槛，推动“模型即服务”（MaaS）生态。

线性分类器

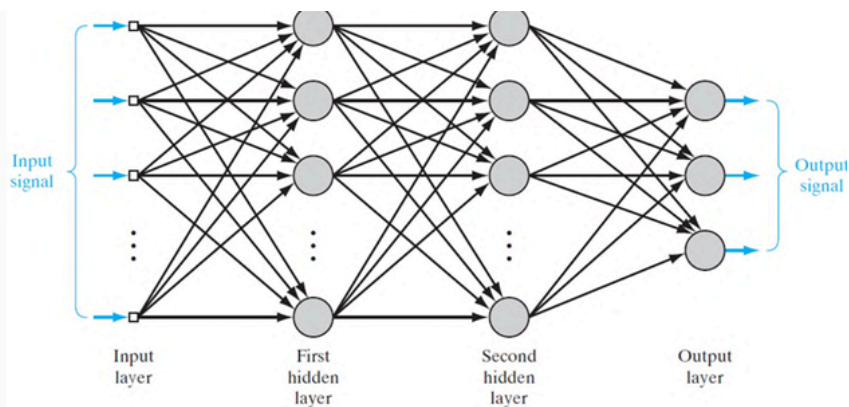
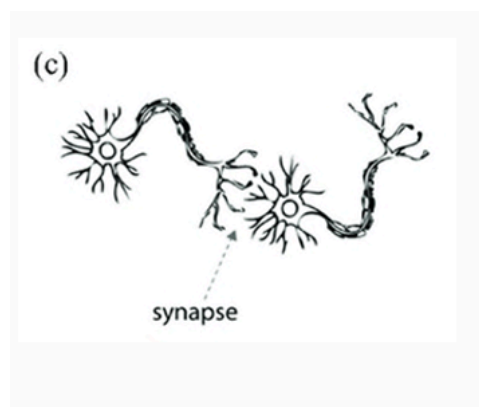
生物神经网络 vs 人工神经网络



生物神经元：树突接收信号 → 细胞体整合 → 轴突发放脉冲；
人工感知机（Perceptron）：

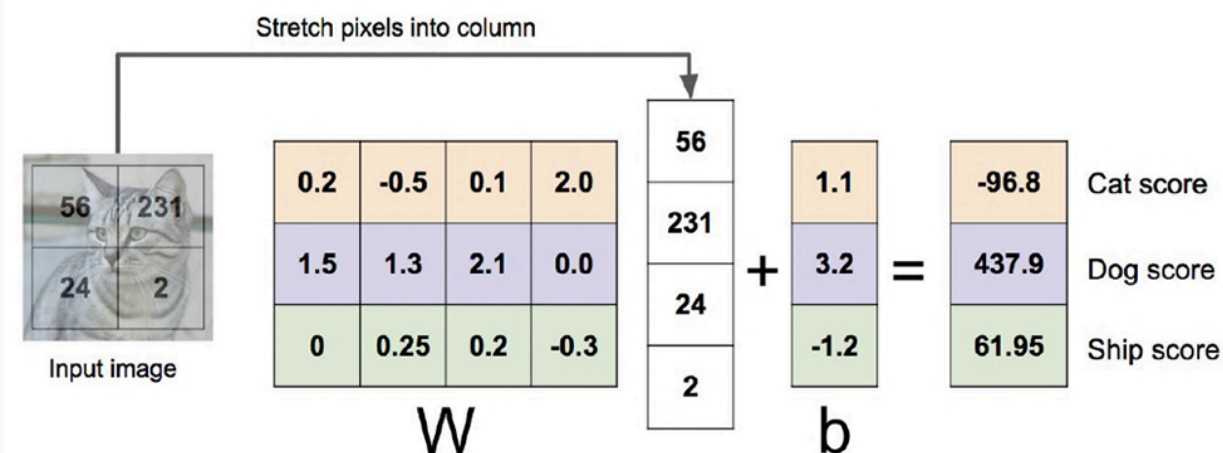
$$y = \sigma \left(\sum_i w_i x_i + b \right), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

多层感知机（MLP）



由多个全连接层堆叠而成；
又称 **前馈神经网络（Feed-Forward Network, FFN）** 或 **稠密网络（Dense Net）**；
表达能力：**万能近似定理** —— 任意连续函数可用 2 层 MLP 逼近。

线性分类



模型： $f(x) = Wx + b$ ，其中 $W \in \mathbb{R}^{C \times D}$ ， $x \in \mathbb{R}^D$ ， C 为类别数；
示例：将 $32 \times 32 \times 3$ 图像展平为 3072 维向量 $\rightarrow W \in \mathbb{R}^{10 \times 3072}$ ；

代数视角：

输入像素拉成列向量 x ；

权重矩阵 W 的每一行对应一个类别的“模板”；

输出分数 $s_c = w_c^\top x + b_c$ ；

如：猫得分 = $[0.2, -0.5, \dots, 2.0] \cdot x + (-96.8)$ 。

损失函数（Loss Function）—— 模型优化的核心驱动力

衡量 **预测值** $f(x_i)$ 与 **真实标签** y_i 之间的“距离”；

用于反向传播更新参数 W, b ；

设计原则：**可微分、凸性/平滑性、符合任务语义**。

回归损失

- Square loss:

$$L(x, y) = \sum_i (y_i - f(x_i))^2$$

- Mean Square Error (MSE):

$$MSE = \frac{1}{N} \sum_i (y_i - f(x_i))^2$$

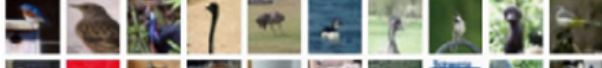
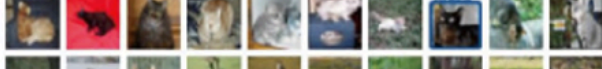
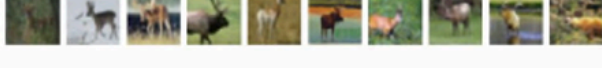
- Mean Absolute Error (MAE):

$$MAE = \frac{1}{N} \sum_i |y_i - f(x_i)|$$

x and y are the input data and target output value, $f(x)$ is the network predicted output.

分类损失

$$p_j = \frac{e^{z_j}}{\sum_k e^{z_k}}$$

		Predicted output Scores (z)	Ground truth Output Labels (y)	Normalized probabilities (p)
airplane		0.7	0	0.109
automobile		0.1	0	0.060
bird		1.6	0	0.269
cat		2.2	1	0.489
deer		0.3	0	0.073

$$L = -\sum_j y_j \log p_j = 0.715 \quad (\text{Note: The log is natural log namely } \ln)$$

23

Softmax + 交叉熵损失 (Cross-Entropy Loss)

首先计算 logits: $z_j = f_j(x)$

Softmax 归一化:

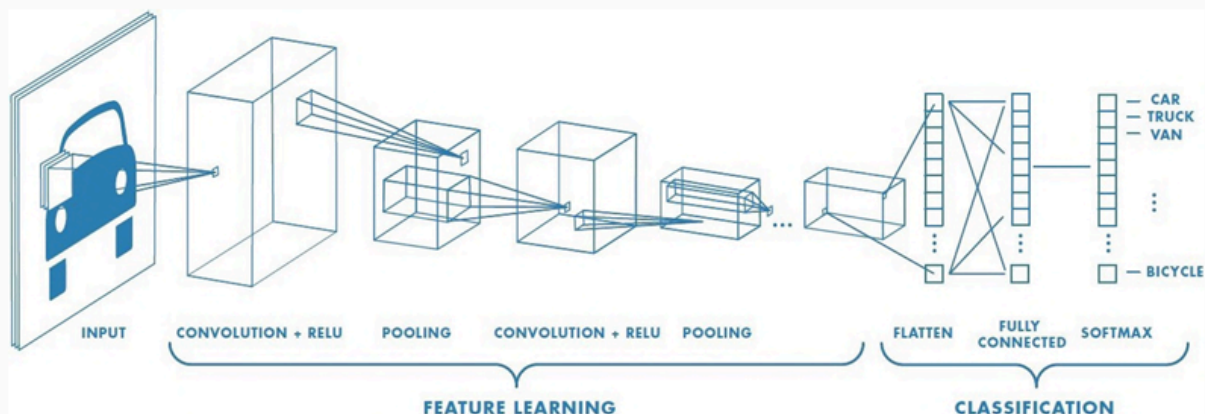
$$p_j = \frac{e^{z_j}}{\sum_k e^{z_k}}, \quad \text{其中 } \sum_j p_j = 1$$

交叉熵损失:

$$L = -\sum_{j=1}^C y_j \log p_j$$

其中 $y_j \in \{0, 1\}$ 为 one-hot 标签 (多分类) 或 $y_j \in [0, 1]$ (软标签)。

CNN



Feature Learning 阶段: 卷积 + 激活 + 池化 → 逐层提取层次化特征;

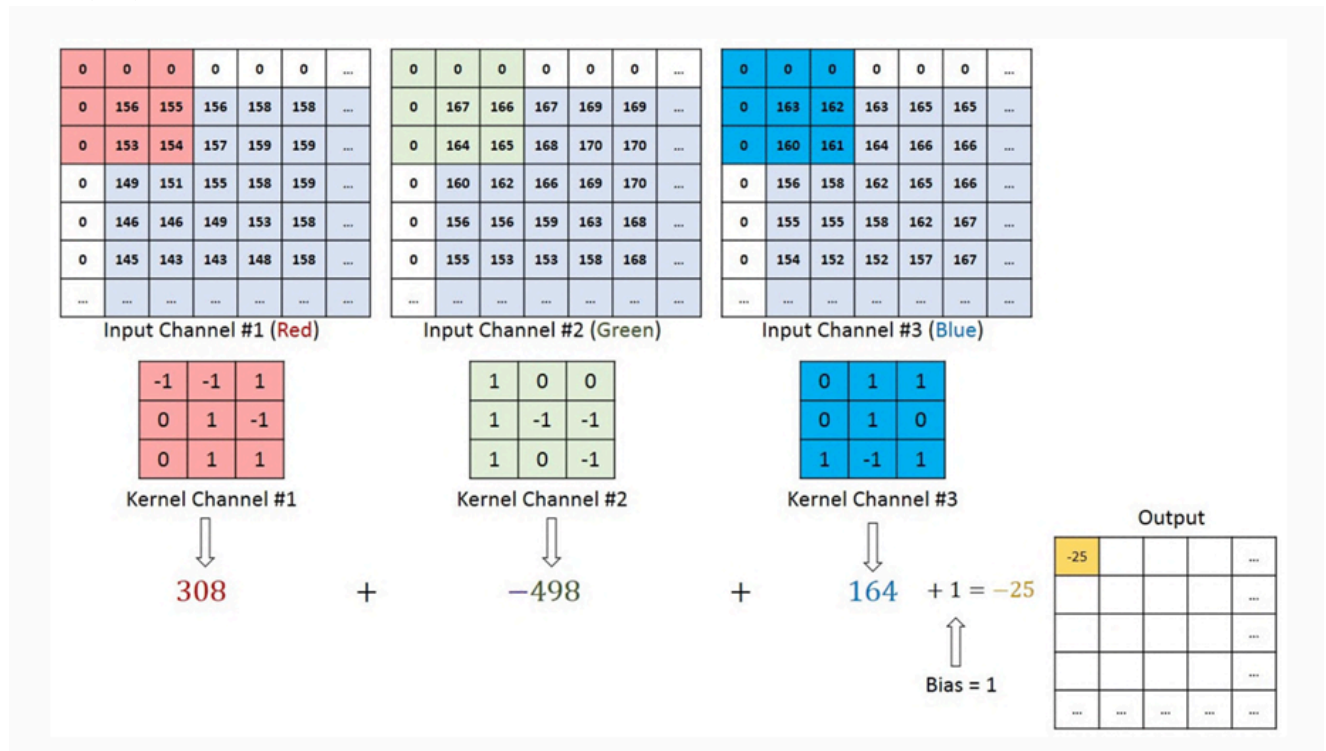
Classification 阶段: 全连接层整合全局信息 → 输出预测。

卷积层 (Convolutional Layer)

对输入体积 $X \in \mathbb{R}^{H \times W \times C_{in}}$ ，使用滤波器（kernel $K \in \mathbb{R}^{F \times F \times C_{in} \times C_{out}}$ ，输出：

$$Y_{i,j,k} = \sum_{f_1=0}^{F-1} \sum_{f_2=0}^{F-1} \sum_{c=0}^{C_{in}-1} K_{f_1,f_2,c,k} \cdot X_{i+f_1,j+f_2,c}$$

其中 (i, j) 为输出空间位置， k 为输出通道索引。



参数	符号	作用
滤波器尺寸	F	控制感受野大小（如 3×3 捕获局部纹理， 7×7 捕获更大结构）
步长	S	控制输出分辨率（ $S = 2 \rightarrow$ 尺寸减半）
填充	P	保持/扩大输出尺寸，避免边界信息丢失

设输入尺寸 $H_{in} \times W_{in}$ ，则：

$$H_{out} = \left\lfloor \frac{H_{in} + 2P - F}{S} \right\rfloor + 1, \quad W_{out} = \left\lfloor \frac{W_{in} + 2P - F}{S} \right\rfloor + 1$$

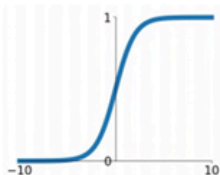
输入 $32 \times 32 \times 3$ ， $F = 5$ ， $S = 1$ ， $P = 2 \rightarrow$

$H_{out} = \lfloor (32 + 4 - 5)/1 \rfloor + 1 = 32 \rightarrow$ 保持尺寸不变（same padding）

激活层（Activation Layer）

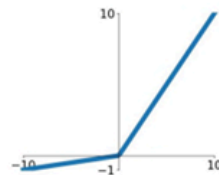
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



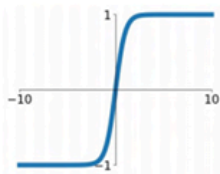
Leaky ReLU

$$\max(0.1x, x)$$



tanh

$$\tanh(x)$$

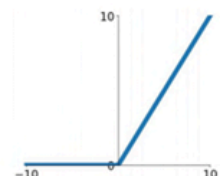


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

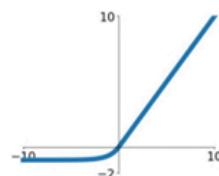
ReLU

$$\max(0, x)$$



ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



池化层 (Pooling Layer) —— 空间下采样与不变性增强

最大池化 (Max Pooling)

操作：在 $k \times k$ 区域内取最大值；

公式（对单通道）：

$$Y_{i,j} = \max_{p=0}^{k-1} \max_{q=0}^{k-1} X_{i \cdot S + p, j \cdot S + q}$$

优势：

保留最显著特征（如边缘、角点）；

提供平移不变性（translation invariance）；

显著降低维度（计算/存储开销↓）。

平均池化 (Average Pooling)

操作：取区域平均值；

$$Y_{i,j} = \frac{1}{k^2} \sum_{p,q} X_{i \cdot S + p, j \cdot S + q}$$

适用场景：需要保留背景信息的任务（如图像生成、语义分割 head）；

缺点：模糊细节，对噪声敏感。

全连接层 (Fully Connected Layer, FC)

对输入特征向量 $\mathbf{x} \in \mathbb{R}^D$ ，FC 层输出：

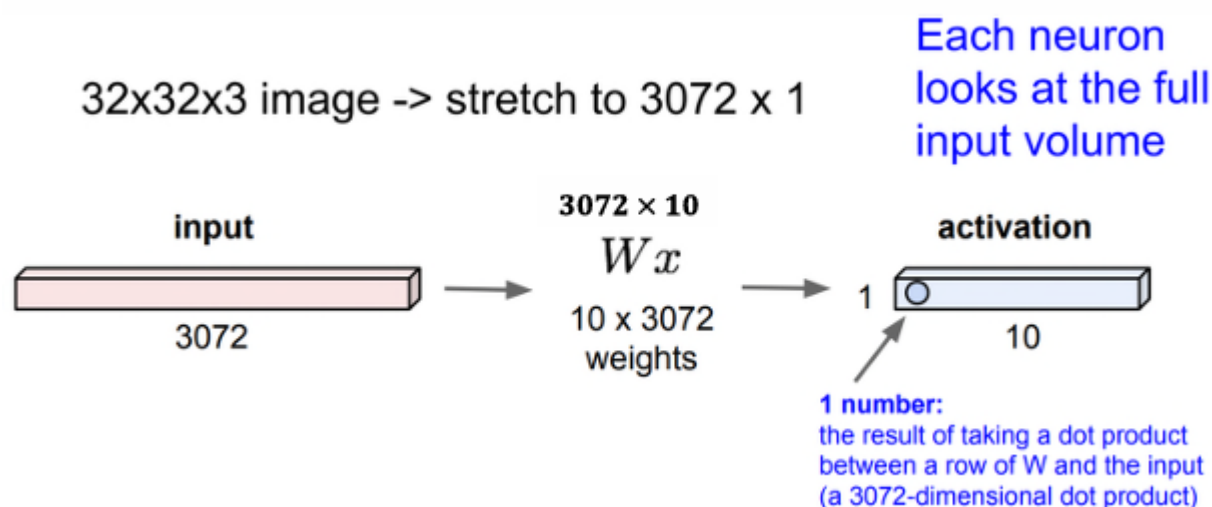
$$\mathbf{z} = W\mathbf{x} + \mathbf{b}, \quad W \in \mathbb{R}^{C \times D}, \mathbf{b} \in \mathbb{R}^C$$

其中：

D ：输入维度（如 $32 \times 32 \times 3 = 3072$ ）；

C ：类别数（如 ImageNet 的 1000）；

每个神经元 z_c 是对 **整个输入体积** 的加权求和（全连接）。



Softmax 层

给定 logits 向量 $\mathbf{z} = [z_1, z_2, \dots, z_C]$ ，输出概率：

$$p_j = \frac{e^{z_j}}{\sum_{k=1}^C e^{z_k}}, \quad \forall j \in \{1, \dots, C\}$$

性质： $p_j > 0$ ，且 $\sum_j p_j = 1$ ；

本质：将实数域映射到概率单纯形（Probability Simplex）。

设真实标签为 one-hot 向量 $\mathbf{y}$ ，则损失为：

$$L = - \sum_{j=1}^C y_j \log p_j = - \log p_{y_{\text{true}}}$$

梯度简洁： $\frac{\partial L}{\partial z_j} = p_j - y_j$

正确类： $p_j - 1$ （负梯度，降低其分数）；

错误类： $p_j - 0 = p_j$ （正梯度，降低其分数）。

训练与优化

目标：最小化经验风险

$$\mathcal{L}(W) = \frac{1}{N} \sum_{i=1}^N L(f(x_i; W), y_i)$$

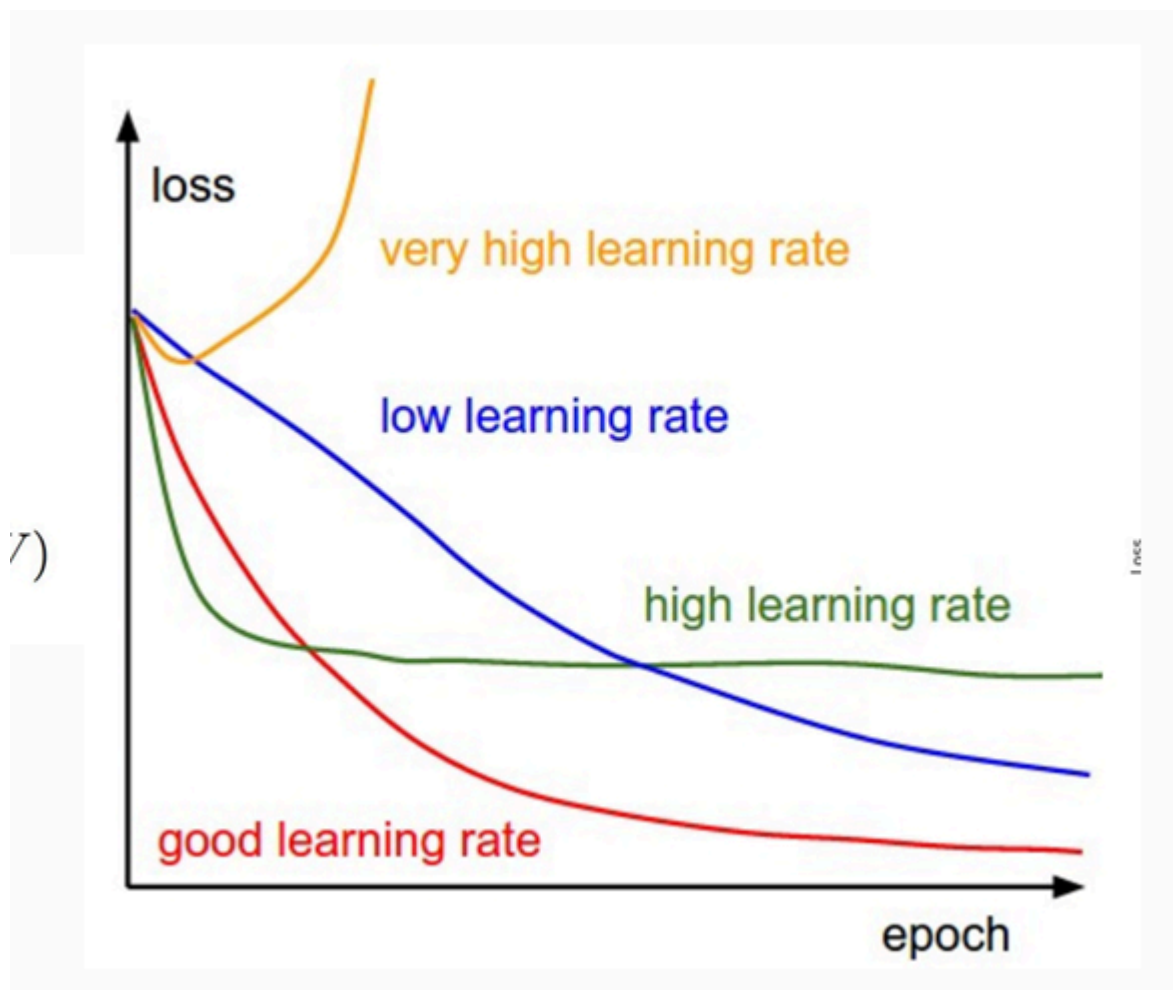
其中 $f(\cdot)$ 为网络前向映射， L 为损失（如交叉熵）。

参数更新规则（SGD）：

$$W_{t+1} = W_t - \alpha \nabla_W \mathcal{L}(W_t)$$

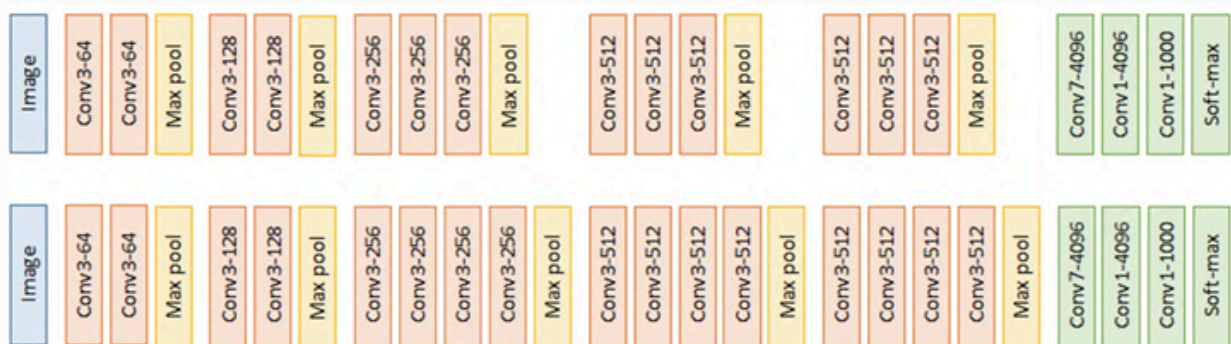
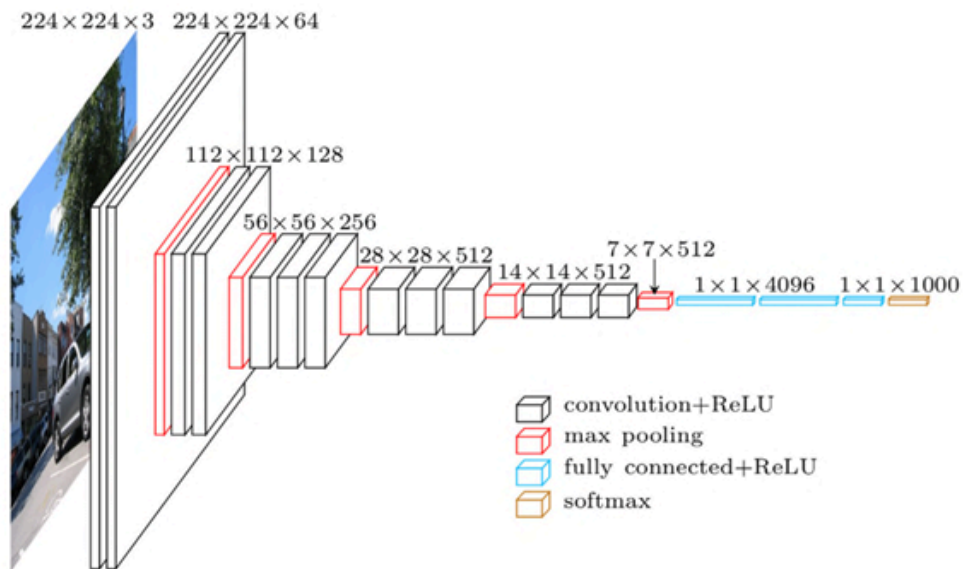
α ：学习率（Learning Rate, LR）

$\nabla_W \mathcal{L}$ ：通过反向传播（Backpropagation）计算



经典 CNN 架构深度剖析

VGG Network (Simonyan & Zisserman, 2014)



核心设计：

全部使用 **3×3 卷积核**（堆叠实现大感受野）；

例如：2 层 3×3 $\equiv$ 1 层 5×5，但参数减少 25 $\rightarrow$ 18；

池化层固定为 **2×2 MaxPool, stride=2**；

全连接层：4096 $\rightarrow$ 4096 $\rightarrow$ 1000（占总参数 90%！）。

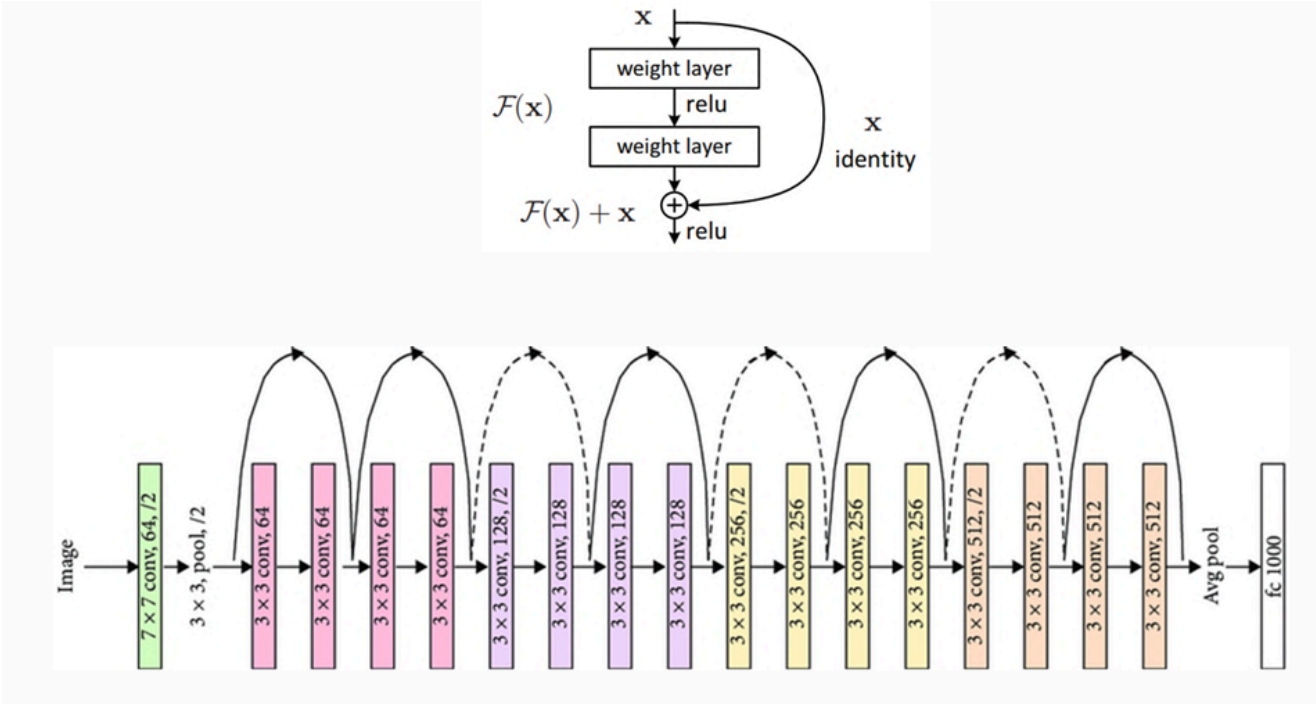
变体：

VGG-16：13 conv + 3 FC；

VGG-19：16 conv + 3 FC；

致命缺点：FC 层参数爆炸（138M），难以部署。

ResNet（He et al., CVPR 2016）



$$\mathcal{F}(x) = W_2\sigma(W_1x) \quad (\text{identity mapping}) \Rightarrow \quad H(x) = \mathcal{F}(x) + x$$

关键创新：添加恒等映射（skip connection）；
若 $\mathcal{F}(x) \approx 0$ ，则 $H(x) \approx x \rightarrow$ 梯度可直接回传；
数学保障： $\frac{\partial H}{\partial x} = \frac{\partial \mathcal{F}}{\partial x} + I$ ，避免梯度趋零。

模型性能评估

指标	公式/定义	适用场景
Accuracy	$\frac{\text{正确预测数}}{\text{总数}}$	分类任务基础指标（但对不平衡数据不敏感）
Memory Footprint	参数量 + 激活内存（activation maps）	边缘设备部署关键约束
FLOPs	Floating Point Operations per sample	衡量计算复杂度（推理延迟预估）
Speed (FPS)	Frames Per Second on target hardware	实时系统硬性要求

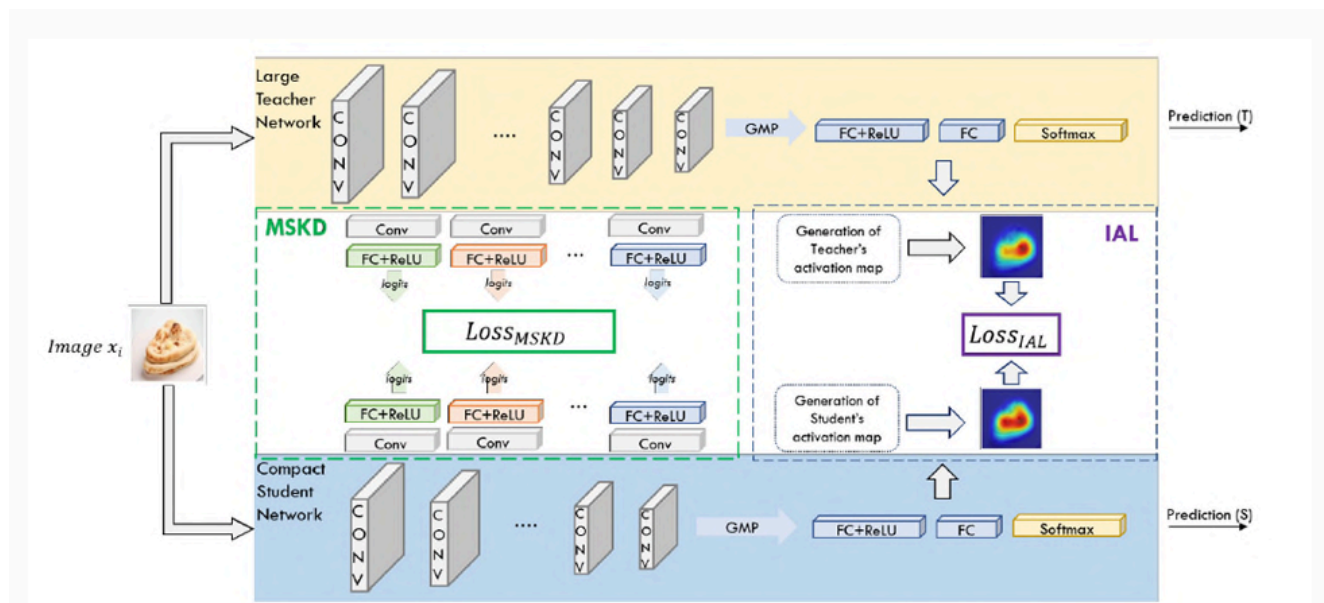
实际应用

Person Re-Identification（行人重识别）



给定查询图像 (Query) → 在跨摄像头数据库中检索同一人图像；
挑战：视角变化、遮挡、光照差异、服装相似。

Visual Food Recognition (视觉食物识别)



RNN

前馈网络 (Feedforward Network) 的局限性

无法处理序列数据 (cannot handle sequential data)。
仅考虑当前输入 (considers only the current input)。
无法记忆之前的输入 (cannot memorize previous inputs)。

为什么需要循环神经网络 (RNN)?

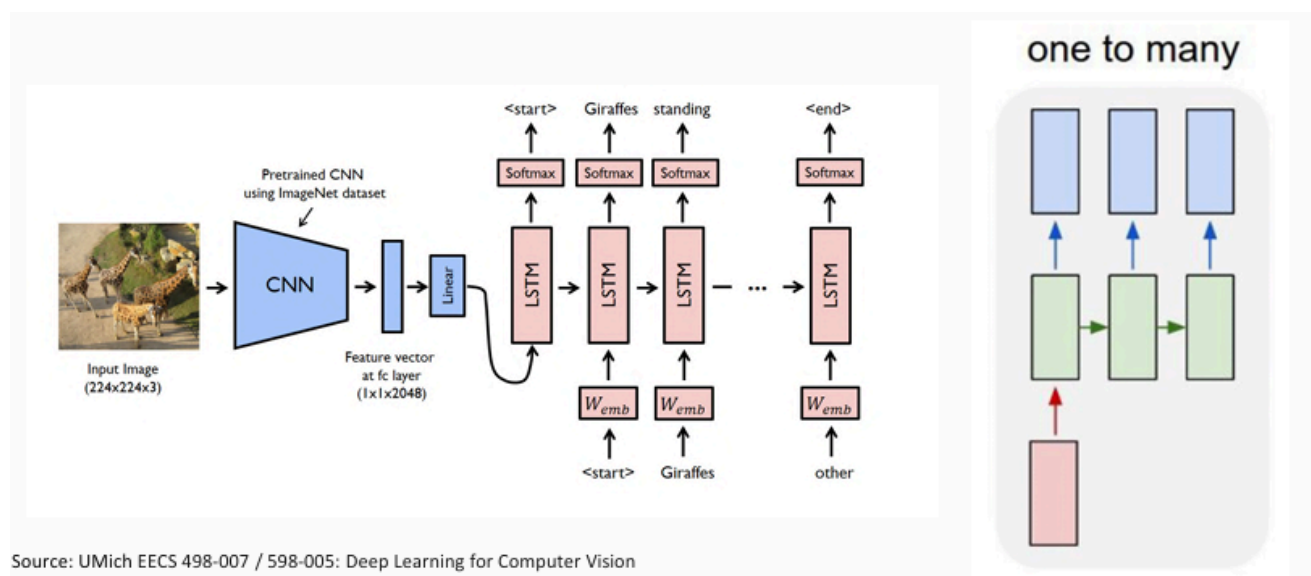
可以处理序列数据 (can handle sequential data)。

考虑当前输入以及之前接收到的输入 (considers the current input and also the previously received inputs)。

由于具有内部记忆，可以记忆之前的输入 (can memorize previous inputs due to its internal memory)。

序列建模 (Sequence Modelling)

一对多映射 (One-to-Many Mapping)

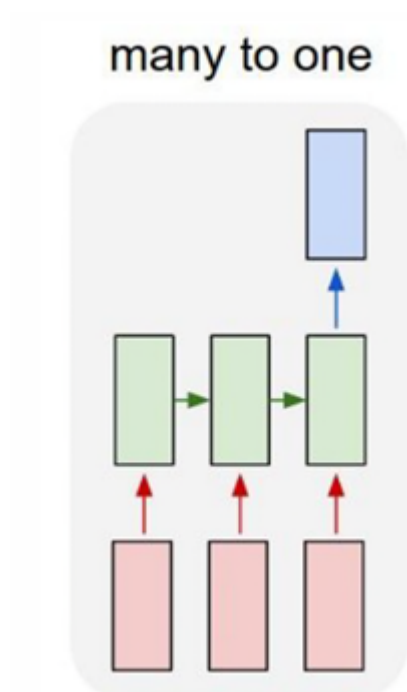


定义： 一个输入，多个输出（输出序列）。

图像描述 (Image captioning)。

图像 → 单词序列。

多对一映射 (Many-to-One Mapping)

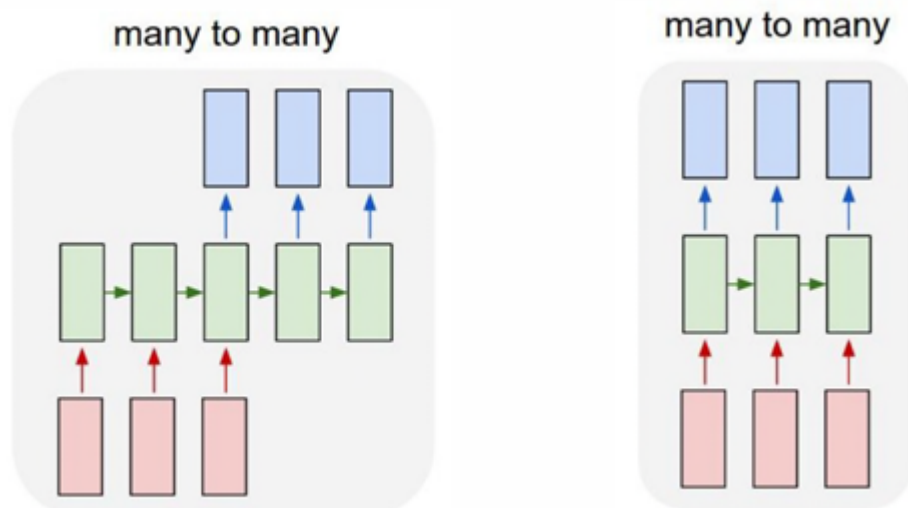


定义： 多个输入（输入序列），一个输出。

视频分类 (Video classification): 图像序列 → 标签。

情感分类 (Sentiment classification): 单词序列 → 标签。

多对多映射 (Many-to-Many Mapping)

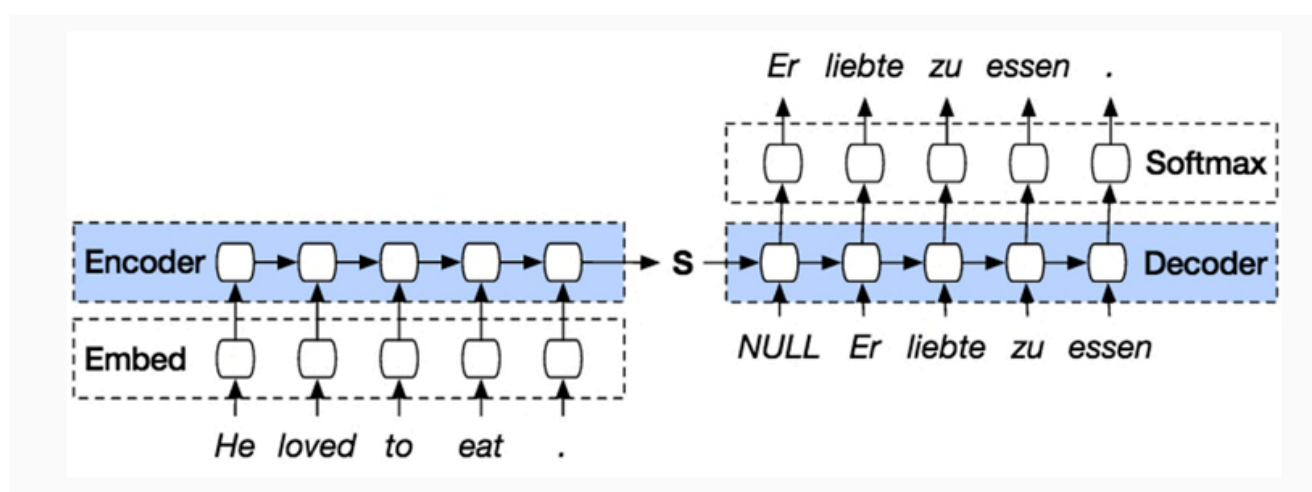


定义: 多个输入 (输入序列), 多个输出 (输出序列)。

机器翻译 (Machine translation): 单词序列 → 单词序列。

逐帧视频分类 (Per-frame video classification): 图像序列 → 标签序列。

序列到序列: 编码器 + 解码器 (Sequence to Sequence: Encoder + Decoder)



常用于机器翻译任务。

Encoder (编码器): 处理输入序列 (如 "He loved to eat"), 将其压缩为上下文向量。

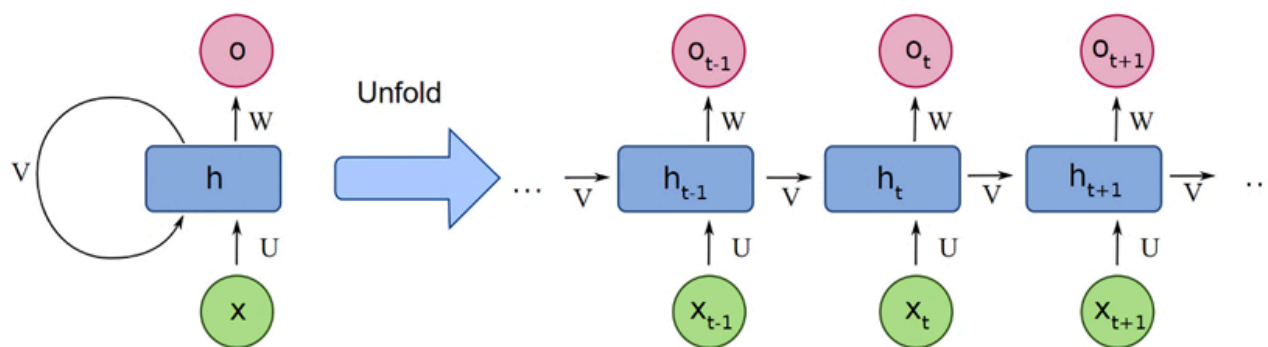
Decoder (解码器): 根据上下文向量生成输出序列 (如 "Er liebte zu essen")。

什么是 RNN?

RNN 是一类输入为**序列数据** (例如时间序列数据) 的神经网络。

常用于分析**时间 / 序列数据**。

广泛应用于自然语言处理 (NLP)、机器翻译、图像描述等。



RNN 的核心在于**循环 (Recurrent)** 结构。

可以将循环结构在时间轴上**展开 (Unfold)**。

关键组件：

x ：输入向量。

h ：隐藏状态 (Hidden state)。

o / y ：输出向量。

U, V, W ：权重矩阵。

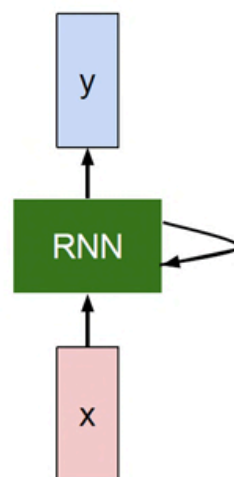
RNN 拥有一个“内部状态” (internal state)，该状态会随着序列的处理而不断更新。

隐藏状态更新 (Hidden State Update)

We can process a sequence of vectors $\mathbf{x}$ by applying a **recurrence formula** at every time step:

$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

new state some function with parameters W old state input vector at some time step



我们可以通过在每个时间步应用**递归公式**来处理向量序列 x ：

$$h_t = f_W(h_{t-1}, x_t)$$

h_t ：新状态 (new state)。

h_{t-1} ：旧状态 (old state)。

x_t ：当前时间步的输入向量 (input vector at some time step)。

f_W ：带有参数 W 的某个函数 (some function with parameters W)。

输出生成 (Output Generation)

$$y_t = f_{W_{hy}}(h_t)$$

y_t : 输出 (output)。

h_t : 新状态 (new state)。

$f_{W_{hy}}$: 带有参数 W_{hy} 的另一个函数 (another function)。

朴素 RNN (Vanilla RNN / Elman RNN)

状态由单个“隐藏”向量 h 组成。

1. 隐藏状态更新:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

2. 输出生成:

$$y_t = W_{hy}h_t$$

优点 (Pros)

可以处理任意长度的输入。

理论上，步骤 t 的计算可以使用许多步之前的信息。

模型大小不随输入长度增加而增加。

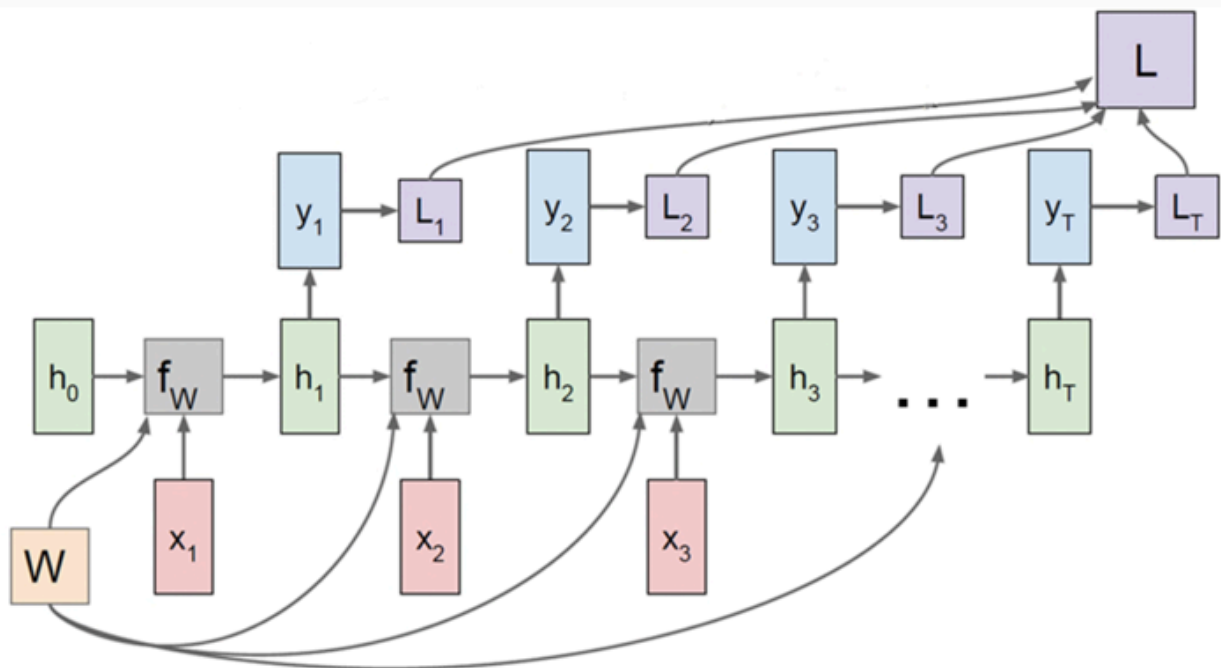
每个时间步应用相同的权重，因此输入的处理方式具有一致性。

缺点 (Cons)

循环计算速度慢 (Recurrent computation is slow) (无法并行化)。

实际上，难以利用许多步之前的信息 (即长距离依赖问题, Long-term dependency problem)。

计算图 (Computational Graph)



RNN 在时间维度上展开 (Unfold)，形成计算图：

单步计算：

输入 x_1 和上一时刻隐藏状态 h_0 进入函数 f_W 。

输出当前时刻隐藏状态 h_1 。

多步展开：

$h_0 \xrightarrow{x_1} h_1 \xrightarrow{x_2} h_2 \cdots \xrightarrow{x_T} h_T$

权重共享 (Weight Sharing)：

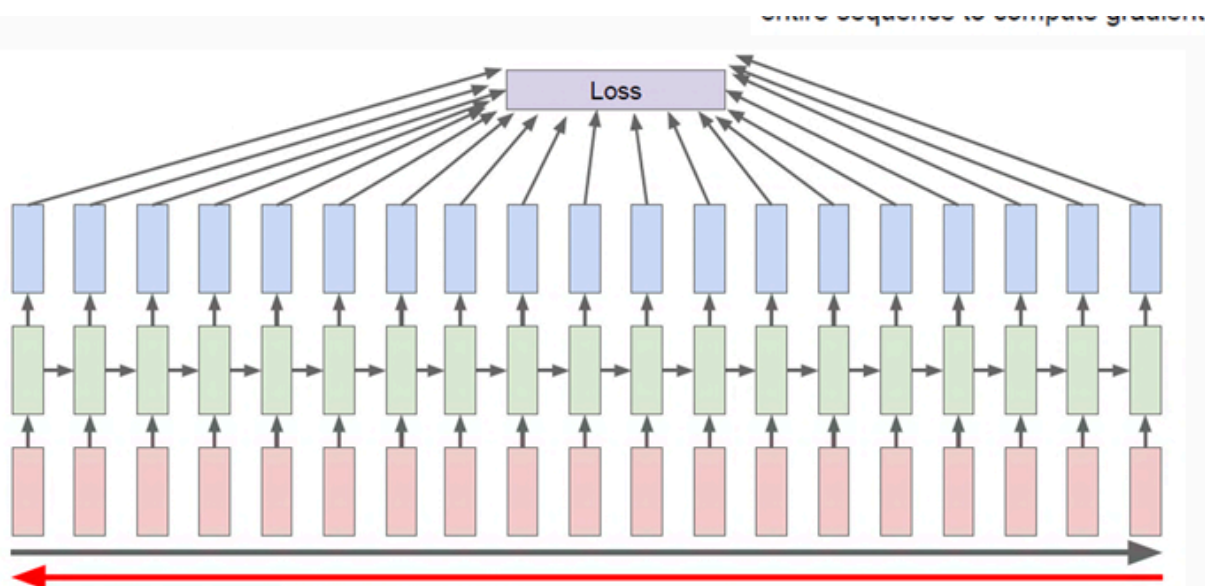
在每一个时间步 (time-step) 复用相同的权重矩阵 W 。

完整计算图：

包含输入序列 x ，隐藏状态序列 h ，输出序列 y 。

每个时间步产生损失 $L_1, L_2, \dots, L_T$ ，最终汇总为总损失 L 。

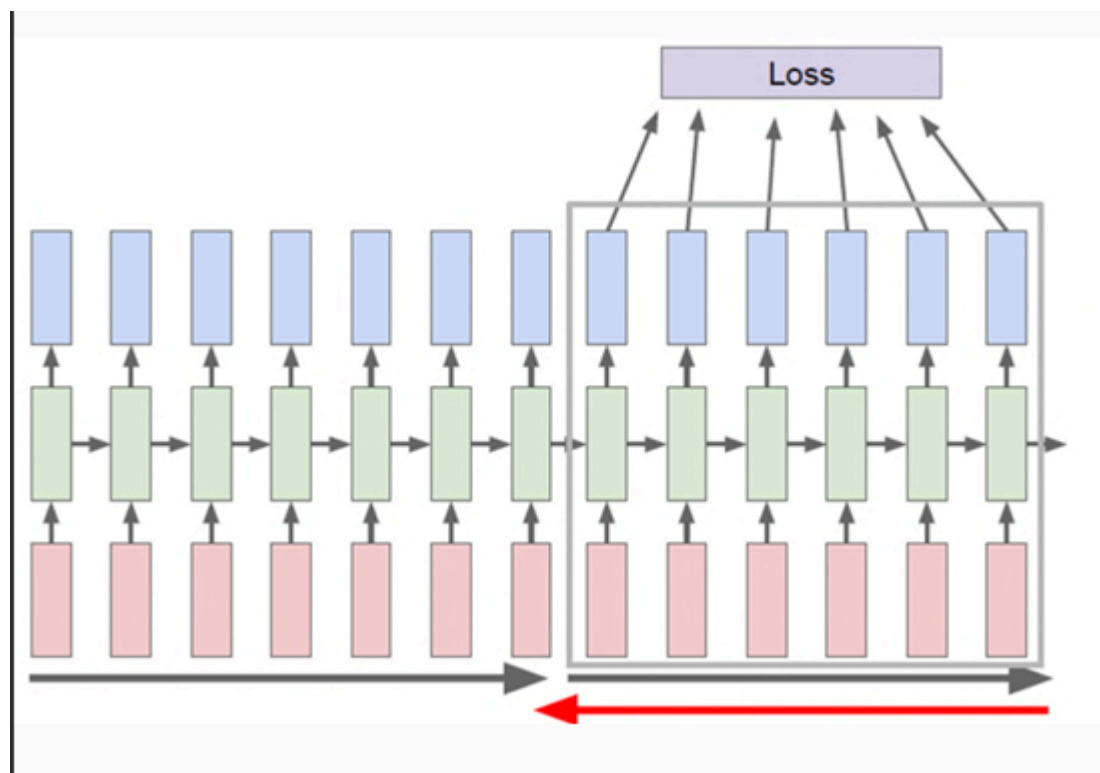
随时间反向传播 (Backpropagation Through Time, BPTT)



标准 BPTT：

向前遍历整个序列以计算损失 (Loss)。
然后向后遍历整个序列以计算梯度 (Gradient)。
缺点：对于长序列，计算量大且显存占用高。

截断的随时间反向传播 (Truncated BPTT)



为了解决长序列问题，通常采用截断策略：

1. 分块处理 (Chunks):

将序列分成小块 (chunks)。

在每个小块内分别进行前向和反向传播，而不是针对整个序列。

2. 状态携带，梯度截断：

隐藏状态 (Hidden states) 在时间上永远向前传递 (Carry forward)。

但是反向传播 (Backpropagate) 只进行较小的步数 (smaller number of steps)。

梯度爆炸与梯度消失 (Exploding / Vanishing Gradients)

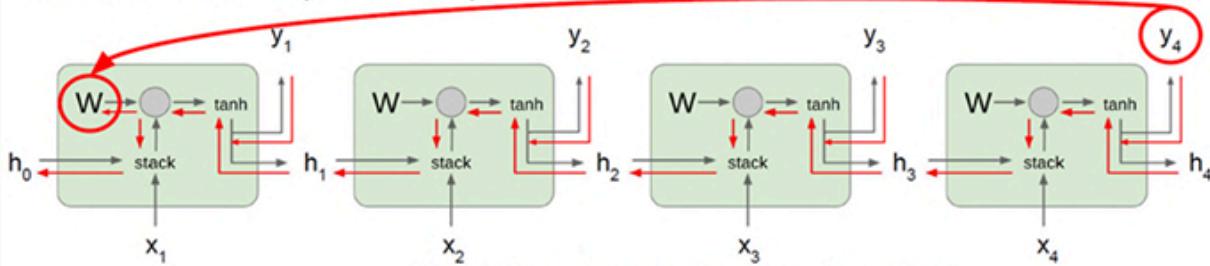
在多个时间步传播梯度时，涉及到权重矩阵 W_{hh} 的连乘。

假设没有非线性激活函数，梯度计算涉及项 W_{hh}^{T-1} 。

$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} W_{hh}^{T-1} \frac{\partial h_1}{\partial W}$$

Gradients over multiple time steps:



What if we assumed no non-linearity?

$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} W_{hh}^{T-1} \frac{\partial h_1}{\partial W}$$

Largest singular value > 1:
Exploding gradients

Largest singular value < 1:
Vanishing gradients

Gradient clipping:
Scale gradient if its norm is too big

```
grad_norm = np.sum(grad * grad)
if grad_norm > threshold:
    grad *= (threshold / grad_norm)
```

取决于 W_{hh} 的最大奇异值 (Largest singular value):

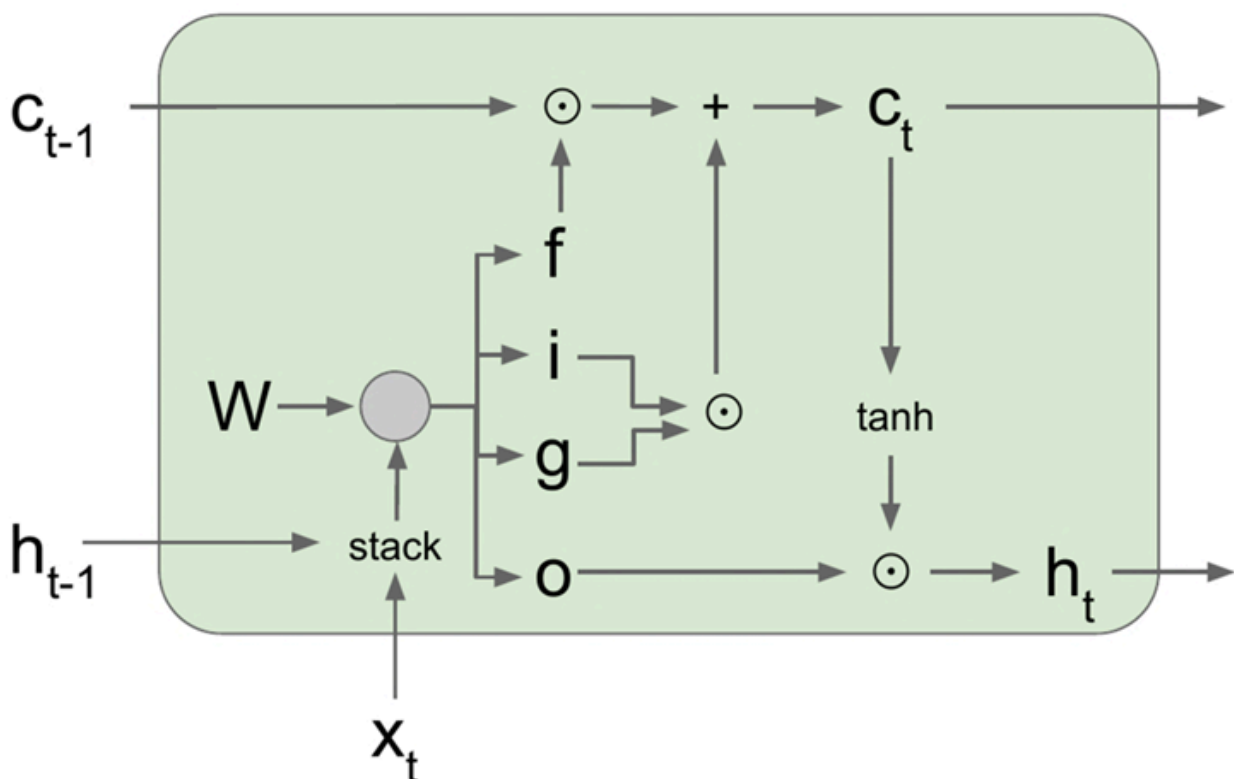
最大奇异值 > 1: 导致 梯度爆炸 (Exploding gradients)。

解决方法: 梯度裁剪 (Gradient clipping)。如果梯度的范数 (norm) 太大, 则对其进行缩放。

最大奇异值 < 1: 导致 梯度消失 (Vanishing gradients)。

解决方法: 改变 RNN 架构 (例如使用 LSTM)。

长短期记忆网络 (Long Short-Term Memory, LSTM)



LSTM 是一种专门设计用来解决梯度消失问题的 RNN 变体。

记忆单元 (Memory cell): 包含一个可以在长时间内维持其状态的单元。

两个状态:

细胞状态 (c_t): 长时记忆 (long-term memory)。

隐藏状态 (h_t): 短时记忆 (short-term memory)。

四个门 (Gates): i (input), f (forget), o (output), g (gate/candidate)。

门通过 Sigmoid 或 Tanh 层获得，用于控制信息流。
使用逐元素乘法 (pointwise multiplication, $\odot$) 更新状态。

架构公式

门控计算 (Gates Calculation):

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

状态更新:

$$c_t = f \odot c_{t-1} + i \odot g$$
$$h_t = o \odot \tanh(c_t)$$

特性	朴素 RNN (Vanilla RNN)	LSTM
公式	$h_t = \tanh(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix})$	包含复杂的门控机制 (i, f, o, g) 和细胞状态 c_t
记忆能力	难以捕捉长距离依赖 (Long-term dependencies)	专门设计用于捕捉长距离依赖
梯度问题	容易出现梯度消失/爆炸	通过细胞状态路径缓解梯度消失

- g (Gate gate / Candidate gate):** 决定写入多少内容到细胞中（候选值）。
- i (Input gate):** 决定是否写入（控制候选值 g 的保留比例）。
- f (Forget gate):** 决定是否擦除旧的记忆（控制 c_{t-1} 的保留比例）。
- o (Output gate):** 决定揭示多少细胞状态作为当前的隐藏状态输出。

i: Input gate, whether to write to cell
f: Forget gate. Whether to erase cell
o: Output gate, How much to reveal cell
g: Gate gate (?), How much to write to cell

应用场景 (Applications)

时间序列预测 (Time Series Prediction)

应用：股票价格预测、销量预测等。
原理：利用 RNN/LSTM 处理序列数据的能力，根据过去的历史数据（Real）预测未来的趋势（Predicted）。

机器翻译 (Machine Translation)

架构: Encoder-Decoder (编码器-解码器) 模型。

流程:

1. **Encoder RNN:** 读取源句子 (Source sentence, 如法语 "il a m' entente"), 将其压缩为一个固定长度的向量 (Context Vector / Information bottleneck)。这个向量需要捕获源句子的所有信息。
2. **Decoder RNN:** 读取该向量, 逐步生成目标句子 (Target sentence, 如英语 "he hit me with a pie")。

特点: 这是一个 Many-to-Many 的序列建模任务, 但输入和输出长度可以不同。

Transformer

什么是注意力 (What is Attention?)

定义: 注意力用于确定哪些输入 token (例如 NLP 中的单词, 或 CV 中的图像块) 与当前的输入/token 相关。

计算方式: 通过两个向量的相关性 (点积) 来计算注意力。

逻辑链条: 相关性 (Correlation) -> 相似度/关联度/重要性 -> 注意力 (Attention)。

如何计算 (How is Attention Computed?)

$$A(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Step 1: 计算 q 和 k 的点积 (相关性)。

Step 2: 将相关性值除以 $\sqrt{d_k}$ (缩放), 然后使用 Softmax 归一化。

Step 3: 将归一化后的权重与对应的 v 向量相乘并求和。

概述

Transformer 使用注意力机制**并行**处理输入序列。

使用注意力机制和稠密/前馈/MLP 层。

高度可并行化 (Highly parallelizable)。

可以提供**全局注意力** (Global attention)。

擅长建模**长距离依赖** (Long-range dependencies)。

在许多视觉和 NLP 应用中实现了最先进 (SOTA) 的性能。

引出了其他 SOTA 方法, 如 **BERT** (Bidirectional Encoder Representations from Transformers)。

架构

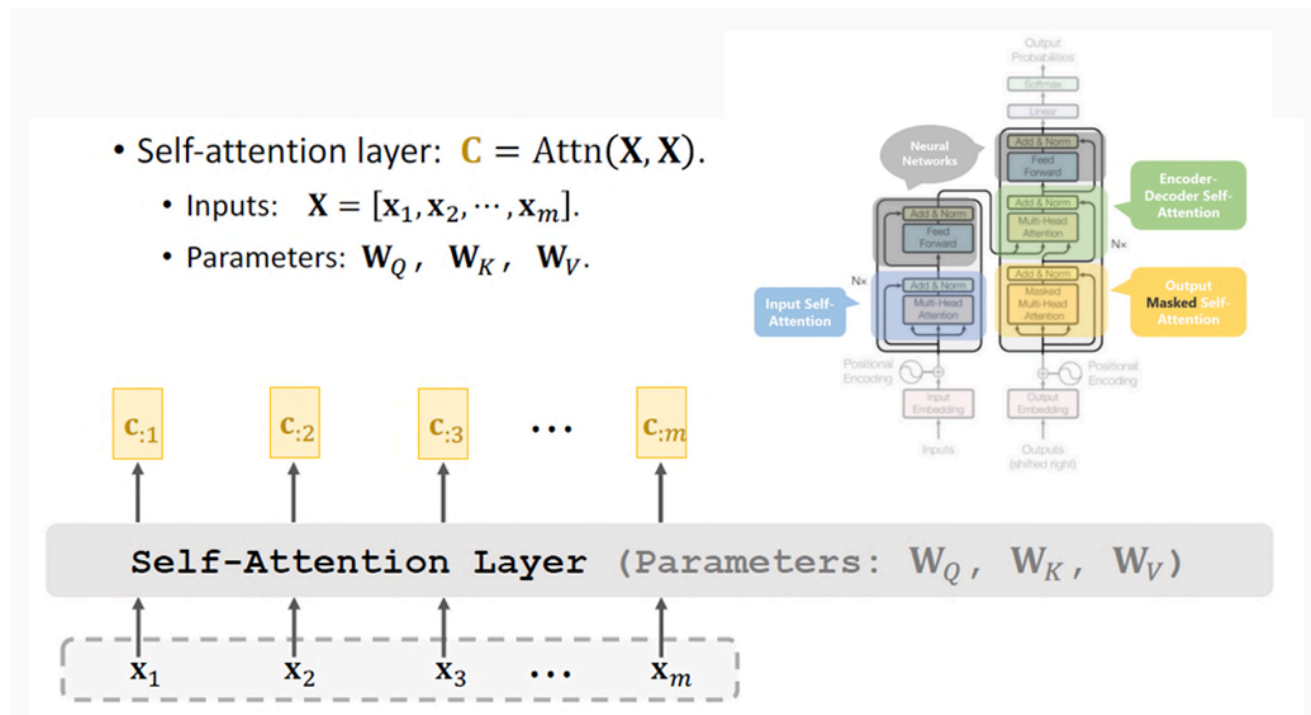
自注意力层(Self-Attention Layer)

公式: $C = \text{Attn}(X, X)$

输入: $X = [x_1, x_2, \dots, x_m]$

参数: W_Q, W_K, W_V

过程：输入序列 X 通过自注意力层，生成上下文表示 $C_1, C_2, \dots, C_m$ 。



交叉注意力 / 编码器-解码器自注意力 (Cross-Attention / Encoder-Decoder Self-Attention)

公式: $C = \text{Attn}(X, X')$

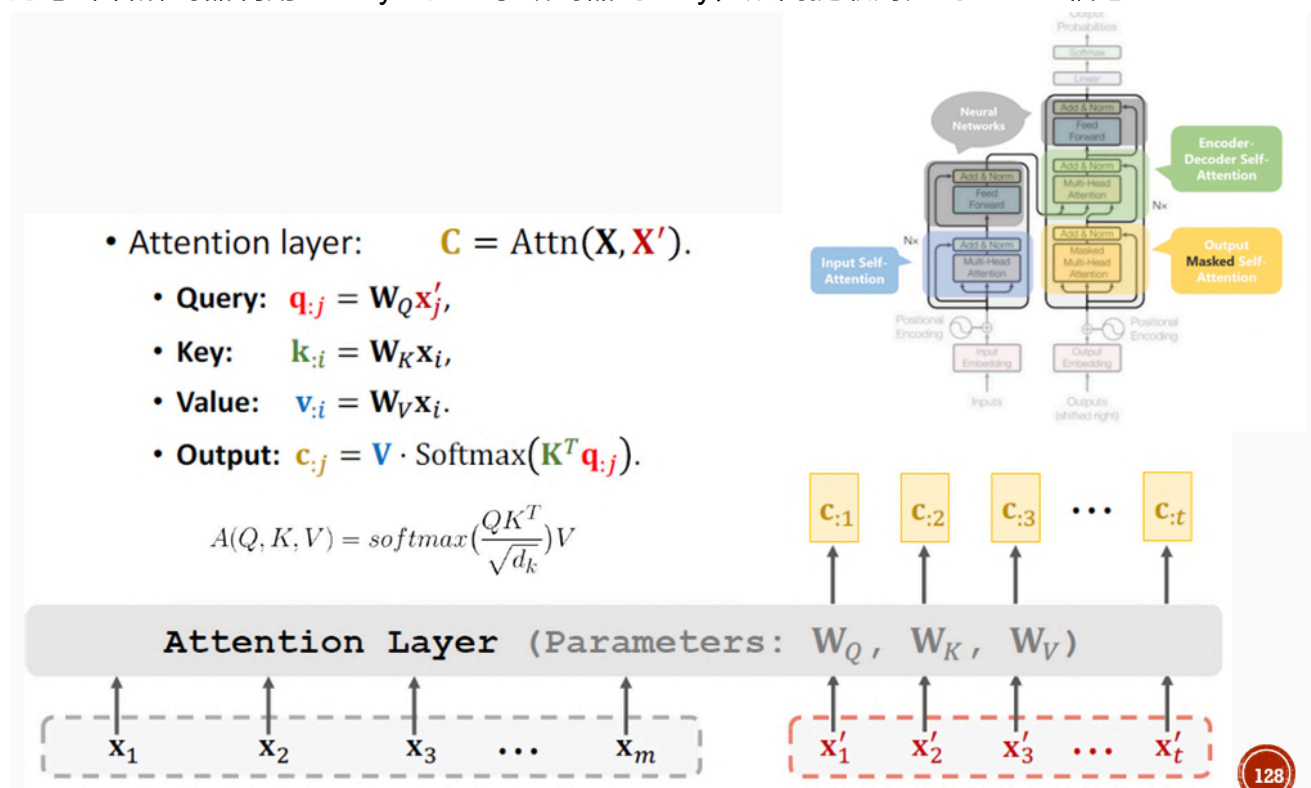
Query (q_j): 来自解码器当前状态 ($W_Q s'_j$)。

Key (k_i): 来自编码器输出 ($W_K x_i$)。

Value (v_i): 来自编码器输出 ($W_V x_i$)。

输出: $c_j = V \cdot \text{Softmax}(K^T q_j)$ 。

这意味着解码器利用 Query 去“查询”编码器的 Key，从而提取对应的 Value 信息



多头注意力 (Multi-Head Attention)

不只是一次注意力计算，而是并行运行多个“头” (Heads)。
每个头学习不同的表示子空间（例如一个头关注语法，一个头关注指代关系）。
最后将结果拼接 (Concat) 并再次线性变换。

位置编码 (Position Encoding)

由于 Transformer 是并行处理的，它本身不包含序列顺序信息。
需要加入位置编码来表示每个输入 token 的位置信息。
通常使用 $\sin / \cos$ 函数生成位置编码，并将其加到输入 Embedding 上。

残差连接与层归一化 (Residual Connection & Layer Norm)

残差连接 (Residual Connection)：借鉴 ResNet 的思想，将输入直接加到输出上 ($x + \text{SubLayer}(x)$)，有助于深层网络训练。

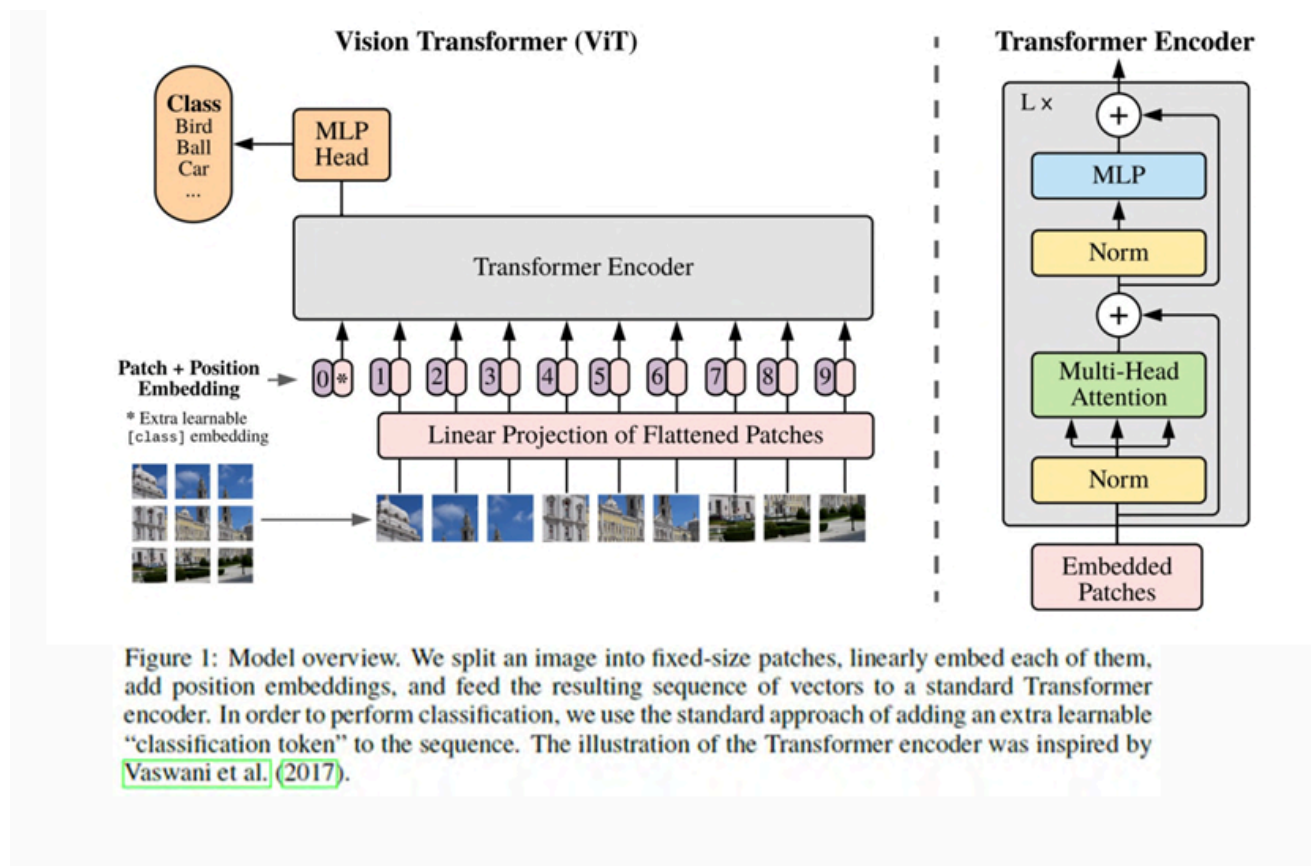
层归一化 (Layer Norm)：用于对每一层的输出进行归一化，稳定训练过程。
结构通常为：**Add & Normalize** (残差连接后接 Layer Norm)。

最终线性层和 Softmax 层 (Final Linear and Softmax Layer)

Decoder 的输出经过一个**线性层 (Linear Layer)**，映射到词汇表大小 (Vocab Size)。
结果称为 **Logits**。
经过 **Softmax** 层，将 Logits 转换为概率分布。
取概率最高的索引 (**Argmax**) 作为预测的下一个单词。

视觉 Transformer (Vision Transformer, ViT)

ViT 是将 Transformer 应用于图像分类的 SOTA 模型。



关键步骤：

1. **分块 (Partition)**: 将图像分割成固定大小的块 (Patches / Tokens)。
2. **展平 (Flatten)**: 使用字典序排列展平这些块。
3. **线性投影 (Linear Projection)**: 将展平后的块生成线性嵌入 (Linear Embeddings)。
4. **Class Token**: 引入一个额外的可学习的分类 token (Class Embedding)。
5. **位置编码 (Positional Embeddings)**: 添加位置信息。
6. **Encoder**: 将上述序列传入 Transformer Encoder。
7. **分类头 (MLP Head)**: Encoder 输出的 Class Token 通过一个 MLP 头网络进行分类。
8. **训练**: 通常涉及使用预训练模型, 并在目标数据集上进行微调 (Finetune)。

目标检测

目标 (Objective)

核心任务: 预测图像/视频中的物体类别 (Object Classes) 并使用**边界框 (Bounding Boxes)** 定位它们。

任务性质: 这是一个 **回归 (Regression)** + **分类 (Classification)** 的任务。

回归: 确定框的位置和大小。

分类: 确定框内物体的类别。

应用 (Applications)

下游计算机视觉 (CV) 任务的关键步骤:

物体跟踪 (Object tracking)

姿态估计 (Pose estimation)

动作识别 (Action recognition)

现实生活中的应用:

自动驾驶 (Autonomous driving)

视频监控 (Video surveillance / monitoring)

挑战 (Challenges)

不同的成像条件: 包括视角变化 (viewpoints)、尺度变化 (scales)、光照 (lighting)、背景杂乱 (background clutter)、遮挡 (occlusion) 等。

计算效率 (Computational efficiency): 需要在速度和精度之间取得平衡。

封闭世界 vs 开放世界问题 (Close-world vs open-world problem): 模型能否识别训练集中未出现的类别。

常用数据集 (Common Datasets)

MS COCO:

训练和评估的关键基准数据集。

训练、验证和测试集包含超过 **200,000** 张图像和 **80** 个物体类别。

ImageNet:

包含 1000 个图像分类类别。

其中 **200** 个类别是专门为物体检测挑选的。

包含超过 **500k** 张用于物体检测的图像。

Google Open Images:

包含 **190 万** 张用于物体检测的图像。

包含 **1600 万** 个边界框。

包含 **600** 个类别。

性能指标 (Performance Metrics)

平均精度均值 (mean Average Precision, mAP)

也简称为 **AP**。

用于评估目标检测模型准确性的指标。

依赖条件: 取决于所选的 **交并比 (Intersection over Union, IoU)** 值。

判定标准:

如果 $IoU > \text{threshold}$, 预测被视为 **真阳性 (True Positive, TP)**。

如果 $IoU < \text{threshold}$, 预测被视为 **假阳性 (False Positive, FP)**。

常见表示法:

mAP_{50} 或 $mAP@0.5$ (IoU 阈值为 0.5)。

mAP_{COCO} (COCO 数据集的标准, 通常取多个 IoU 阈值的平均值)。

公式: $mAP_{COCO} = \frac{mAP_{50} + mAP_{55} + \dots + mAP_{95}}{10}$

$$IoU = \frac{\text{Area of Overlap (重叠面积)}}{\text{Area of Union (并集面积)}}$$

1. 使用模型生成预测的边界框和标签。
2. 计算混淆矩阵: **真阳性 (TP)**, **假阳性 (FP)**, **假阴性 (FN)**。
3. 计算 **精确率 (Precision)** 和 **召回率 (Recall)** 指标。

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{TP}{\# \text{ ground truths}}$$

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{TP}{\# \text{ predictions}}$$

4. 计算当前类别的 **AP**: 精确率 - 召回率曲线 (Precision-Recall Curve) 下的面积。
5. 计算 **mAP**: 所有类别的 AP 的平均值。

$$mAP = \frac{1}{N} \sum_{i=1}^N AP_i$$

Loose (宽松): IoU = 0.5 (阈值低, 容易判定为 TP)。

Tight (严格): IoU = 0.9 (阈值高, 要求预测框非常精准)。

浮点运算次数 (Floating Point Operations, FLOPs)

用于评估任何 AI 模型计算复杂度的指标。

常用于描述运行给定模型的单个实例需要多少次运算。

衡量单次前向传播 (Forward Pass) 所需的浮点运算总数。

反映了不同模型的检测时间 (速度)。

目标检测器分类 (Object Detector Categories)

1. **单阶段检测器 (One-stage detectors):**

直接从图像中生成检测结果 (分类 + 回归)。

代表模型：YOLO, SSD。

特点：速度快。

2. 两阶段检测器 (Two-stage detectors):

第一阶段：生成区域建议 (Region Proposal / Proposal generator)。

第二阶段：对建议区域进行分类和回归 (Detection generator)。

代表模型：R-CNN 系列 (Fast R-CNN, Faster R-CNN)。

特点：精度通常较高，但速度较慢。

3. 轻量级检测器 (Lightweight detectors):

专为移动设备或嵌入式系统设计，注重效率。

两阶段检测器 (Two-Stage Detectors)

核心机制：使用如区域建议网络 (Region Proposal Network, RPN) 等技术来提出可能的物体区域。

代表方法：Faster-RCNN, Mask-RCNN。

其他方法：SPP-Net, FPN (Feature Pyramid Networks) 等。

特点：与单阶段检测器相比，通常更复杂且速度较慢，但精度往往更高。

R-CNN 系列检测器 (Region-based Convolutional Neural Network Series)

主要包括：

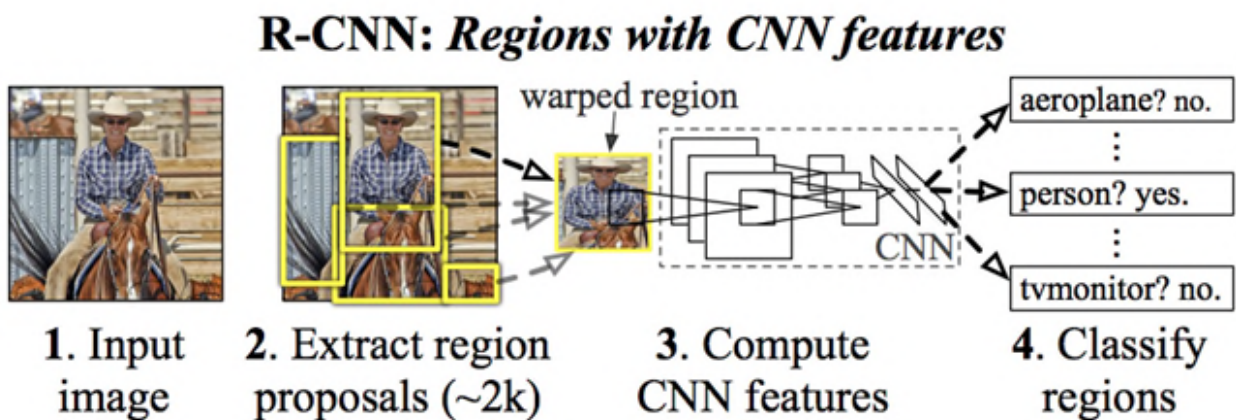
R-CNN

Fast R-CNN

Faster R-CNN

Mask R-CNN

R-CNN (Region-based Convolutional Neural Network)



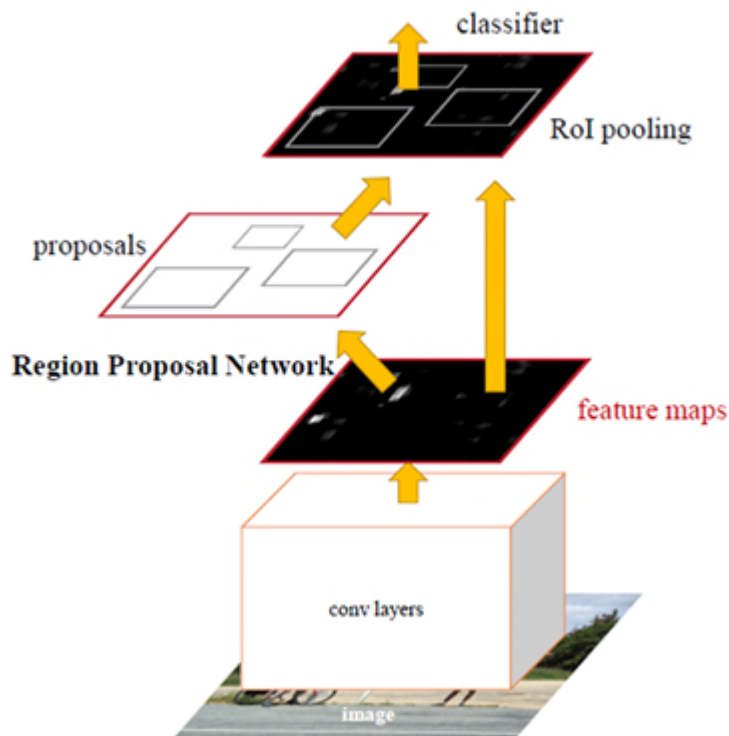
步骤：

生成区域建议：生成约 2000 个候选区域 (Region Proposals)。

特征提取：使用 CNN 从每个区域提取特征向量 (需要将区域变形 warped 以适配 CNN 输入)。

分类与回归：使用特定类别的线性支持向量机 (SVM) 对每个区域进行分类，并执行边界框回归 (Bounding-box regressions)。

Faster R-CNN



核心改进：引入了**区域建议网络 (RPN)**。

优势：

RPN 与检测分支共享全图的卷积特征，使得生成区域建议几乎是**零成本 (cost-free)** 的。

RPN 同时预测物体边界 (Object boundaries) 和物体置信度分数 (Objectness scores)。

速度对比 (Speed Comparison)

R-CNN: ~49 秒/图 (最慢)

SPP-Net: ~4.3 秒/图

Fast R-CNN: ~2.3 秒/图

Faster R-CNN: ~0.2 秒/图 (显著提速，接近实时)

单阶段检测器 (One-Stage Detectors)

核心机制：直接从特征图 (Feature Map) 进行回归和分类，不需要生成区域建议的步骤。

代表方法：YOLO, SSD。

其他方法：CenterNet, EfficientDet 等。

特点：与两阶段检测器相比，通常**复杂度更低，速度更快**。

YOLO 系列检测器 (You Only Look Once)

发展历史：从 YOLO v1 到 v5 及后续变体 (如 PP-YOLO, Scaled YOLO v4)，在速度 (FPS) 和精度 (mAP) 之间不断取得平衡。

通用架构 (General Architecture)：

Input (输入)：图像，通过数据增强生成的 patches。

Backbone (主干网络)：预训练的分类模型，用于特征提取，针对不同尺度的物体提取不同层级的特征。

Neck (颈部): 上采样和融合机制, 用于融合不同阶段的特征图 (如 FPN, PANet)。

Head (头部): 输出预测结果, 包括物体类别和检测到的边界框。

近期版本:

YOLOv7 (2022)

YOLOv10 (2024)

YOLOv12

性能对比: 在延迟 (Latency) - 精度 (mAP) 和 计算量 (FLOPs) - 精度 (mAP) 的权衡曲线上表现优异。

轻量级检测器 (Lightweight Detectors)

设计目标: 专为资源受限设备 (如手机、嵌入式设备) 设计。

特点: 网络小、速度快、效率高。

代表方法: 基于 **MobileNetV2** 的 SSD (如 MediaPipe 中使用)。

其他方法: SqueezeNet, ShuffleNet 等。

优缺点:

Pros: 超快, 高效。

Cons: 精度相对较低 (Less accurate)。

MobileNet

核心技术: 使用深度可分离卷积 (Depth-wise Separable Convolutions) 来减少计算量。

1. **Depth-wise Convolution (深度卷积):** 对每个输入通道应用单个滤波器。
2. **Pointwise Convolution (逐点卷积):** 使用 1×1 卷积将深度卷积的输出组合起来。

新兴方法

Swin Transformer

地位: 一种流行的近期图像分类/物体检测方法。

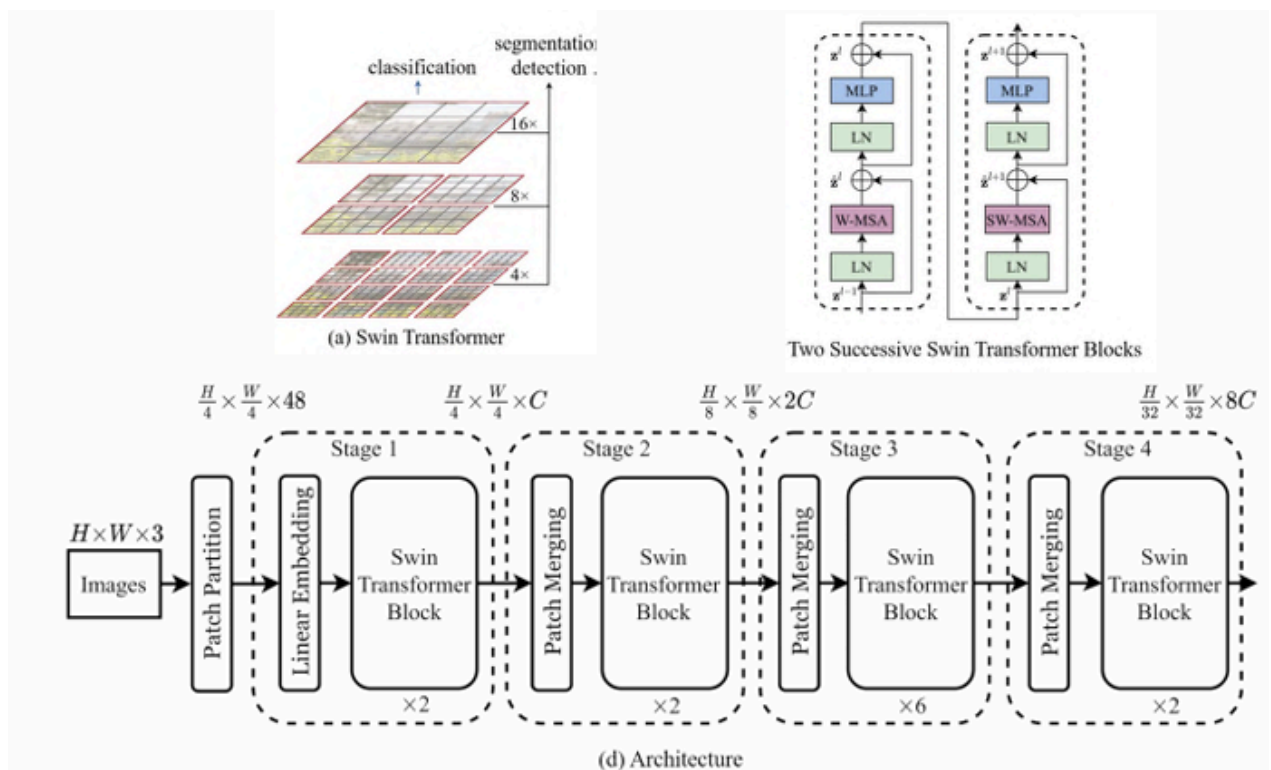
用途: 作为计算机视觉的通用主干网络 (Backbone)。

核心特点: 层级化特征图 (Hierarchical feature maps)。

与 ViT (Vision Transformer) 不同, Swin Transformer 通过合并图像块 (merging image patches) 在更深层构建层级特征图, 具有线性计算复杂度。

ViT 产生单一分辨率的特征图, 计算复杂度随图像尺寸呈二次方增长。

1. **Patch Partition:** 将输入 RGB 图像分割成不重叠的 4×4 块。
2. **Linear Embedding:** 线性嵌入。
3. **Swin Transformer Blocks:** 包含多个阶段 (Stage 1-4), 阶段之间通过 **Patch Merging** 进行下采样。



W-MSA (Window-based Multi-head Self-Attention): 基于窗口的多头自注意力。在层中，采用规则的窗口划分方案，自注意力仅在每个窗口内计算。

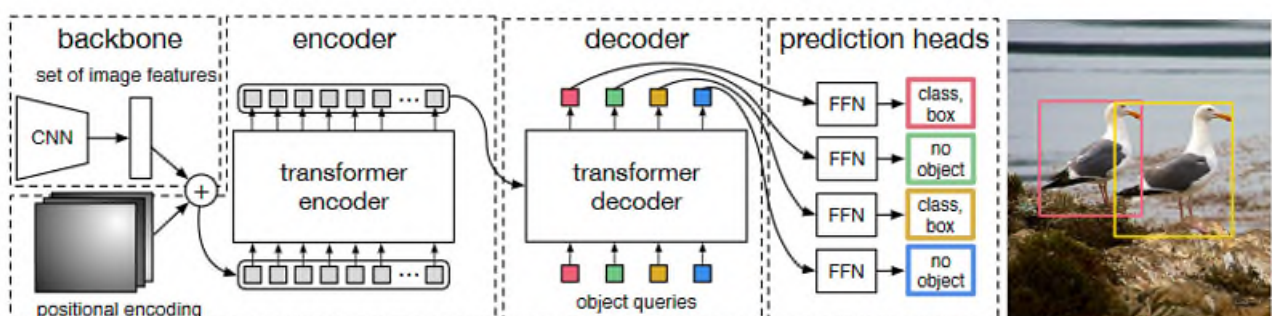
SW-MSA (Shifted Window-based Multi-head Self-Attention): 移位窗口多头自注意力。在层中，窗口划分发生移位，从而在新的移位窗口中进行计算。这实现了跨窗口的连接。

DETR: 基于 Transformer 的端到端物体检测 (Detection Transformer)

一种流行的近期物体检测方法。

核心思想: 结合通用的 CNN 和 Transformer 架构，**并行预测**最终的检测结果集合。

输入: 解码器 (Decoder) 的输入是一组位置编码序列，称为**物体查询 (Object Queries)**。



- Backbone (主干网络):** 使用传统的 CNN 主干网络学习图像的 2D 表示。
- Positional Encoding (位置编码):** 将 2D 特征图展平 (Flatten) 并补充位置编码。
- Transformer Encoder (编码器):** 处理带有位置编码的特征。
- Transformer Decoder (解码器):**
 - 输入: 一小部分固定的可学习位置编码 (即 **Object Queries**)。
 - 输出: 共享的前馈网络 (FFN)，预测检测结果 (类别 + 边界框) 或“无物体”类。

Grounding DINO: 开放集物体检测 (Open-Set Object Detection)

目标：允许通过**自然语言文本提示 (Text Prompts)** 检测任意物体。

成就：实现了最先进的**开放词汇 (Open-Vocabulary)** 检测性能。

对比：

(a) 闭集检测 (Closed-set)：只能检测预定义类别的物体。

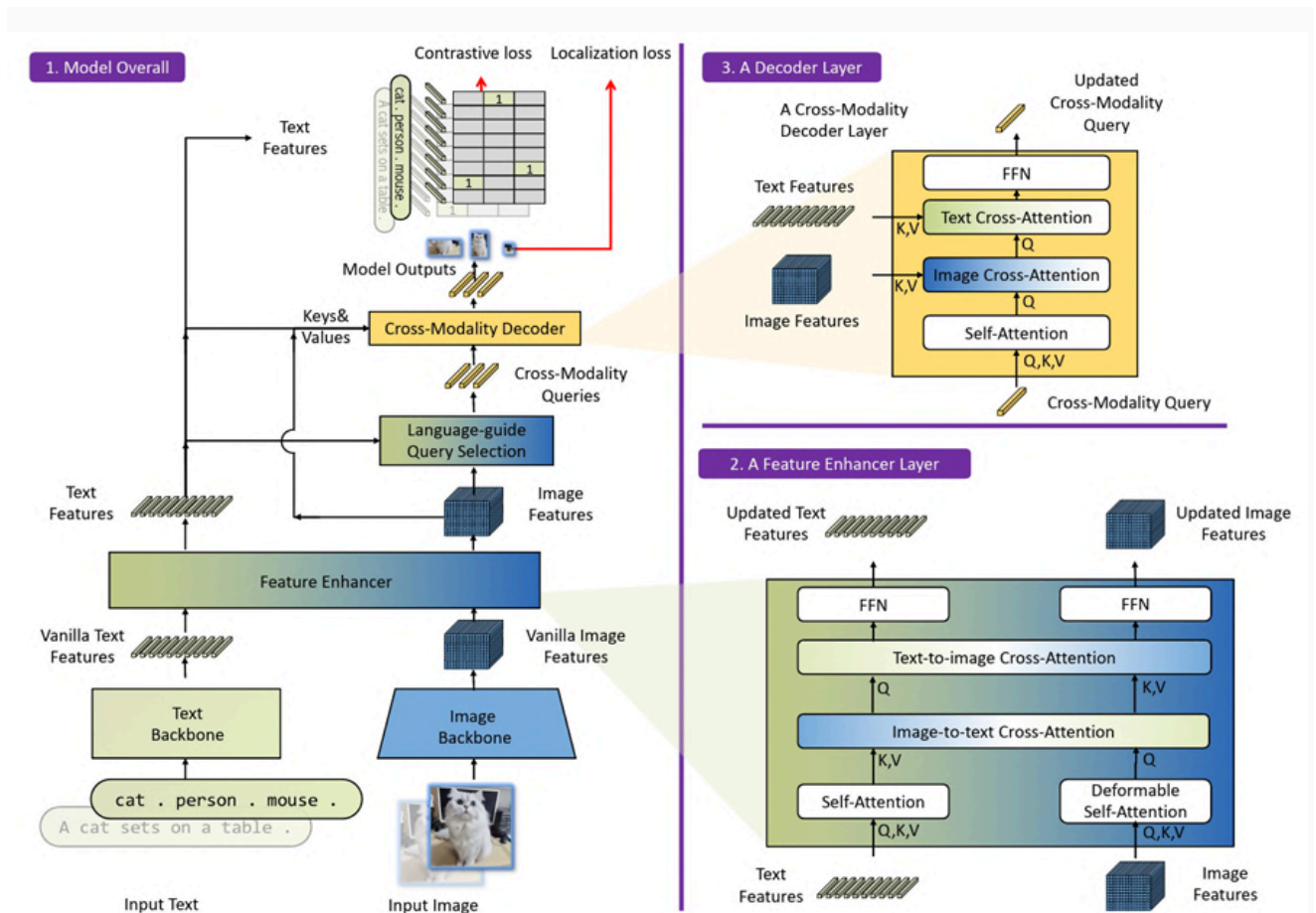
(b) 开放集检测 (Open-set)：可以检测训练集中未出现的类别（通过文本描述，如 "brown bear" 或 "red hat"）。

Grounding DINO = DINO + Grounding。

DINO: DETR with Improved DeNoising Anchor Boxes (带有改进去噪锚框的 DETR)。

Grounding: 对齐文本和图像 (Aligning text and image)。

它是一个 **DETR 风格的检测器 + CLIP 风格的文本接地 (Text Grounding)**。



特征提取与增强 (Feature Extraction and Enhancer)： 图像主干：使用 Swin Transformer 提取多尺度图像特征。

文本主干：使用 BERT 提取文本特征。

特征增强器 (Feature Enhancer)：

将图像和文本特征输入进行**跨模态特征融合 (Cross-modality feature fusion)**。

包含多个特征增强层。

利用**可变形自注意力 (Deformable self-attention)** 增强图像特征。

利用**自注意力 (Self-attention)** 增强文本特征。

执行 **图像到文本 (Image-to-Text)** 和 **文本到图像 (Text-to-Image)** 的交叉注意力 (Cross-Attention) 进行融合。

语言引导的查询选择 (Language-Guided Query Selection)：

选择与输入文本更相关的物体作为解码器查询 (Decoder Queries)。

跨模态解码器 (Cross-Modality Decoder)：

结合图像和文本模态特征。

每个查询馈送到：自注意力层 -> 图像交叉注意力层 -> 文本交叉注意力层 -> FFN 层。
每个解码器层都有一个额外的文本交叉注意力层，用于将文本信息注入查询以实现更好的模式对齐。

零样本 (Zero-Shot)：在未见过的类别上表现优异（例如在 COCO 上训练，在 Objects365 上测试，或者反之）。

微调 (Fine-Tuning)：在特定数据集上微调后，性能进一步提升。

对比：Grounding DINO (Ours) 在多个基准测试中优于 Faster R-CNN, DINO, GLIP 等模型。

目标跟踪 (Object Tracking)

目标：在视频的连续帧中跟踪同一个人或物体。

表示方式：物体通常由带有 ID 的边界框 (Bounding Boxes) 表示。

输入：通常物体检测 (Object Detection) 的结果是跟踪算法的输入。

流程：从时间步 t 到 $t + 1$ ，系统需要维持物体的 ID 一致性。

跟踪主要依赖两个重要因素：

1. 外观特征 (Appearance features)：

物体看起来是什么样子的？(How does the object look like?)

2. 运动特征 (Motion features)：

物体的运动情况如何？(What is the movement of objects?)

它们在下一帧中会位于哪里？(Where will they be located in the next frame?)

多目标跟踪的挑战 (Challenges of MOT)

外观相似：多个物体外观非常相似（例如人群中的行人）。

无关物体干扰：背景中存在其他不感兴趣的物体（如消防栓、背景杂物）。

严重遮挡 (Heavy occlusion)：物体之间互相遮挡，导致检测困难。

基于检测的跟踪 (Tracking-by-Detection)

目标 (Goal)：

在单个视频帧中检测物体。

在帧之间关联 (Associate) 检测集合，从而随时间创建独立的物体轨迹。

问题 (Problem)：

检测结果并不总是准确的，存在**假阳性 (False Positive)** 和 **漏检 (Missing Detections)**。

核心步骤 (Key Steps)

1. 检测 (Detection)：

使用检测器在时间步 t 初始化轨迹 (Initialize tracks)。

2. 运动预测 (Motion Prediction)：

使用**运动模型 (Motion Model)** 预测物体在下一帧 ($t + 1$) 的位置。

3. 数据关联 (Data Association)：

在时间步 $t + 1$ ，将**预测的位置**与**新的检测结果**进行匹配 (Match)。

运动模型 (Motion Model)

原理：预测基于物体的运动。

假设：例如，物体以恒定速度移动 (Constant speed)。

常用方法：

卡尔曼滤波 (Kalman Filtering)：最常用的方法，用于估计物体的状态（位置、速度）。

粒子滤波 (Particle filtering)。

数据关联 (Data Association)

卡尔曼滤波匹配：

计算检测框与预测框之间的距离。

距离度量：通常基于 IoU (Intersection over Union) 或像素距离。

匈牙利算法 (Hungarian Algorithm)：

解决一对一的线性分配问题 (Linear Assignment Problem)，找到最优匹配。

Deep SORT (Simple Online and Realtime Tracking) 是对 SORT 算法的改进，引入了深度学习特征。

外观模型 (Appearance Model)：

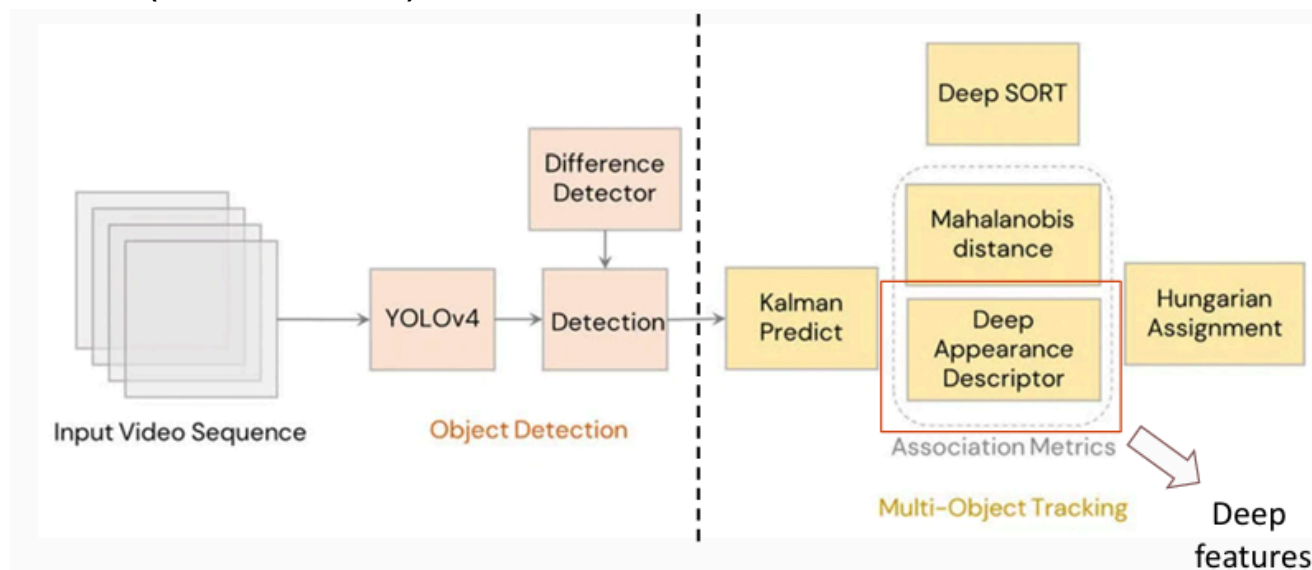
使用 **CNN (卷积神经网络)** 提取深度外观特征 (Deep Appearance Descriptor / Embedding)。

即使物体被遮挡或暂时丢失，也能通过外观特征找回 ID。

关联度量 (Association Metrics)：

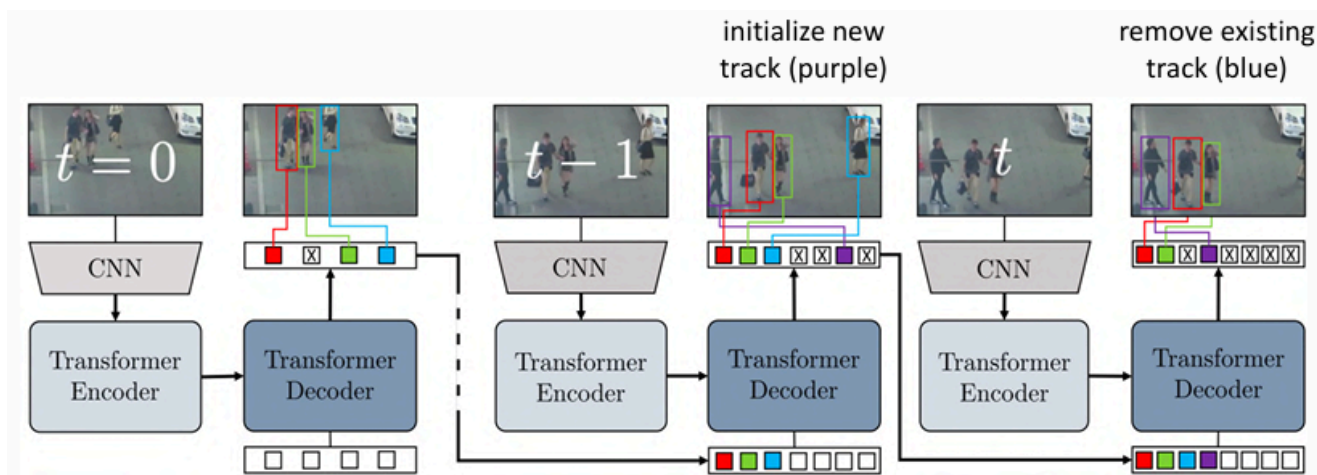
马氏距离 (Mahalanobis Distance)：基于卡尔曼滤波预测的运动信息（判断位置是否合理）。

余弦距离 (Cosine Distance)：基于 CNN 提取的外观特征（判断长得像不像）。



新兴方法

传统的 Tracking-by-Detection 是多步骤的，而 **Trackformer** 尝试使用 Transformer 进行端到端的跟踪。



使用 Transformer 联合执行物体检测和基于注意力的跟踪 (Joint object detection and tracking-by-attention)。

利用**对象查询 (Object queries)** 和 **自回归轨迹查询 (Autoregressive track queries)** 来推理轨迹的初始化、身份和时空轨迹。

编码器 (Encoder): CNN + Transformer Encoder。

解码器 (Decoder): Transformer Decoder + 对象/轨迹查询 (Object / Track queries)。

输出: 边界框和类别预测 (通过 MLP)。

1. 对象查询 (Object queries):

用于检测当前帧中的**新物体**，并为下一帧形成轨迹。

2. 轨迹查询 (Track queries):

通过携带身份信息并在适应位置变化的同时，在视频中**跟随物体**。

将已跟踪的物体与当前帧的边界框关联。

姿态估计 (Pose Estimation)

目标: 从图像或视频中定位人体/面部的**关键点 (Keypoints)**。

动作识别 (Action Recognition): 使用身体关键点来分类人类动作 (例如: 识别“拥抱”动作)。

动画与动作捕捉 (Animation and Motion Capture): 基于人体姿态构建虚拟角色 (例如: 将真人的动作映射到 3D 卡通人物上)。

性能指标 (Performance Metrics)

PCK (Percentage of Correct Keypoint):

测量预测关键点落在归一化距离内的百分比。

例如: 在 FLIC 数据集中, **PCK@0.2** 表示预测点与真实点的距离 $< 0.2 \times$ 躯干直径。

在 MPII 数据集中, **PCKh@0.5** 表示阈值 = 50% 的头部线段长度。

PCP (Percentage of Correct Parts):

测量肢体 (limb) 的定位精度。

如果预测关节与真实关节之间的距离小于肢体长度的一定比例 (如 0.1-0.5), 则视为检测正确。

OKS (Object Keypoint Similarity):

类似于检测任务中的 IoU (Intersection over Union)。

定义：预测点与真实点之间的距离，并根据人体尺度 (scale) 进行归一化。

AP (Average Precision) & AR (Average Recall):

基于 OKS 阈值计算（类似于检测中的 mAP）。

AP: 主要挑战指标 (Primary challenge metric)，通常取 OKS 阈值从 0.50 到 0.95 的平均值。

AP@0.50: 宽松指标 (Loose metric)。

AP@0.75: 严格指标 (Strict metric)。

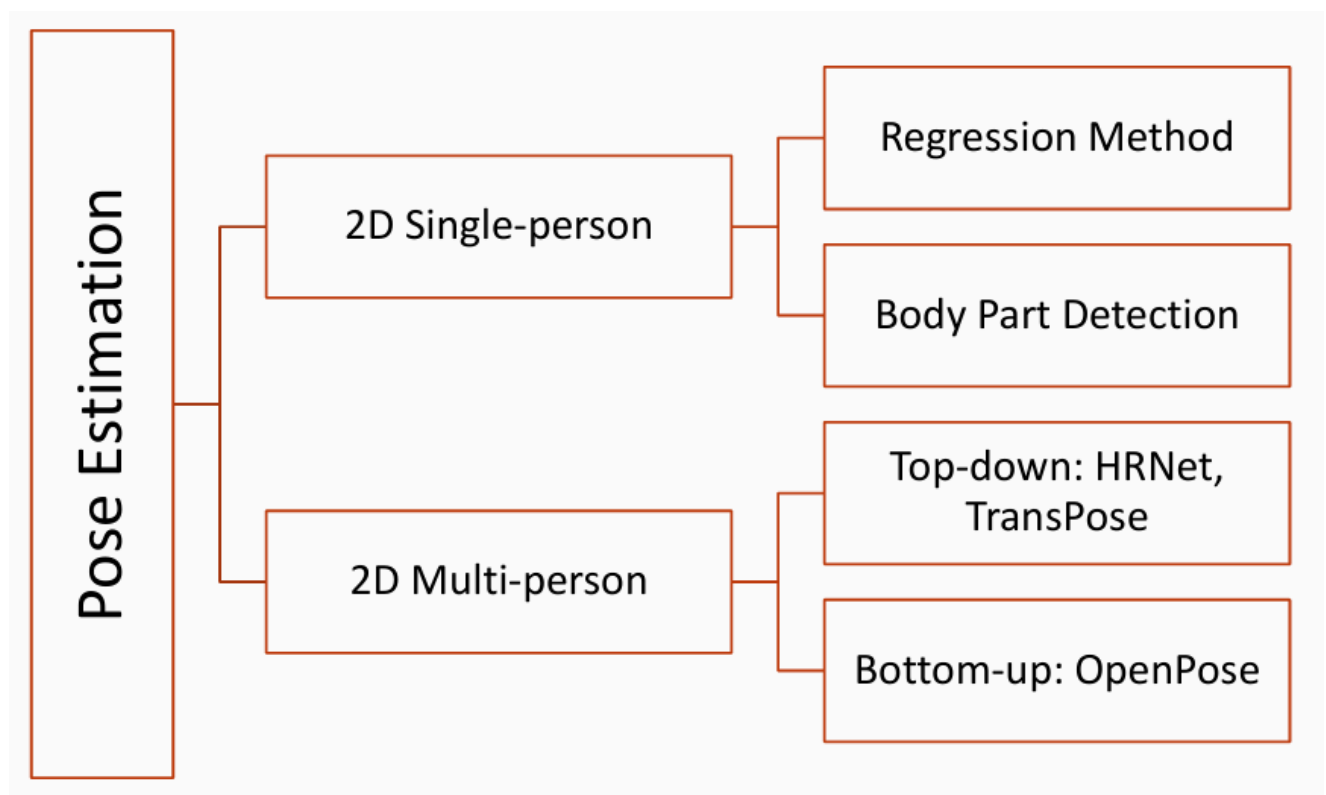
挑战 (Challenges)

计算效率 (Computational efficiency)。

稀有姿态数据有限 (Limited data for rare poses)。

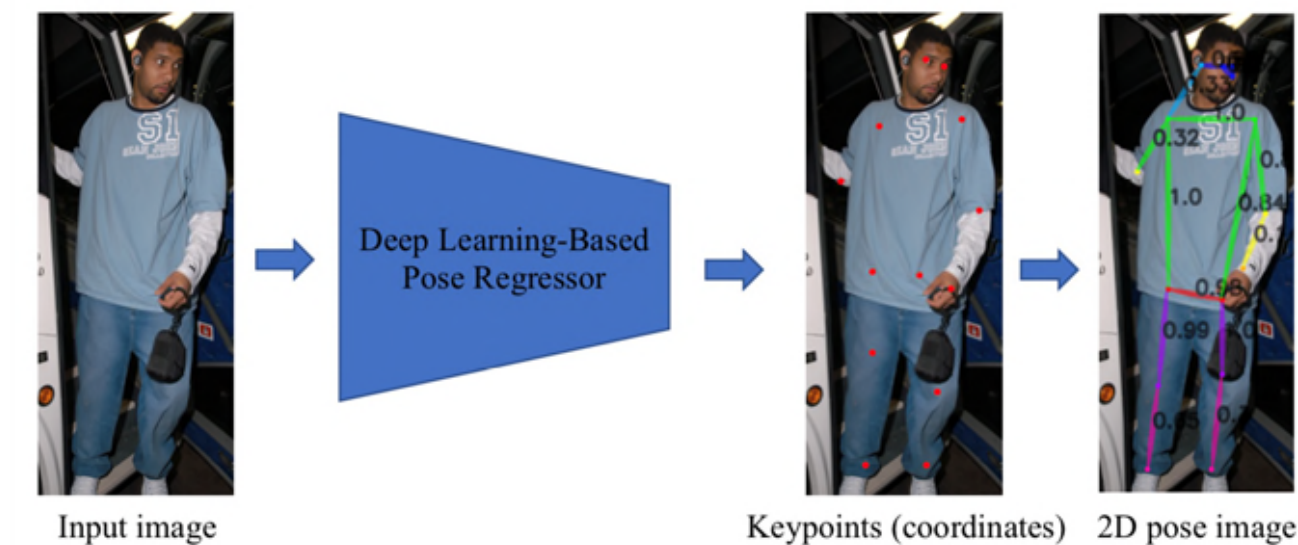
遮挡 (Occlusion)。

分类 (Pose Estimation Methods)



单人姿态估计方法

回归方法 (Regression Method)



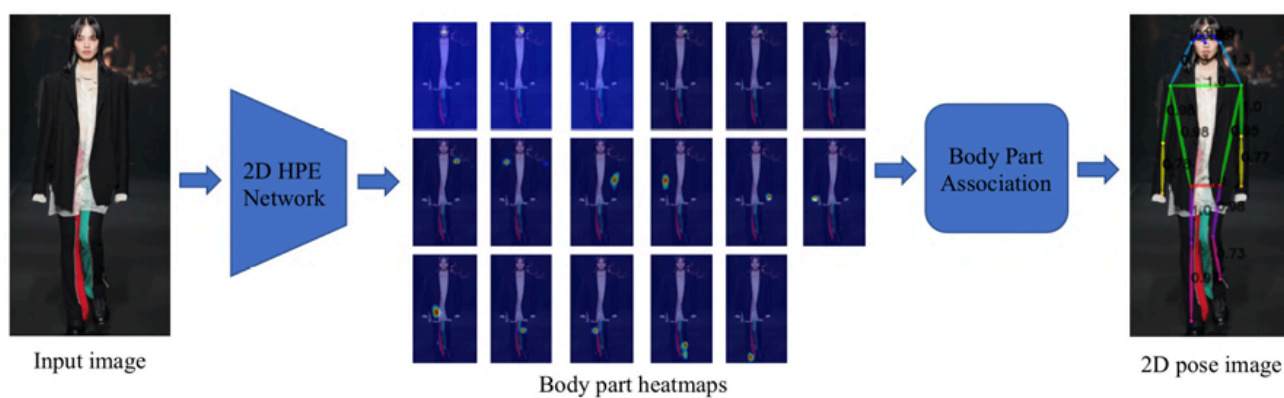
方法:

1. 从 CNN 获取特征图。
2. 从特征图计算回归器 (Regressor) 直接输出坐标。

优点: 简单且快速 (Easy and fast)。

缺点: 精度较低 (Less accurate)

身体部位检测 (Body Part Detection)



方法:

1. 估计 **K** 个热力图 (Heatmaps) 对应 **K** 个关键点。
2. 将检测到的关键点组装成整体身体姿态或骨架。

优点: 精度高 (Accuracy)。

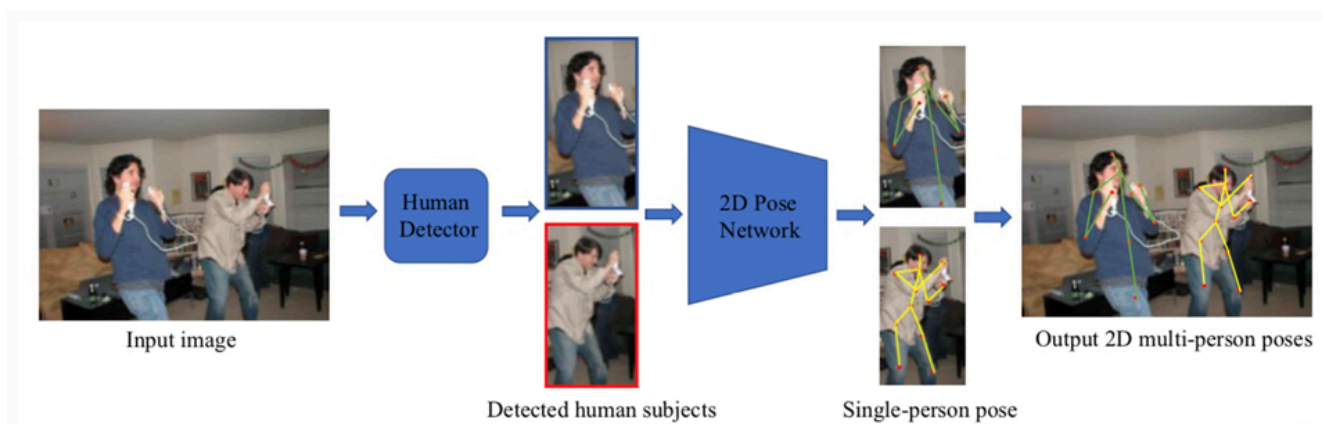
缺点: 较慢且复杂 (Slower and complex)。

代表工作: **Stacked Hourglass Networks** (2016 ECCV)。

通过多次下采样和上采样 (沙漏形状) 来捕捉多尺度特征。

多人姿态估计方法

自顶向下方法 (Top-Down Methods)



流程：

1. 使用现成的检测器 (Off-the-shelf detector) 检测图像中的人 (Bounding Box)。
2. 对每个单人裁剪区域估计姿态。

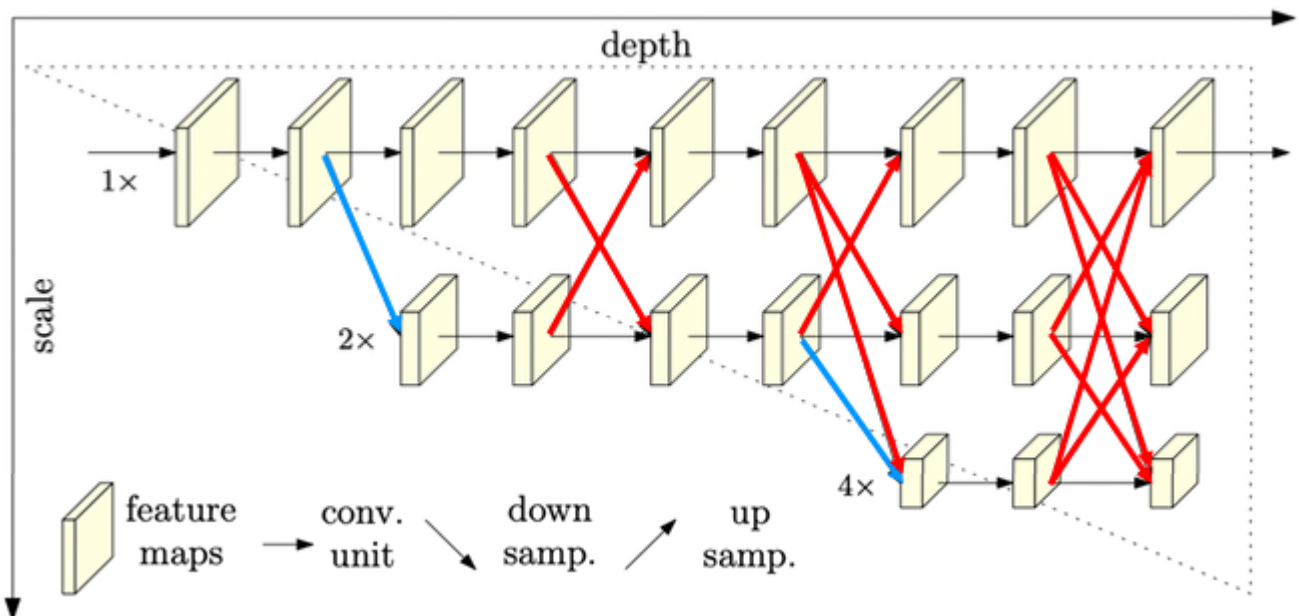
代表工作：HRNet (High-Resolution Net)。

优点：损失网络复杂度，精度高 (Less network complexity, accurate)。

缺点：运行时间随着检测到的人数增加而增加 (Runtime increases with # of persons)。

适用场景：人数较少的场景。

HRNet (高分辨率网络) 架构



核心思想：始终保持高分辨率表示。

结构特点：

1. 从高分辨率子网开始，逐渐并行加入高分辨率到低分辨率的子网。
2. 并行连接多分辨率子网 (Connect multi-resolution subnets in parallel)。
3. 多尺度融合 (Multi-scale fusions)：在并行子网之间反复交换信息 (Repeatedly exchange information across parallel multi-resolution subnets)。

自底向上方法 (Bottom-up Methods)



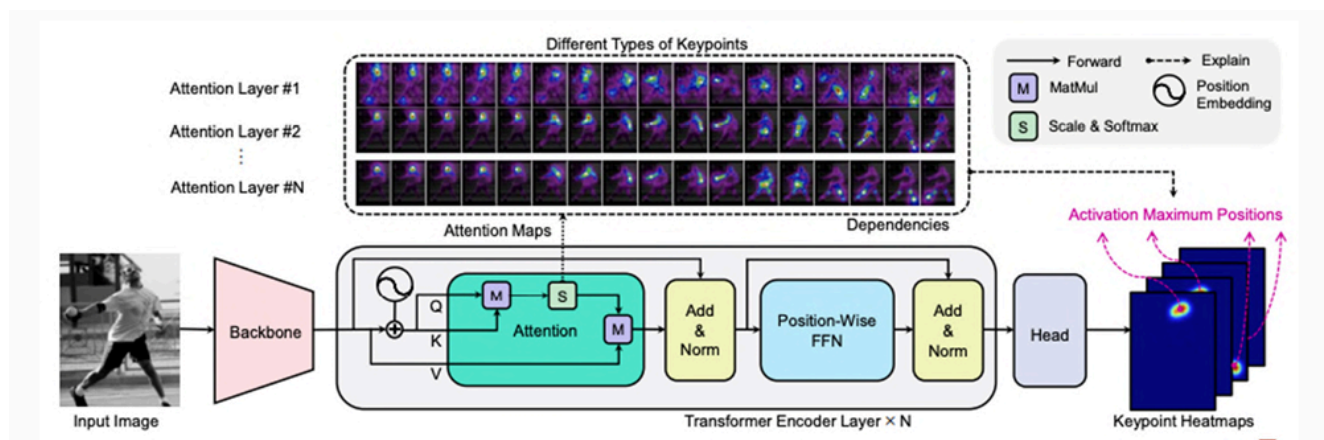
流程：检测图像中所有的关键点，然后通过分组算法 (Grouping/Association) 将属于同一个人的关键点聚合起来。

优点：运行时间通常与人数量无关，适合拥挤场景。

缺点：后处理复杂，精度通常略低于 Top-Down。

新兴方法 (Emerging / New Methods)

TransPose



这是一种基于 **Transformer** 的姿态估计方法，结合了 CNN 和 Transformer 的优势。

Backbone (主干网络)：使用常见的 CNN（如 ResNet 和 HRNet），但仅保留初始的几层用于提取特征。

Transformer：仅使用 **Encoder (编码器)**。包含 N 个注意力层 (attention layers) 和前馈网络 (FFN)。

Head (头部)：使用卷积层来预测关键点的热力图 (keypoint heatmaps)。

3D姿态估计 (3D Pose Estimation)

2D 姿态估计的局限性 (Limitations of 2D Pose Estimation)

缺乏深度信息 (Lack of depth information)：

2D 姿态仅提供 (x, y) 坐标。

深度模糊性导致对现实世界空间排列的理解较差。

没有 3D 场景理解能力。

遮挡模糊性 (Occlusion ambiguity)：

在没有深度信息的情况下，很难区分重叠或互相遮挡的身体部位。

不足以应对现实应用 (Inadequate for real-world applications)：

2D 输出不足以满足机器人 (Robotics)、AR/VR、动作捕捉 (motion capture) 等应用的需求。

挑战 (Challenges in 3D Pose Estimation)

1. **深度模糊 (Depth Ambiguity)**: 很难从单目 (monocular) 2D 视图中推断深度。
2. **遮挡 (Occlusion)**: 身体部位可能被其他肢体、物体或人遮挡。
3. **外观变化 (Variability in Appearance)**: 服装、体型、光照和相机视角的差异。
4. **复杂姿态与交互 (Complex Poses & Interactions)**: 复杂的身体关节活动、多人交互难以建模。
5. **数据限制 (Data Limitations)**: 标注好的 3D 数据集稀缺且昂贵。
6. **跨域泛化 (Generalization Across Domains)**: 在受控数据集上训练的模型在野外 (in-the-wild) 条件下表现往往较差。
7. **计算复杂度 (Computational Complexity)**: 高维输出 (许多关节的 3D 坐标) 需要在准确性和实时性之间取得平衡。
8. **多视角 vs. 单目权衡 (Multi-view vs. Monocular Trade-off)**:
多视角设置能提供更可靠的深度线索, 但在许多现实场景中不切实际。
单目设置实用但模糊性更大。

主要的 3D 人体姿态估计方法 (Main 3D Human Pose Estimation Methods)

直接 3D 姿态估计 (Direct 3D Pose Estimation / End-to-End)

原理: 模型直接从输入图像或视频预测 3D 关节坐标。

优点: 流程简单, 没有中间步骤。

缺点: 由于深度模糊和有限的 3D 数据, 训练难度较大。

示例: 端到端 CNNs, 用于 3D 关节回归的 Transformers。

两阶段方法 (Two-Stage / 2D-to-3D Methods)

原理:

Stage 1: 在图像中估计 2D 关键点。

Stage 2: 使用回归模型将 2D 关键点“提升” (Lift) 到 3D 空间。

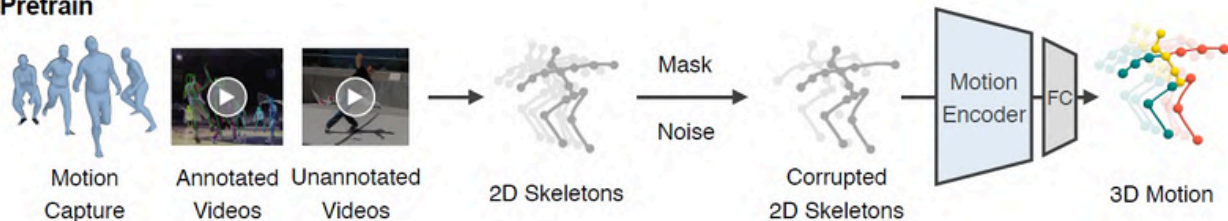
优点: 利用了大型 2D 姿态数据集; 模块化设计。

缺点: 2D 估计中的误差会传播到 3D 阶段。

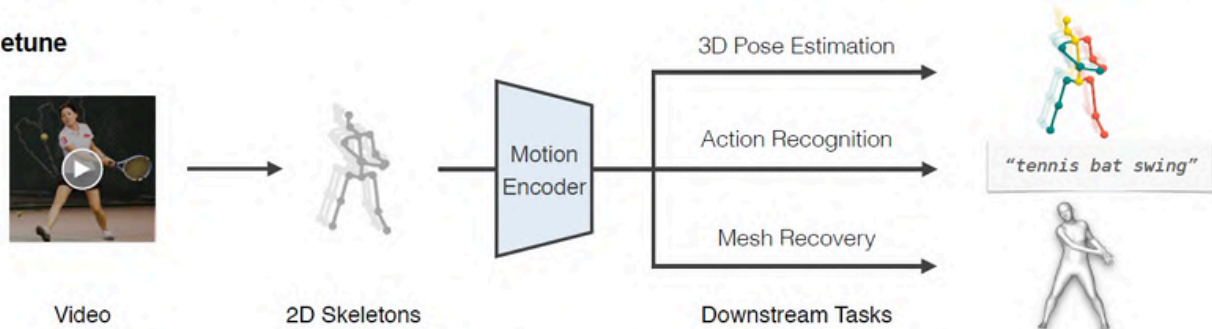
示例: 2D 姿态检测器 (如 OpenPose) + 3D 提升网络 (如 **MotionBERT**)。

MotionBERT: 学习人体运动表示的统一视角

I. Pretrain



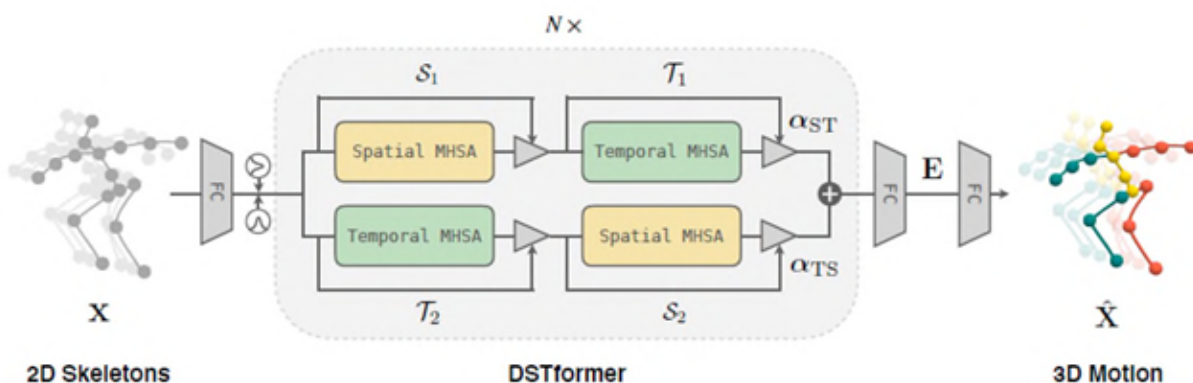
II. Finetune



地位：3D 人体姿态估计的领先方法 (Leading method)。

两个阶段 (Two stages):

1. **预训练 (Pretraining)**: 预训练一个运动编码器 (motion encoder) 以完成 **2D 到 3D 的提升 (2D-3D lifting)**。即从 2D 姿态序列恢复 3D 人体运动表示。
2. **微调 (Finetuning)**: 将预训练好的运动表示进行微调，以适应不同的下游任务（如动作识别、动作生成等）。



核心组件：双流时空 Transformer (Dual-stream Spatio-temporal Transformer, **DSTformer**)。

组成模块：

空间 MHSA 模块 (Spatial MHSA Block): 对同一时间步 (same time step) 内关节之间的关系进行建模。

时间 MHSA 模块 (Temporal MHSA Block): 对身体关节跨时间步 (across time steps) 的关系进行建模。

融合机制: DSTformer 在流程中融合了**空间 (spatial)** 和 **时间 (temporal)** 信息。

动作识别 (Action Recognition)

识别视频中的人类动作。

示例动作：握手 (Shaking hands)、板球投球 (Cricket bowling)、滑板 (Skate boarding)、切菜 (Cutting in kitchen)、拉伸 (Stretching leg)、骑车 (Riding a bike)、拉小提琴 (Playing violin)、运球 (Dribbling basketball)。

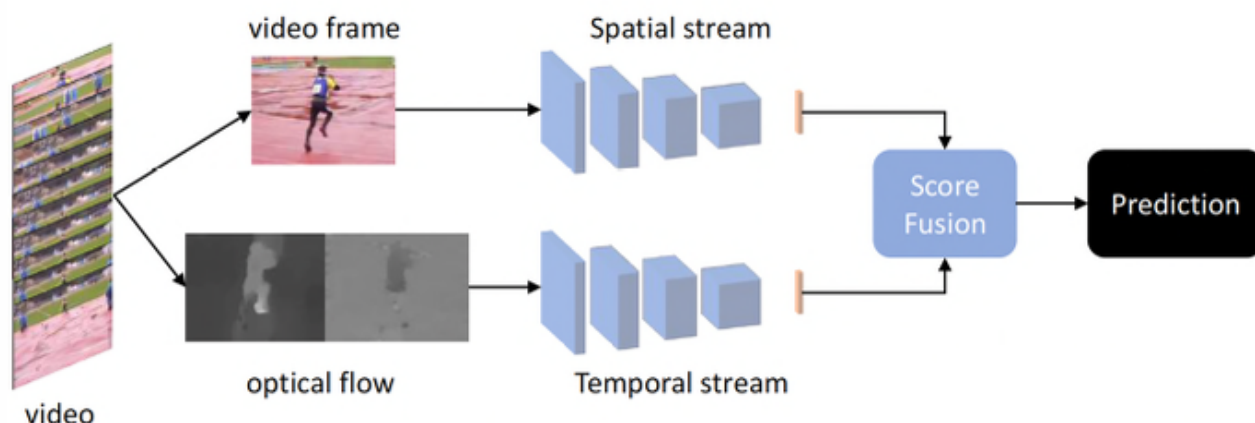
监控 (Monitoring): 例如驾驶员行为监控 (打电话、疲劳驾驶)。

控制 (Control): 通过手势控制设备 (如电视、游戏)。

危险检测 (Danger detection): 例如检测跌倒 (Fall detection)。

事件分析 (Event analysis): 例如零售店中的行为分析 (偷窃检测)。

双流网络 (Two-Stream Networks)



组成: 包含一个空间流 (Spatial Stream)、一个时间流 (Temporal Stream) 和一个分数融合模块 (Score Fusion Module)。

空间流: 从采样的 RGB 帧中捕获外观信息 (Appearance information)。

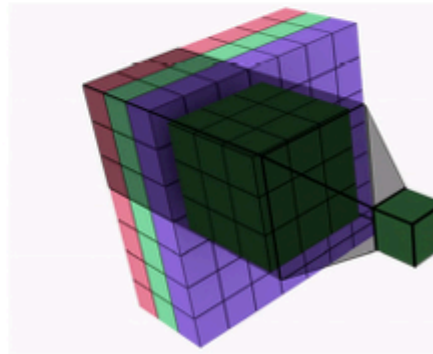
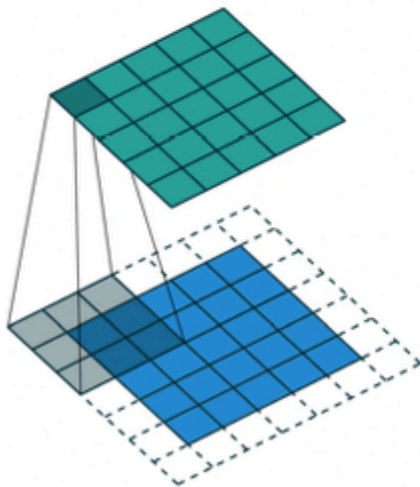
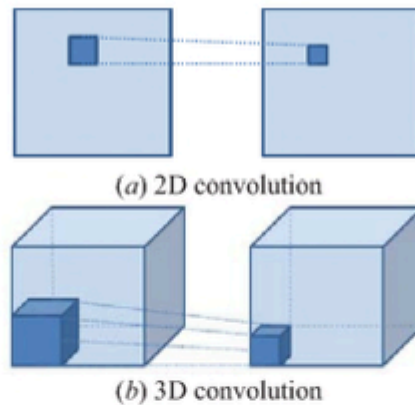
时间流: 从光流 (Optical Flow) 中识别运动信息。

融合模块: 结合来自空间流和时间流的分数。

Pros: 网络结构简单; 光流是表示运动信息的准确方式。

Cons: 光流的提取和存储成本高昂。

3D 卷积神经网络 (3D CNNs)



定义：将视频视为具有两个空间维度（高、宽）和一个时间维度的 **3D 张量 (3D tensor)**。

对比 2D CNN：

2D 卷积：输入体积与滤波器之间的点积（忽略深度/时间维度，仅在 2D 矩阵上操作）。

3D 卷积：在 3D 张量上进行点积操作。

代表工作：SlowFast Network。

Pros：特征提取器可以同时获取时间和空间信息。

Cons：由于模型庞大且推理效率低，难以训练和部署。

高效视频建模 (Efficient Video Modelling)

同时提高视频动作识别的准确性和效率。

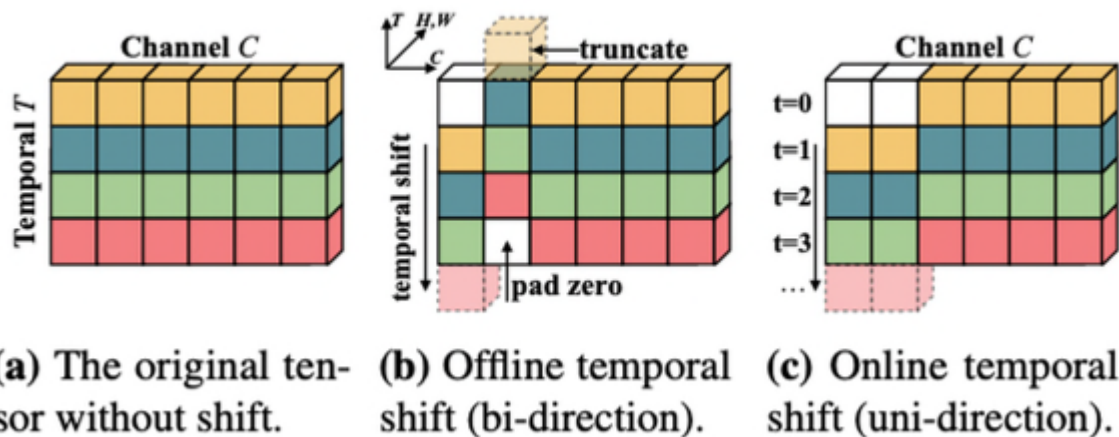
不使用光流即可编码运动信息。

无需 3D 卷积即可进行时间建模。

时间移位模块 (Temporal Shift Module, TSM)

移位操作 (Shifting Operation)：

沿时间维度移动部分通道 (channels)，从而促进相邻帧之间的信息交换。



两种移位选项：

1. **双向移位 (Bi-direction)**：用于离线设置 (Offline settings)。
2. **单向移位 (Uni-direction)**：用于在线设置 (Online settings)。

优缺点：

Pros：快速且高效。

Cons：准确性不如 3D CNNs。

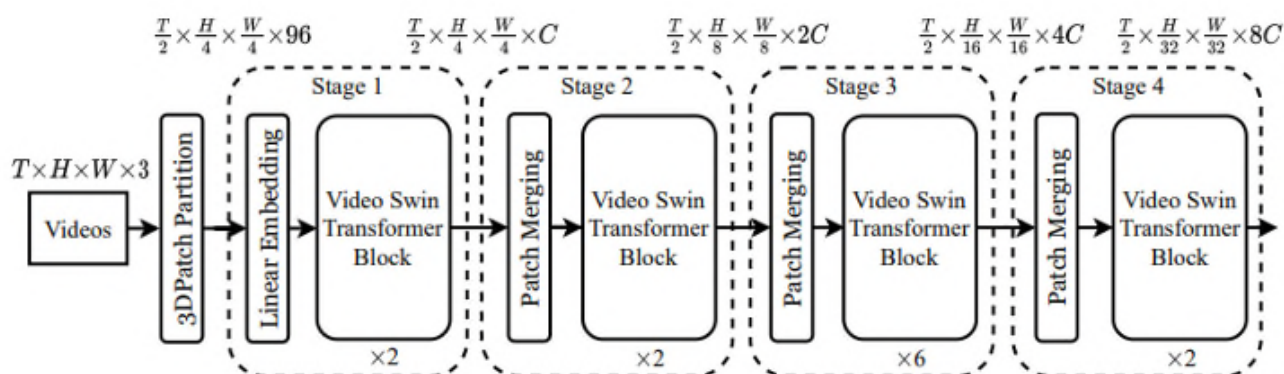
新兴方法：基于 Transformer 的方法

现有方法（如 CNN）通常仅编码短程依赖 (Short-range dependencies)。

许多应用（如动作识别）需要**长程依赖建模 (Long-range dependency modelling)**。

代表工作：Video Swin Transformer。

Video Swin Transformer



背景：基于 ViT (Vision Transformer) 的方法需要巨大的计算资源，因为自注意力模块考虑了所有 tokens。

目标：降低模型复杂度，同时保持捕捉长程依赖的能力。

1. **3D Patch 划分 (3D Patch Partition)**：

输入视频尺寸： $T \times H \times W \times 3$ 。

3D token / patch 尺寸： $2 \times 4 \times 4 \times 3$ 。

Token 数量： $T/2 \times H/4 \times W/4$ 。

使用线性层将 3D tokens 投影到通道维度。

2. Video Swin Transformer Block:

Layer l : 将 3D tokens 划分为大小为 $P \times M \times M$ 的窗口 (windows), 并在每个局部窗口内执行自注意力 (Self-attention)。

Layer $l + 1$: 以一定步长 ($P/2 \times M/2 \times M/2$) **移位 (Shift)** 窗口, 将 tokens 组织成新的窗口, 并在每个移位窗口内计算自注意力。这实现了跨窗口的连接。

3. Patch Merging (图块合并):

执行 2x 空间下采样, 并拼接每个 2x2 空间邻域的特征。

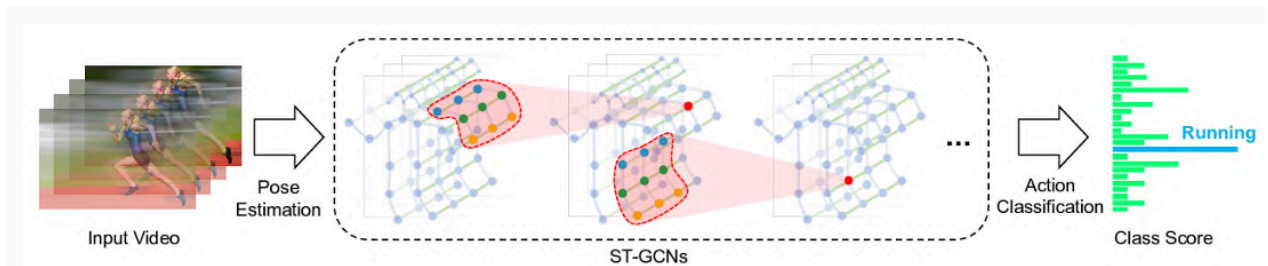
不降低时间维度。

应用线性层投影拼接后的特征。

基于骨架的网络 (Skeleton-based Networks)

利用时间姿态信息 (Temporal pose information) 来预测动作。

1. 从视频中提取姿态信息 (Pose information)。
2. 使用技术 (如图神经网络 Graph Neural Network, GNN) 对动作进行分类。



基础模型 (Foundation Models, FMs)

定义: 一种基于大规模预训练 (large-scale pre-training) 和下游适配 (downstream adaptation) 的人工智能新范式。

预训练 (Pretraining):

模型在大规模多样化数据上进行训练。

通常使用**自监督学习 (self-supervised learning)**, 以减轻昂贵的数据收集和标注成本。

适配/微调 (Adaptation / fine-tuning):

预训练好的模型被适配用于随后的下游任务。

基础模型的核心理念 (Key Ideas of FMs)

集中化信息: 基础模型可以将来自各种模态 (文本、图像、语音、结构化数据、3D信号等) 的所有数据信息集中起来。

广泛适配: 这同一个模型随后可以被适配到广泛的下游任务中 (如问答、情感分析、信息提取、图像描述、物体识别、指令跟随等)。

阶段

基础模型的生命周期是一个更广泛生态系统的一部分, 从数据创建延伸到部署:

1. **数据创建 (Data Creation)**
2. **数据策展 (Data Curation)**

3. 训练 (Training)
4. 适配 (Adaptation)
5. 部署 (Deployment)

领域应用

语言 (Language)

语言习得 (Language Acquisition):

人类与基础模型在语言习得上存在不同的归纳偏置 (inductive biases)。

人类与物理和社会世界互动，拥有多样的需求和欲望。

基础模型主要观察和建模由他人产生的数据。

视觉 (Vision)

潜力：通过在大规模上利用自监督，基础模型有潜力将原始的、多模态的感官信息提炼为视觉知识。

能力：

有效支持传统的感知任务。

可能在具有挑战性的高阶技能上取得进展，如时间推理 (temporal reasoning) 和常识推理 (commonsense reasoning)。

数据来源：感知源、相机和传感器、环境代理、机器人传感器。

机器人 (Robotics)

需求：构建新型机器人基础模型需要跨越不同环境和行为的大规模数据集。

数据源：机器人交互、人类视频、模拟 (Simulation)、自然语言。

潜力：在任务规范和机器人学习的问题表述方面具有巨大潜力。

任务：直观的多模态任务规范、任务学习的快速适配。

基础模型的现状 (Current States of FMs)

趋势：正朝着开源 (open-sourced) 方向发展。

闭源模型优于开源模型 (Closed-source models outperform open-sourced models)。

行业主导基础模型 (Industry dominates Foundation Models)。

训练基础模型的成本极高。

在许多智力任务类别（如图像分类、视觉推理、机器翻译等）上，AI 已经**超越 (exceeded)**了人类的表现。

全球观点：2023年人们表示 AI 让他们感到紧张。

微调 (Fine-tuning)

定义：针对特定下游任务/用例，调整预训练模型参数的过程。

背景：

预训练模型拥有广泛多样的知识，但在特定领域缺乏专业化。

微调通过让模型从领域特定数据中学习来弥补这一差距，使其在目标应用中更准确、更有效。

流程：原始文本数据 (Raw Text Data) → 预训练 (Pre-training) → 预训练大语言模型 (Pre-trained LLM) → 微调 (Fine-tuning) → 微调后的大语言模型 (Fine-tuned LLM)。

过程 (Fine-tuning Process)

- 数据准备 (Data preparation):**
数据清洗、处理缺失值、格式化文本以匹配模型输入要求、数据增强等。
- 预训练模型选择 (Pre-trained model selection):**
理解模型的架构、输入/输出规格。
考虑模型大小、训练数据、性能等因素。
- 微调方法选择 (Method selection for fine-tuning):**
为特定应用选择合适的微调或适配方法。
- 验证 (Validation):**
在目标验证集上评估微调后模型的性能。
- 模型迭代 (Model iteration):**
如有必要，重复上述过程。
- 部署 (Deployment):**
将微调后的模型部署到现实世界的目标应用中。

适配器微调 (Adaptor Tuning)

定义：一种微调大型预训练模型的技术。

原理：引入**适配器 (adaptors)**，即小型的网络层。

训练策略：仅训练**适配器**，冻结预训练模型的原始权重。

优点：适用于计算资源有限或标注数据有限的情况。显著减少了需要更新的参数数量，提高了计算和内存效率。

在预训练模型的层之间引入小型轻量级适配器（网络）。

微调期间，仅训练适配器，基础模型的原始权重（参数）保持冻结。

常见架构包括：Prefix-Tuning, LoRA, Series Adapter, Parallel Adapter。

低秩适配 (Low-Rank Adaptation, LoRA)

定义：一种针对基础模型（如大型语言模型）的高效微调方法。

优势：

计算开销和参数更新量低。

例如：GPT-3 (175B) 使用 LoRA 微调，可训练参数减少 **10,000倍**，GPU 内存需求减少 **3倍**。

- 选择预训练基础模型 (Select a pre-trained FM)。**
- 确定目标层进行适配 (Identify target layers for adaptation):**
常见选择层：Transformer 中的注意力层 (attention layer)。
- 引入低秩矩阵进行低秩分解 (Introduce low-rank matrices for low-rank decomposition):**
对于每个选定的层，引入两个低秩矩阵 A 和 B ，以近似原始权重矩阵 W 的变化。
公式： $W' = W + A \times B$

- 4. 微调模型 (Fine-tune the model):
冻结原始权重矩阵 W 。
仅更新低秩矩阵 A 和 B ，使用特定用例数据。
- 5. 评估适配后的模型性能 (Evaluate the adapted model performance)。
- 6. 优化 (Optimization): 超参数调整、模型评估。
- 7. 部署与监控 (Deploy and Monitor)。

LLM

定义：旨在理解、生成和处理人类语言的 AI 模型。

关键特征 (Key Features):

大规模 (Massive Scale): 数十亿参数，使其能够处理各种语言任务。

训练数据 (Training Data): 在各种来源的大量文本数据上训练，学习人类语言的不同方面。

能力 (Capabilities): 处理语言翻译、问答、文本生成、情感分析等任务。

应用 (Applications): 自然语言处理、聊天机器人、内容创作、语言理解等。

挑战 (Challenges): 偏见、滥用、社会影响等。

LLM 架构分类 (LLM Architectures)

架构类型	代表模型	预训练任务	适用场景
Encoder-only (仅编码器)	BERT	MLM, NSP	分类任务 (Classification tasks)
Decoder-only (仅解码器)	GPT	Next word / token prediction	生成任务 (Generation tasks) 预训练后泛化能力更好
Encoder-Decoder (编码器-解码器)	T5 (Text-to-Text Transfer Transformer)	提供文本作为输入并训练其生成目标文本	机器翻译 (Machine translation) 等文本生成任务

BERT (Bidirectional Encoder Representations from Transformers)

开发者/时间：Google 于 2018 年左右开发。

架构：带有双向自注意力 (bidirectional self-attention) 的 Transformer 编码器 (Encoder)。

核心能力：

理解单词之间的双向上下文关系。

生成上下文相关的词嵌入 (contextualized word embeddings)。

第一阶段：预训练

BERT 的输入嵌入 (Embedding) 是以下三种嵌入的总和：

Token Embedding (词元嵌入): 使用 WordPieces 分词，词汇表大小为 30k。

Segment Embedding (段嵌入): 用于区分句子编号（例如句子 A 和句子 B）。

Position Embedding (位置嵌入): 捕捉单词在句子中的位置信息。

BERT 使用无监督/自监督学习来理解语言结构和上下文。主要包含两个关键任务：

1. 掩码语言模型 (Masked Language Model, MLM):

从周围单词中预测被掩码 (Masked) 的单词。

旨在实现上下文学习。

2. 下一句预测 (Next Sentence Prediction, NSP):

预测文本对中的第二个句子是否紧随第一个句子之后。

学习句子之间的逻辑上下文。

预训练语料库:

英文维基百科 (English Wikipedia) - 25 亿单词。

图书语料库 (Book Corpus) - 8 亿单词。

第二阶段：针对特定任务的微调

方法: 使用有监督学习 (Supervised learning)。在最后一个编码器层之后添加一个**特定任务的模块 (task-specific module)**，以便映射到目标任务。

示例:

问答任务 (Question Answering, QA): 训练两个额外的向量来标记段落中答案的开始和结束位置。

其他下游任务: 情感分析 (Sentiment analysis)、文本摘要 (Text summarization) 等。

性能评估 (Performance Evaluation)

GLUE (General Language Understanding Evaluation):

包含句子对任务 (Sentence pair task) 和单句分类任务 (Single sentence classification task)。

SQuAD (Stanford Question Answering Dataset):

用于评估问答能力。

结果: BERT 在许多任务特定模型 (task-specific models) 上表现优异 (Outperform)。

GPT (Generative Pre-trained Transformer)

定义: 一种在海量文本上预训练的 AI 模型，旨在学习生成类人文本。

核心机制: 利用自监督学习 (Self-supervised learning) 使用大量未标记数据预训练 LLM。

架构: Transformer 解码器 (Transformer Decoder)。

预训练任务: 根据前面的 token 预测下一个 token (Next token prediction)。

微调: 使用特定任务的提示文本 (Task-specific prompt text) 供模型学习。

应用: 文本生成、翻译、对话等。

GPT-4o (Omni)

特点:

更自然的人机交互。

多模态输入/输出: 接受文本、音频、图像和视频的任意组合作为输入，并生成相应的组合输出。

端到端模型: 跨文本、语音和音频的单一端到端模型。

速度快: 响应时间接近人类对话。

多语言: 改进了非英语语言的处理。

视觉与音频: 相比现有模型，在视觉和音频理解方面表现更好。

新兴趋势 (Emerging Trends)

视觉语言模型 (Vision Language Models, VLMs)

结合视觉和语言的多模态 AI 与推理 (Multimodal AI & Reasoning)。

实现图像理解 (Image Understanding) 和连贯文本生成 (Coherent Text)。

代理智能 (Agentic AI)

AI 代理 (Agents) 能够自主执行任务、使用工具并进行规划。

具身智能 (Robotic Embodied AI)

将 AI 模型应用于机器人，使其能够在物理世界中感知和交互 (例如 NVIDIA 的 Jim Fan 展示的项目)。